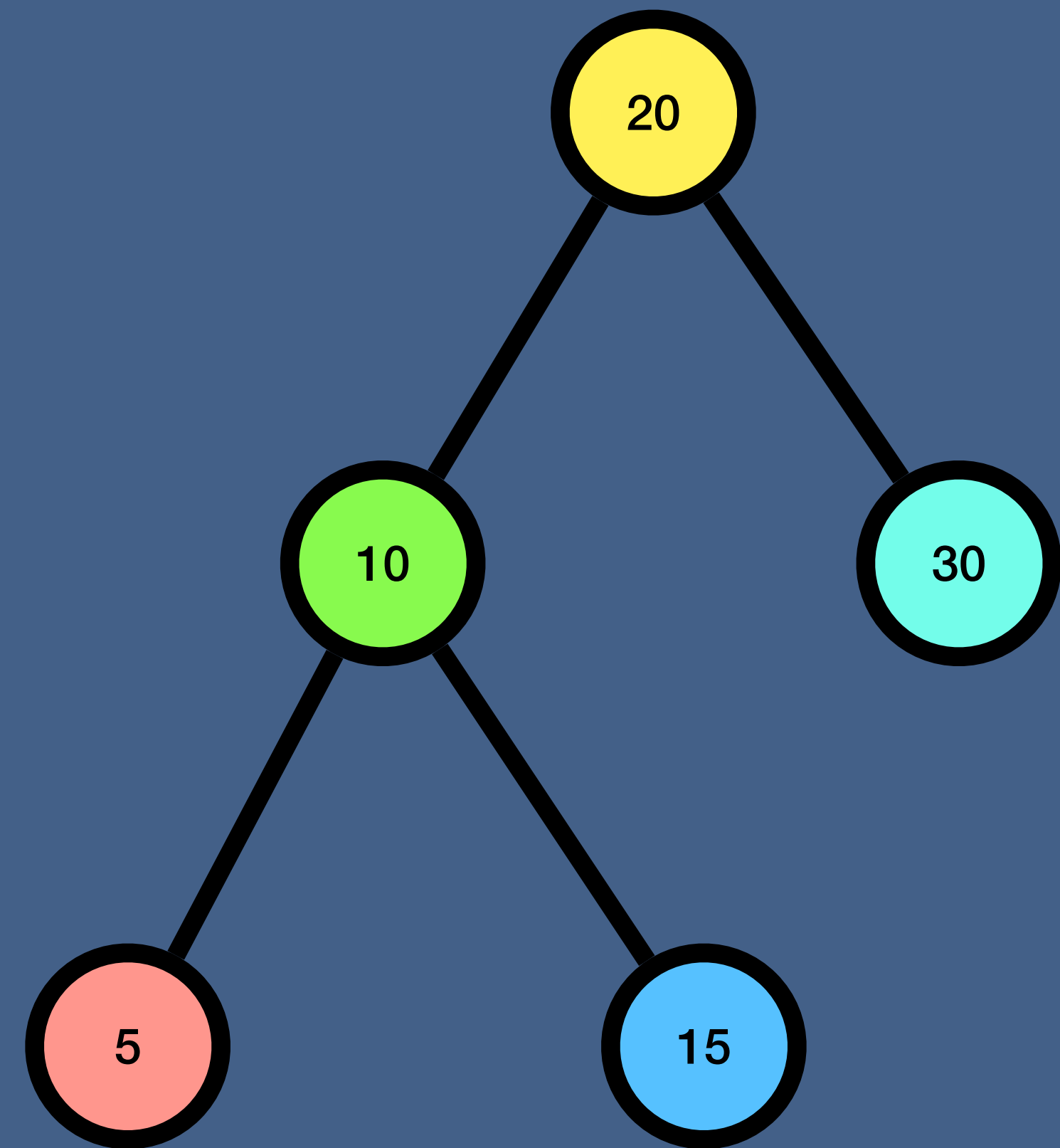
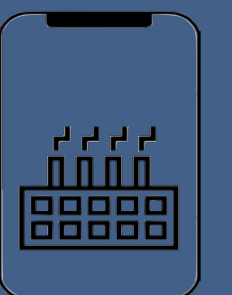
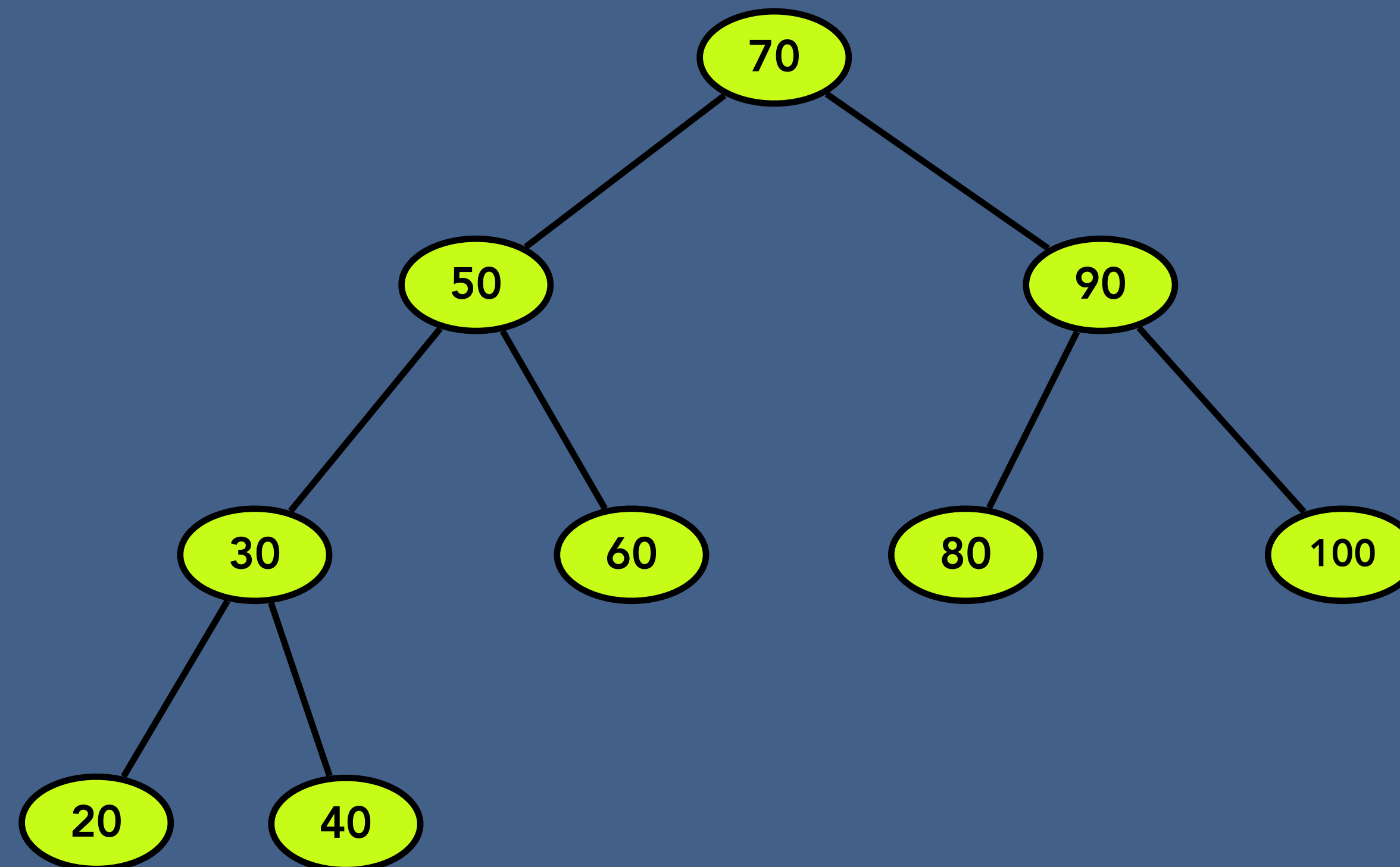


AVL Tree



What is an AVL Tree?

An AVL tree is a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees cannot be more than one for all nodes.



What is an AVL Tree?

An AVL tree is a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees cannot be more than one for all nodes.

Height of leftSubtree = 3

Height of rightSubtree = 2

difference = 1

Height of leftSubtree = 1

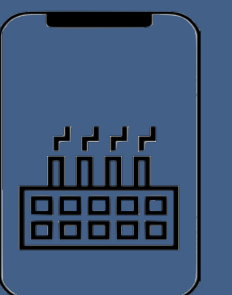
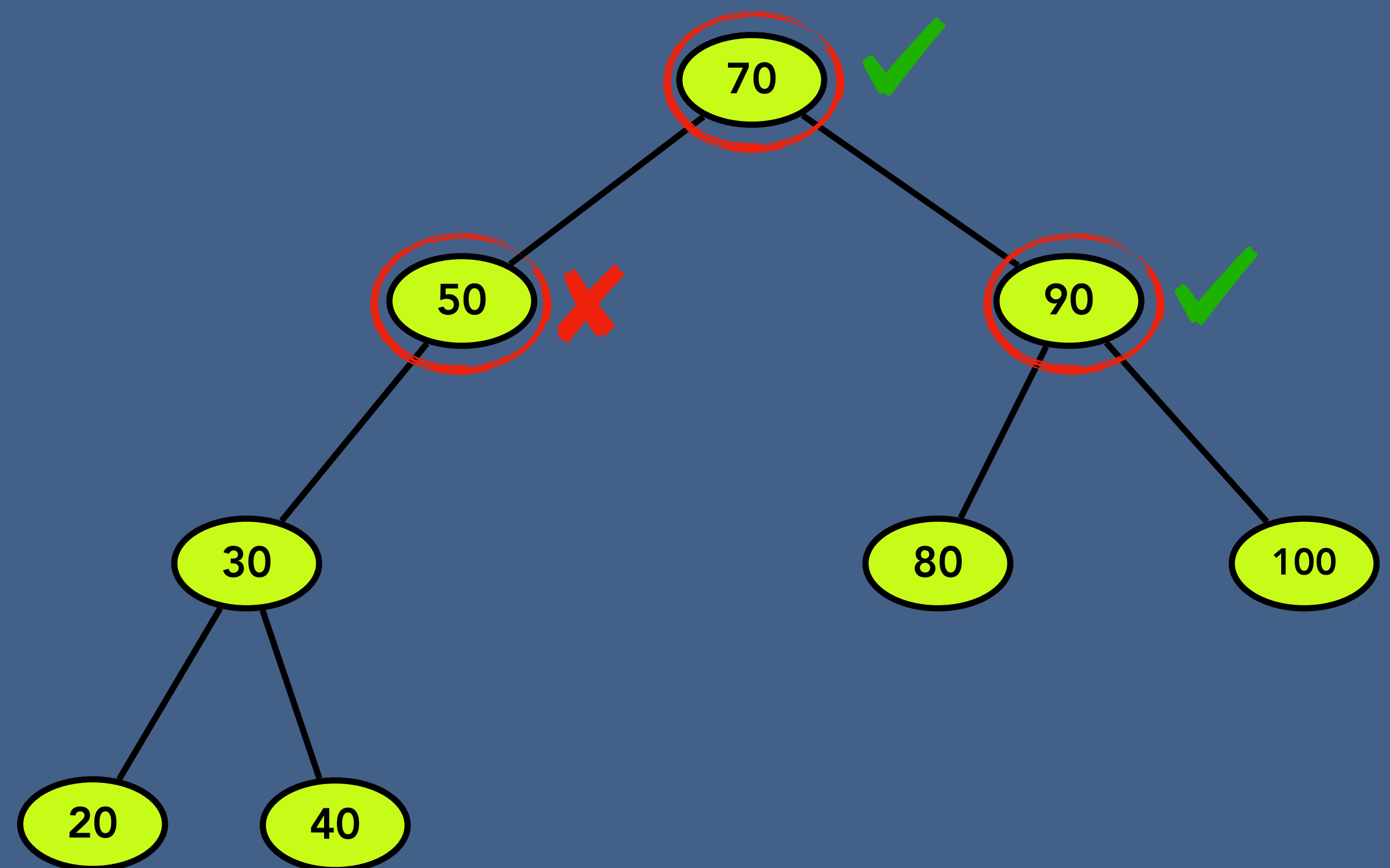
Height of rightSubtree = 1

difference = 0

Height of leftSubtree = 2

Height of rightSubtree = 0

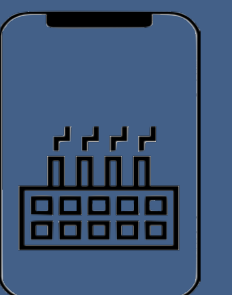
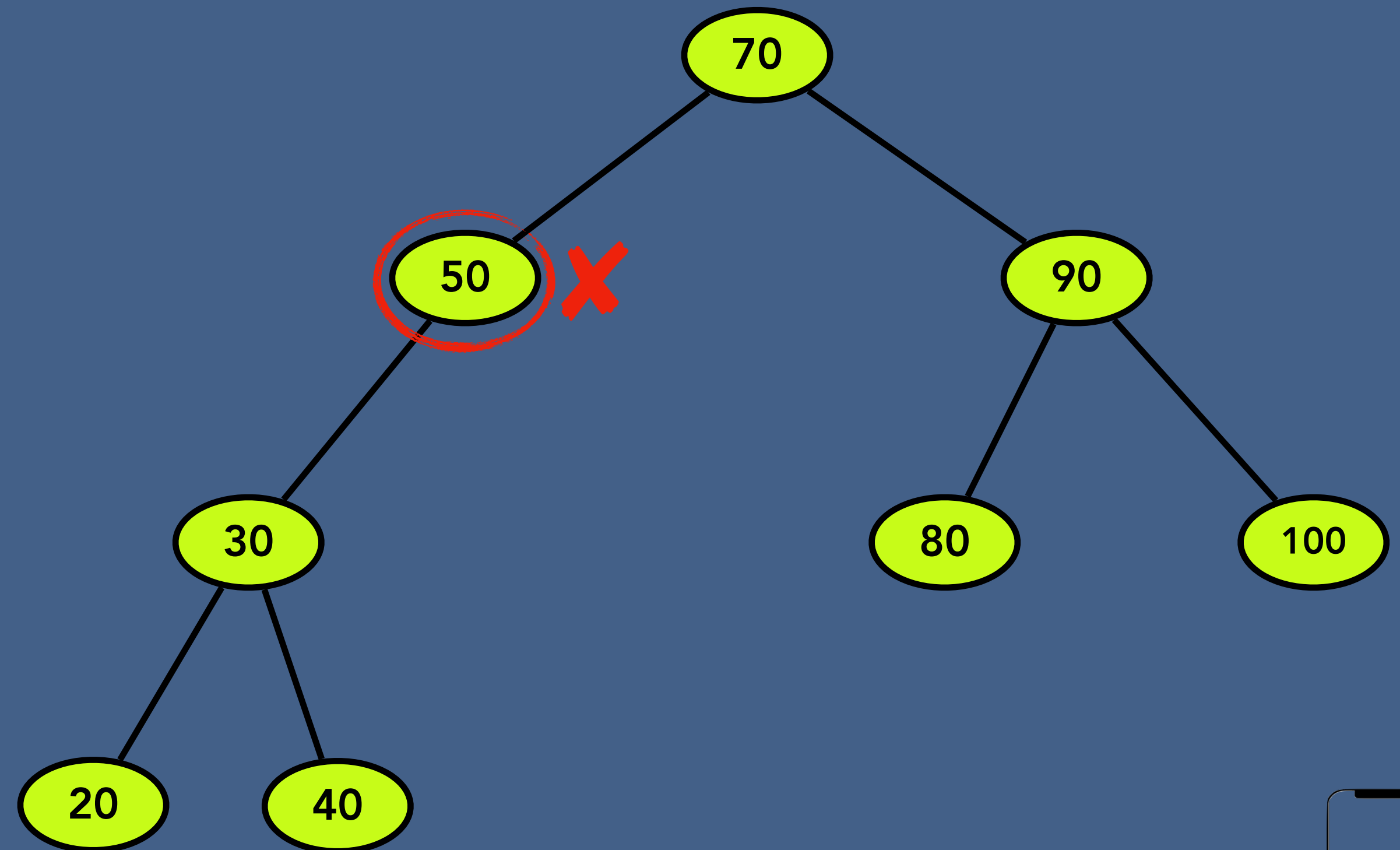
difference = 2



What is an AVL Tree?

An AVL tree is a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees cannot be more than one for all nodes.

If at any time heights of left and right subtrees differ by more than one, then rebalancing is done to restore AVL property, this process is called **rotation**



What is an AVL Tree?

Examples

Height of leftSubtree = 2
Height of rightSubtree = 2

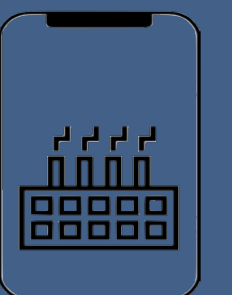
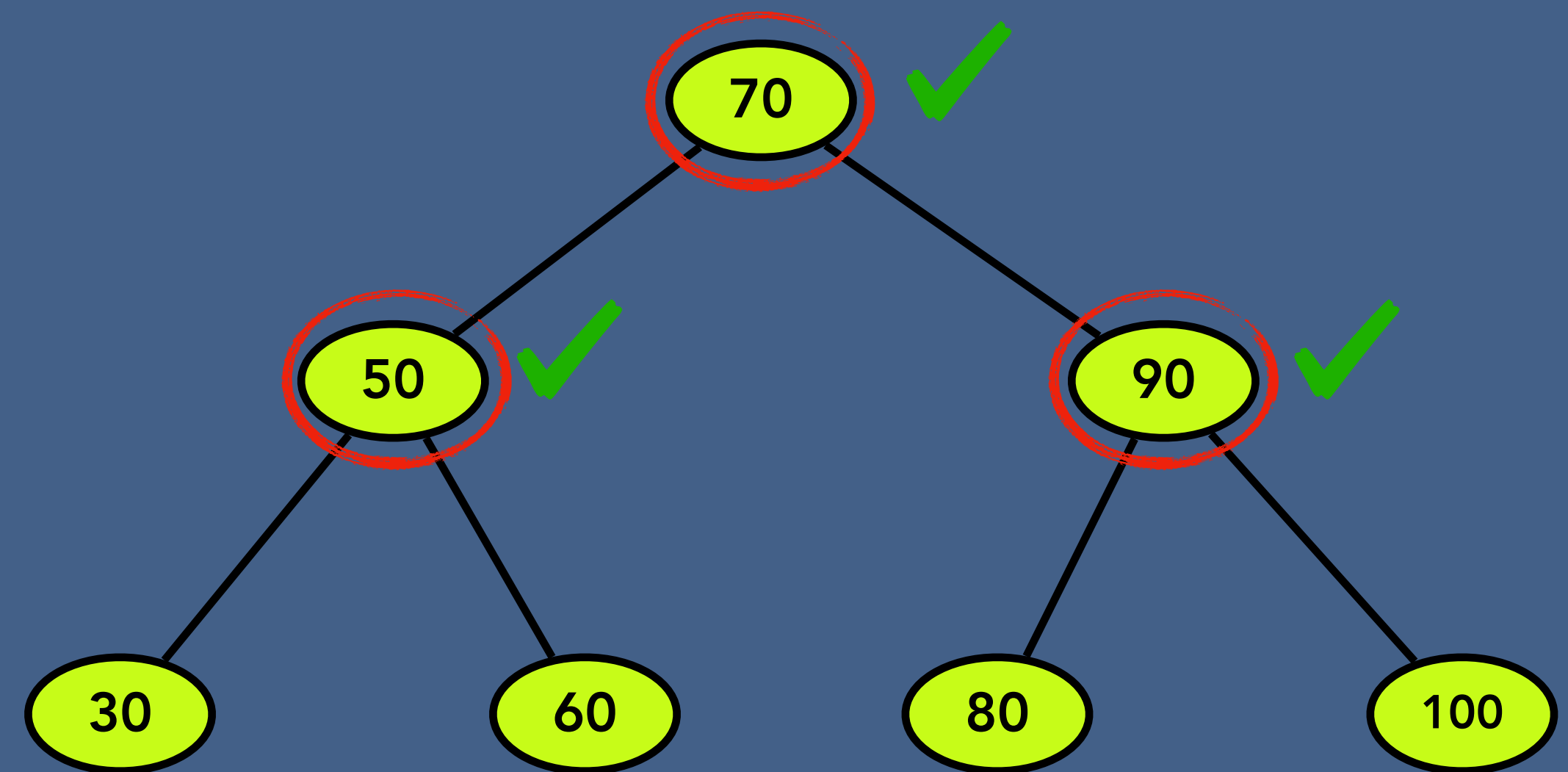
difference = 0

Height of leftSubtree = 1
Height of rightSubtree = 1

difference = 0

Height of leftSubtree = 1
Height of rightSubtree = 1

difference = 0



What is an AVL Tree?

Examples

Height of leftSubtree = 3
Height of rightSubtree = 2

difference = 1

Height of leftSubtree = 2
Height of rightSubtree = 1

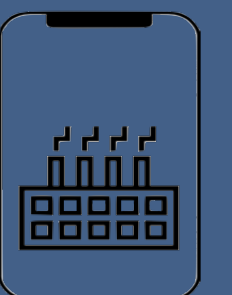
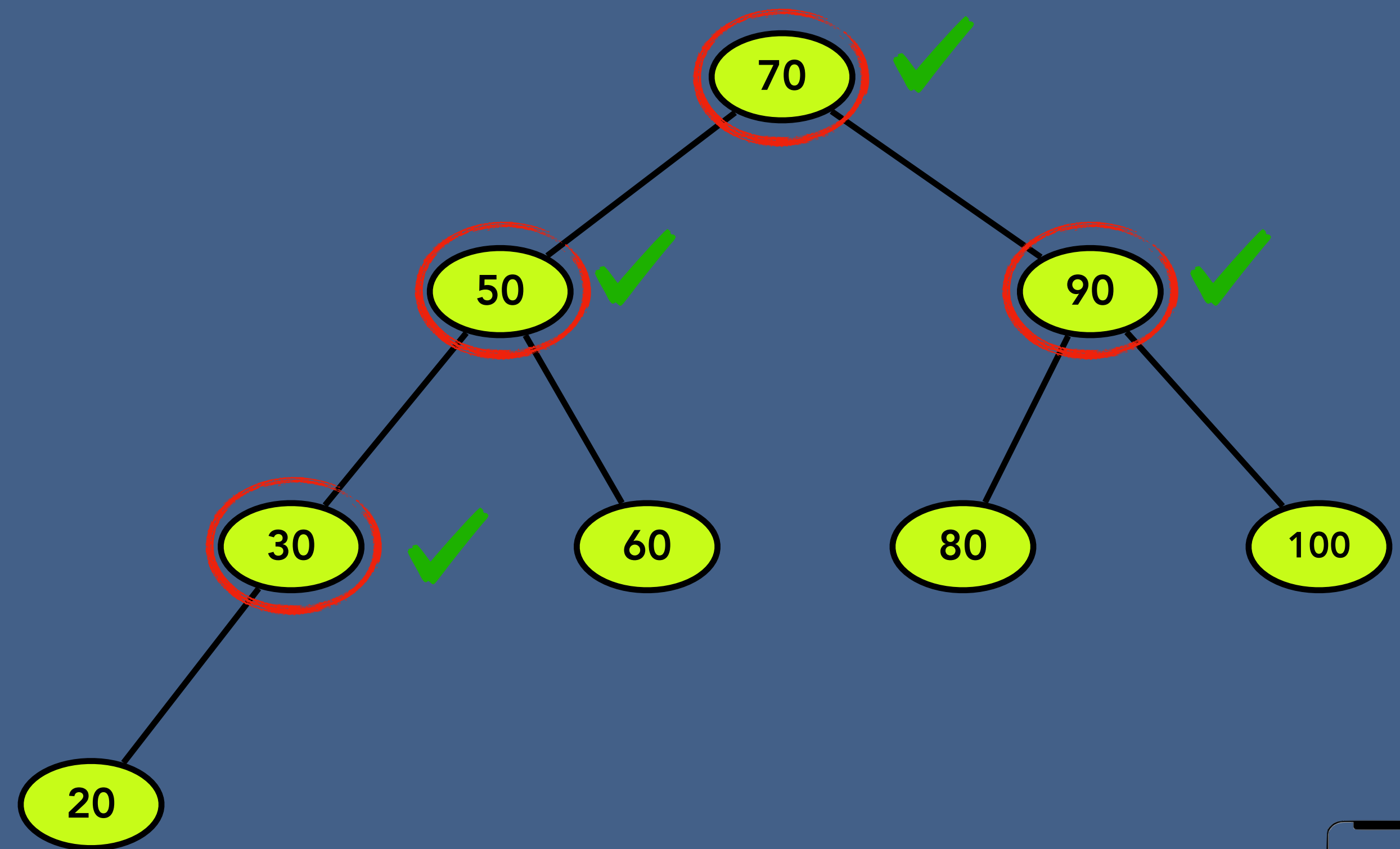
difference = 1

Height of leftSubtree = 1
Height of rightSubtree = 1

difference = 0

Height of leftSubtree = 1
Height of rightSubtree = 0

difference = 1

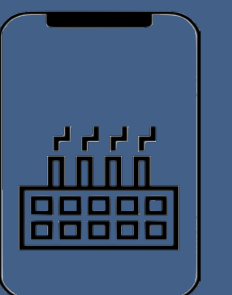
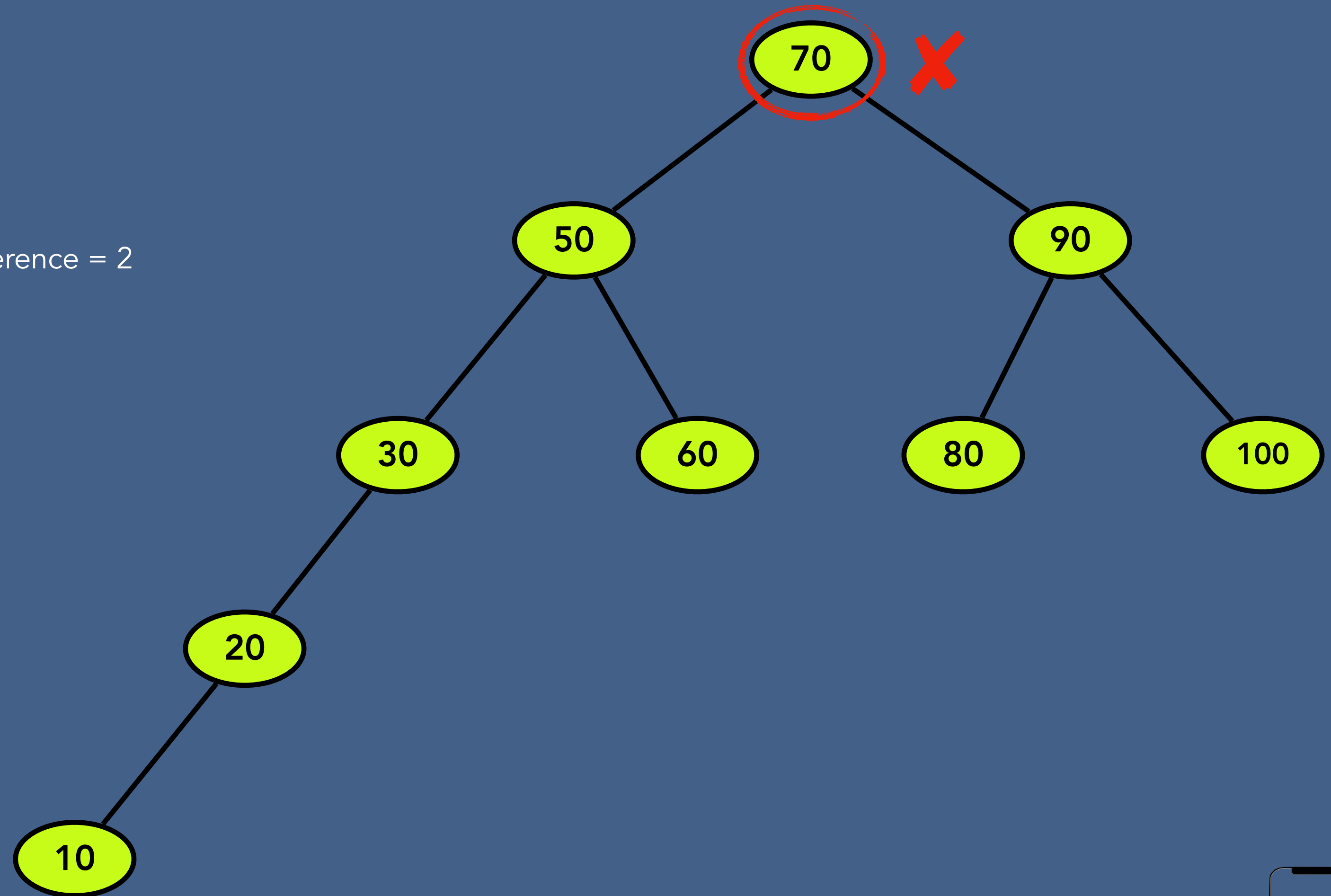


What is an AVL Tree?

Examples

Height of leftSubtree = 4
Height of rightSubtree = 2

difference = 2

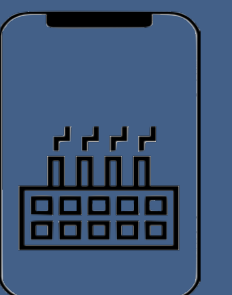
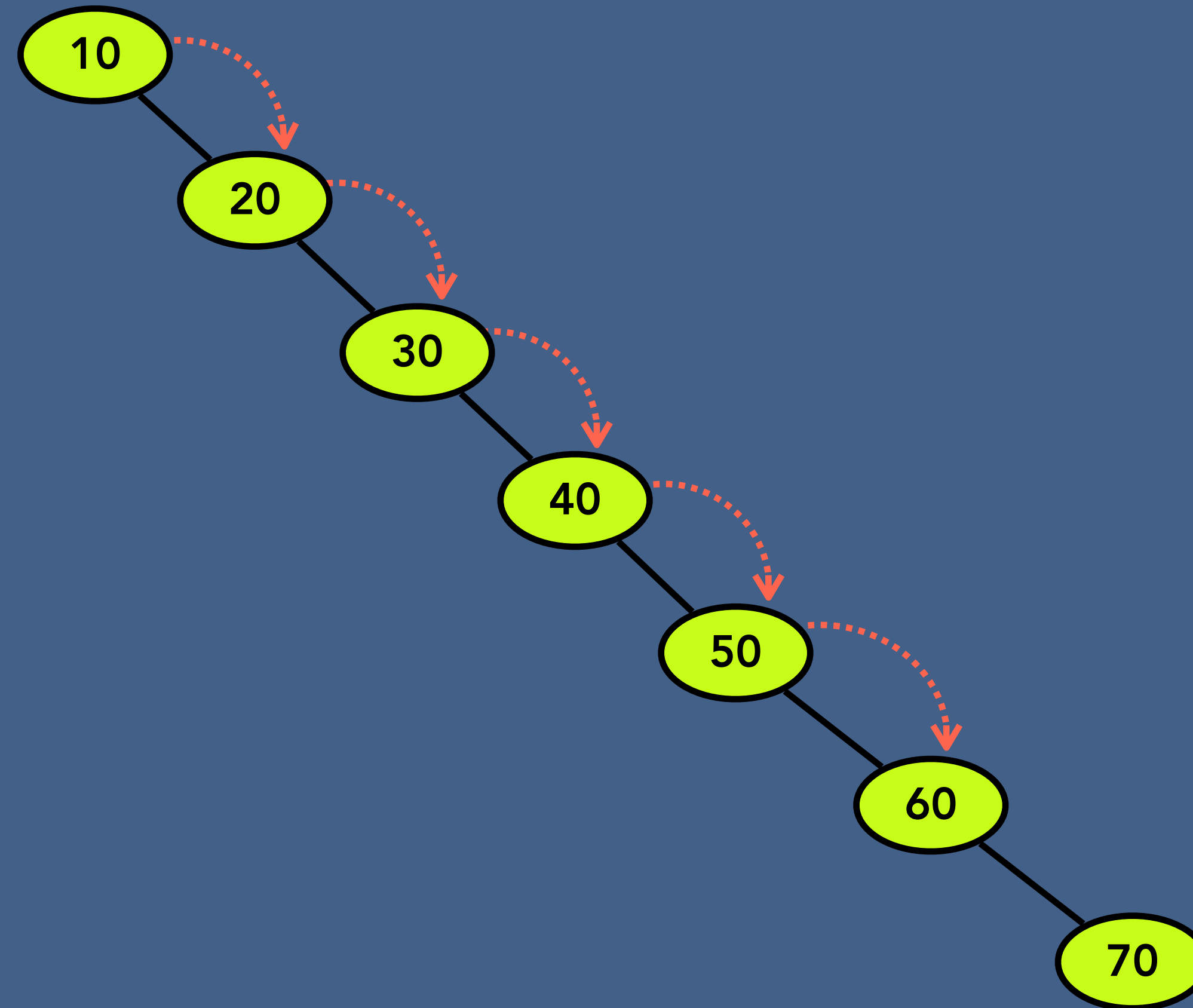


What do we need AVL Tree?

10, 20, 30, 40, 50, 60, 70

Search for 60

Time complexity is $O(N)$

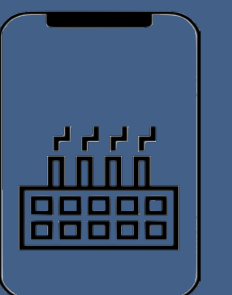
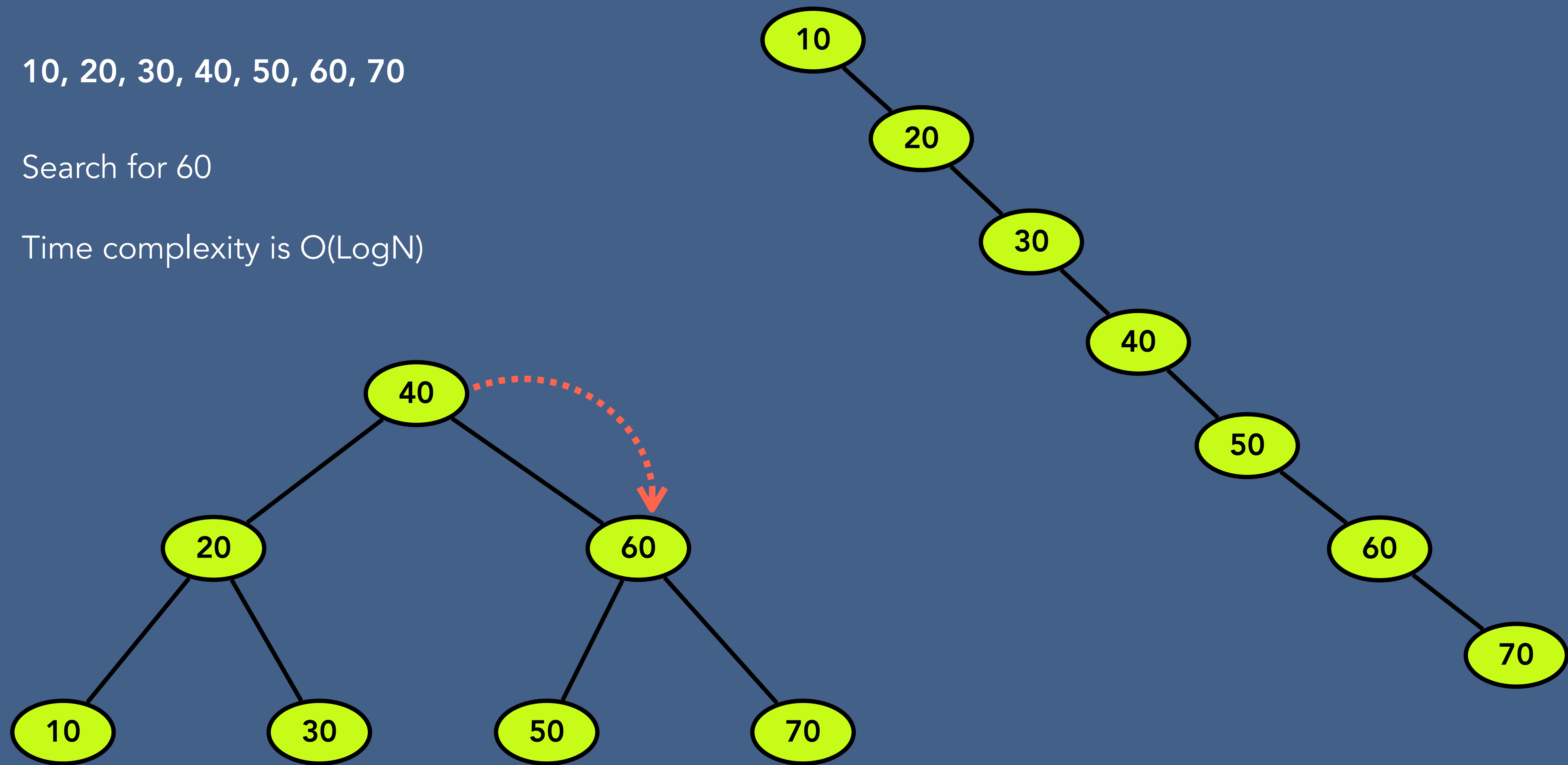


What do we need AVL Tree?

10, 20, 30, 40, 50, 60, 70

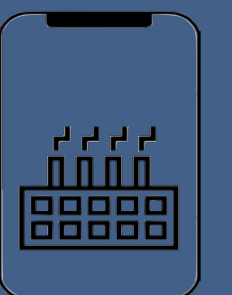
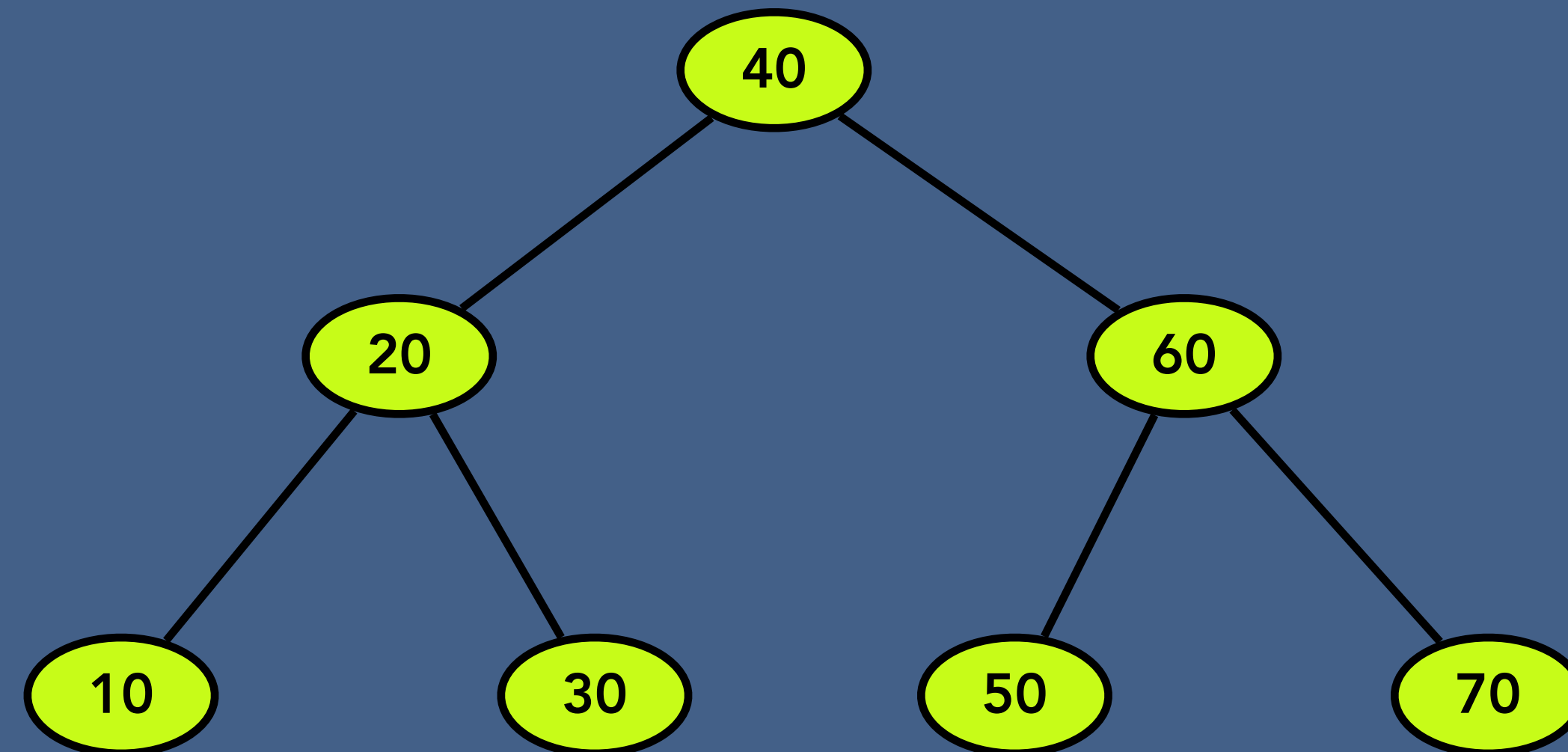
Search for 60

Time complexity is $O(\log N)$



Common Operations on AVL Tree

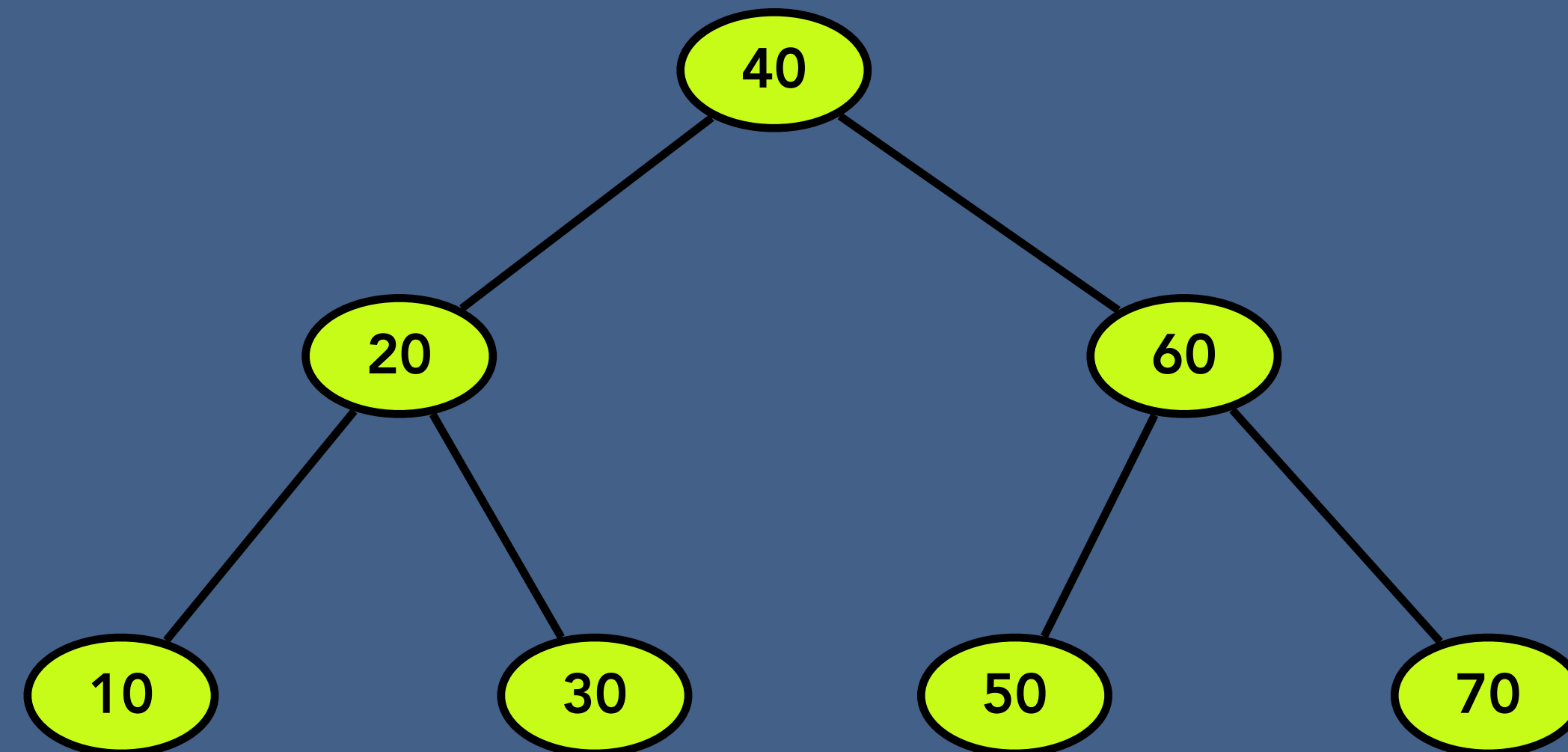
- Creation of AVL trees,
- Search for a node in AVL trees
- Traverse all nodes in AVL trees
- Insert a node in AVL trees
- Delete a node from AVL trees
- Delete the entire AVL trees



Common Operations on AVL Tree

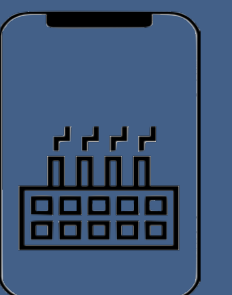
```
newAVL = AVL()
```

```
rootNode = Null
```

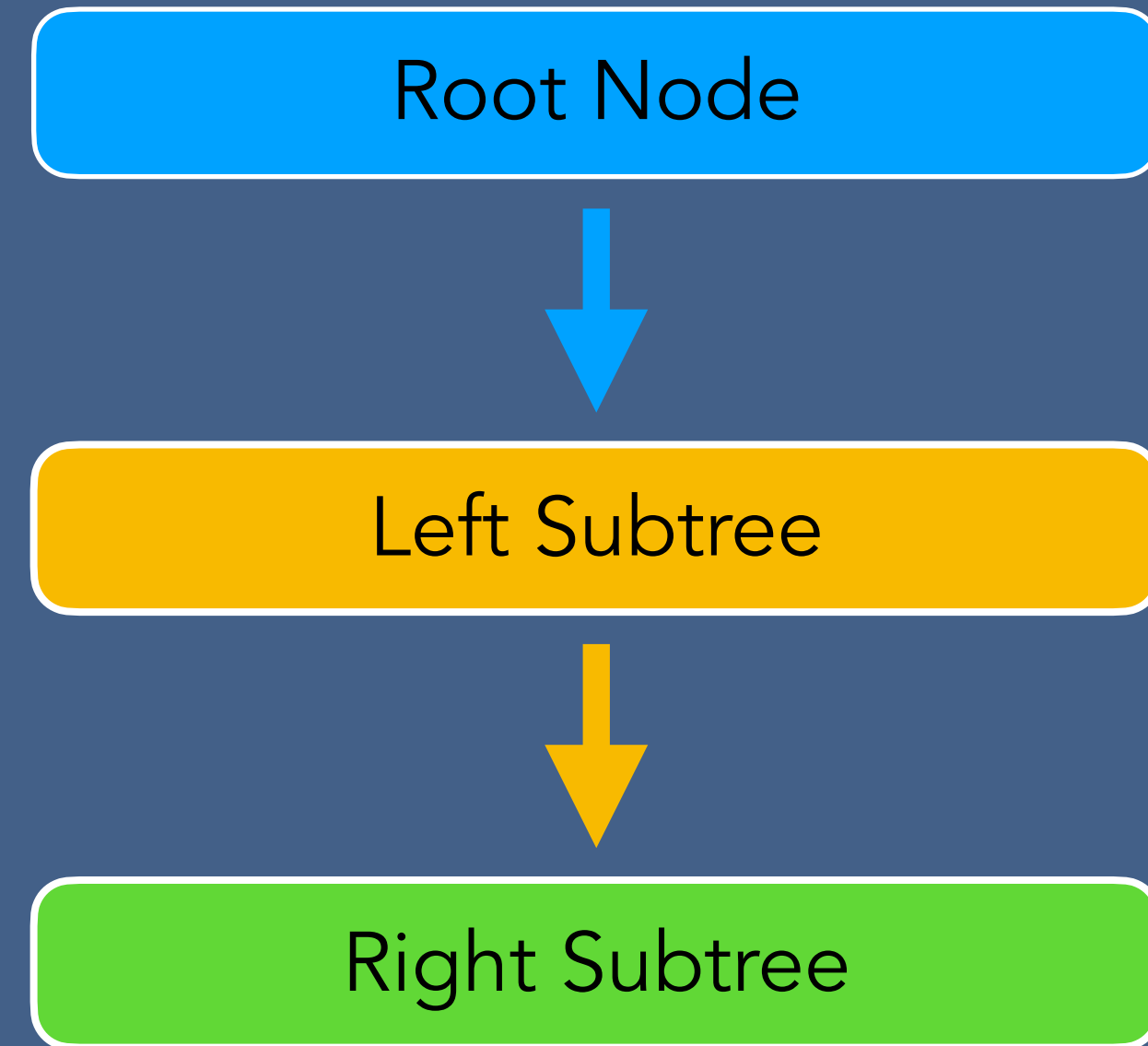


Time complexity : $O(1)$

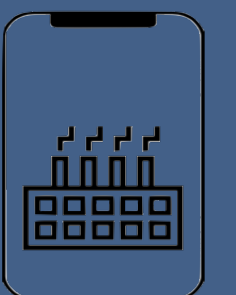
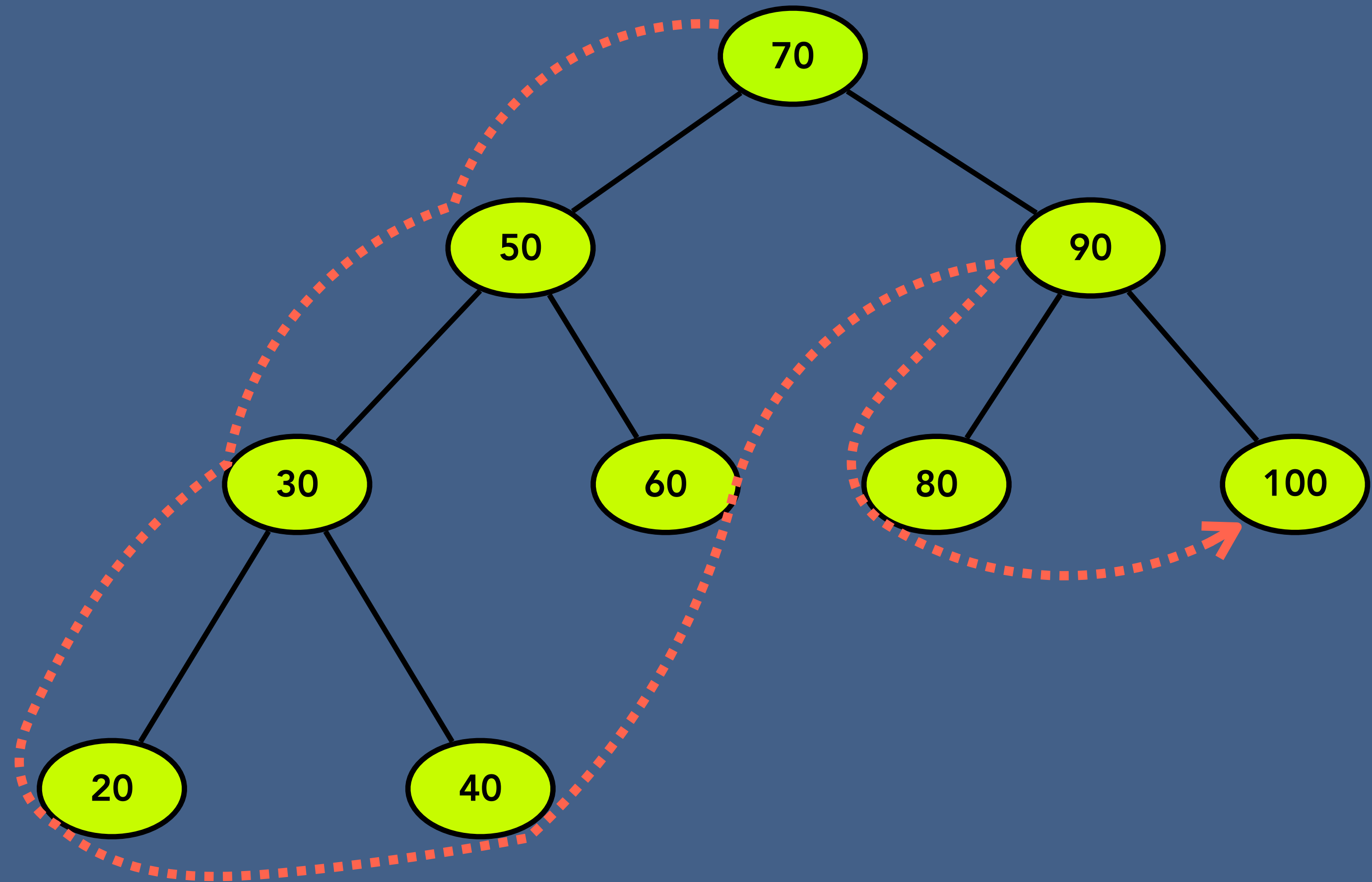
Space complexity : $O(1)$



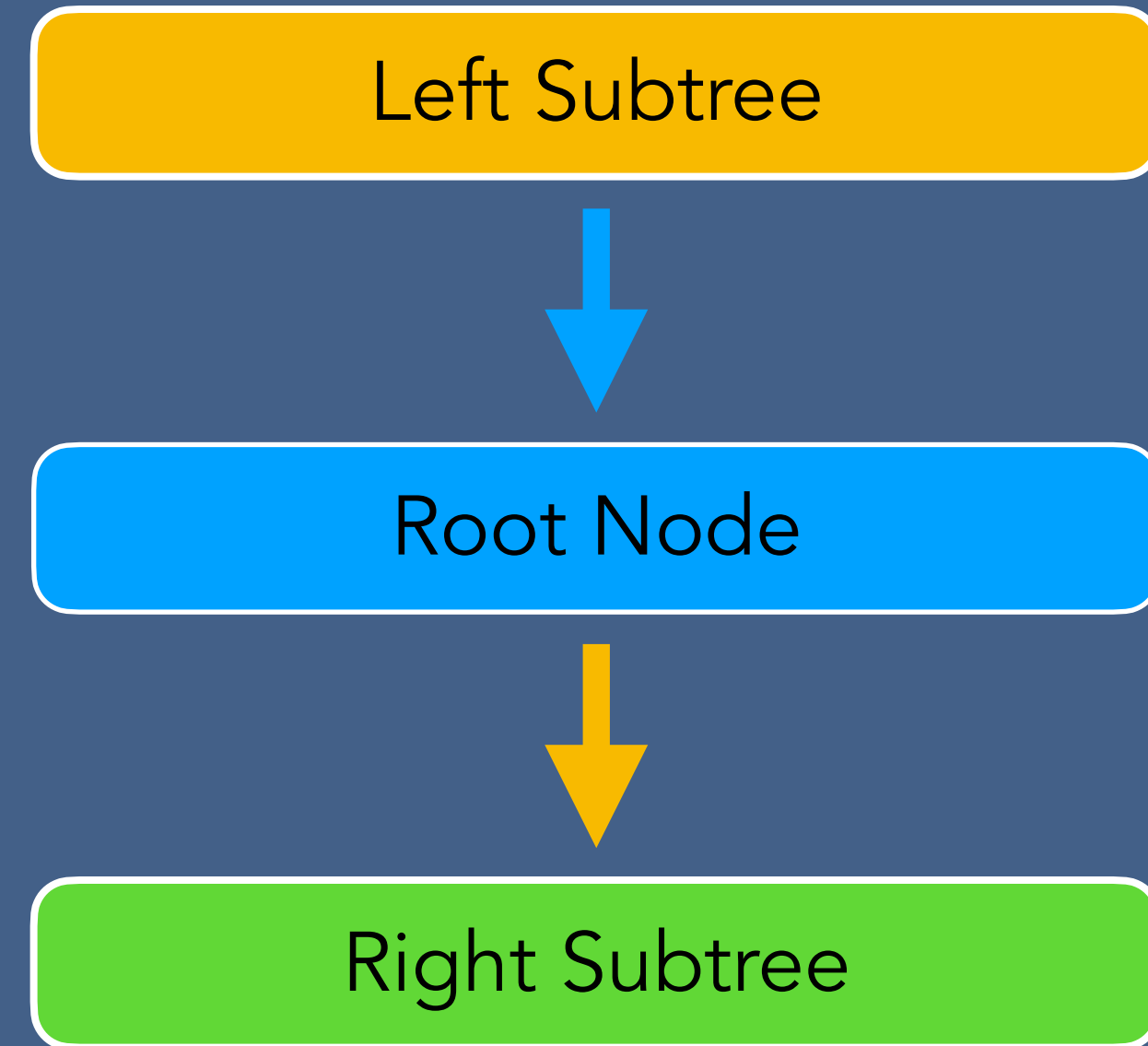
AVL Tree - PreOrder Traversal



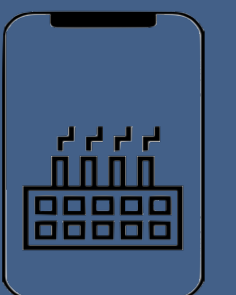
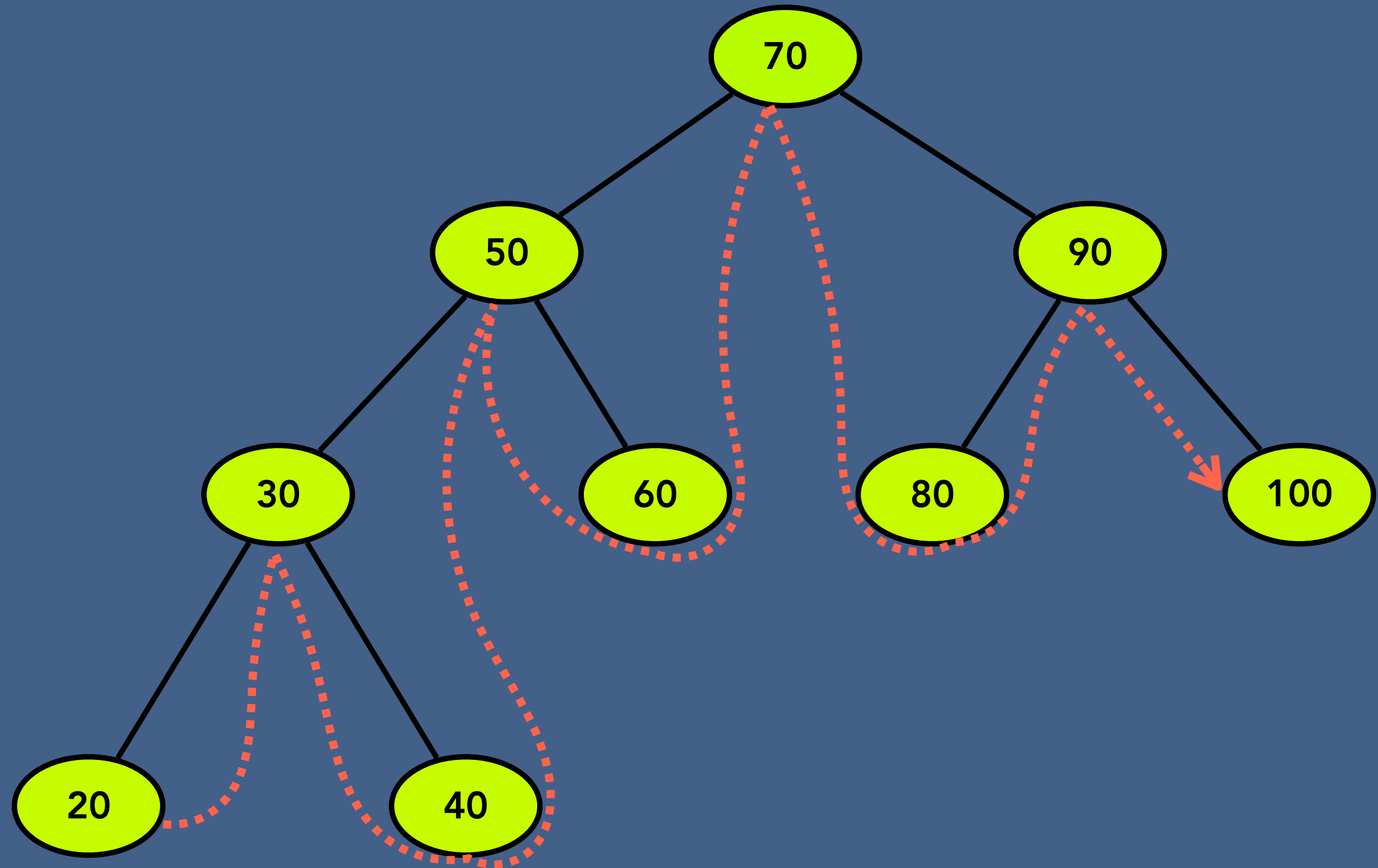
Time complexity : $O(N)$
Space complexity : $O(N)$



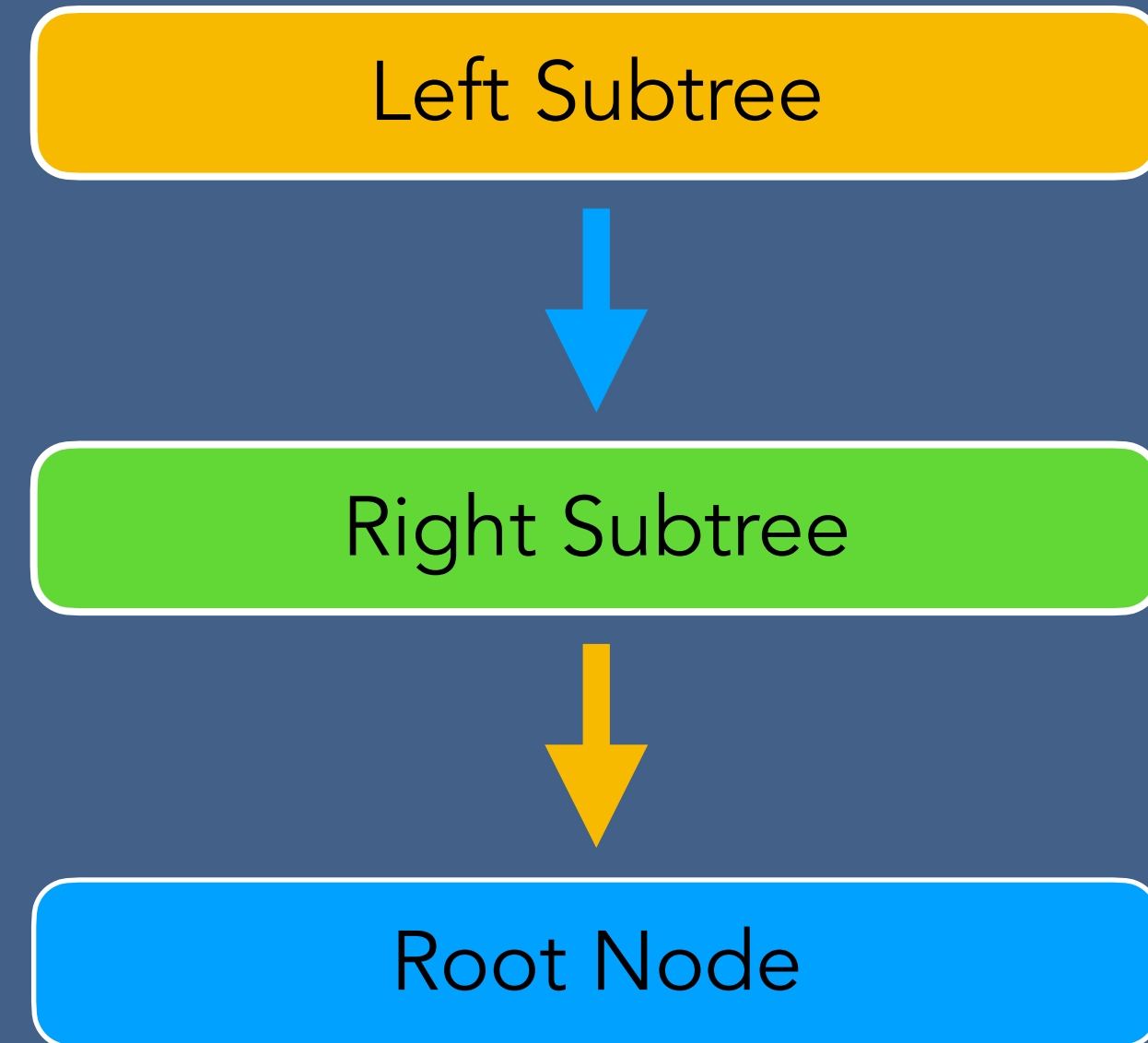
AVL Tree- InOrder Traversal



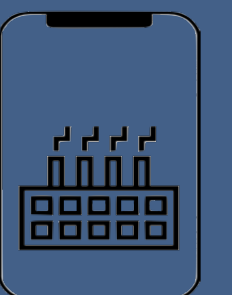
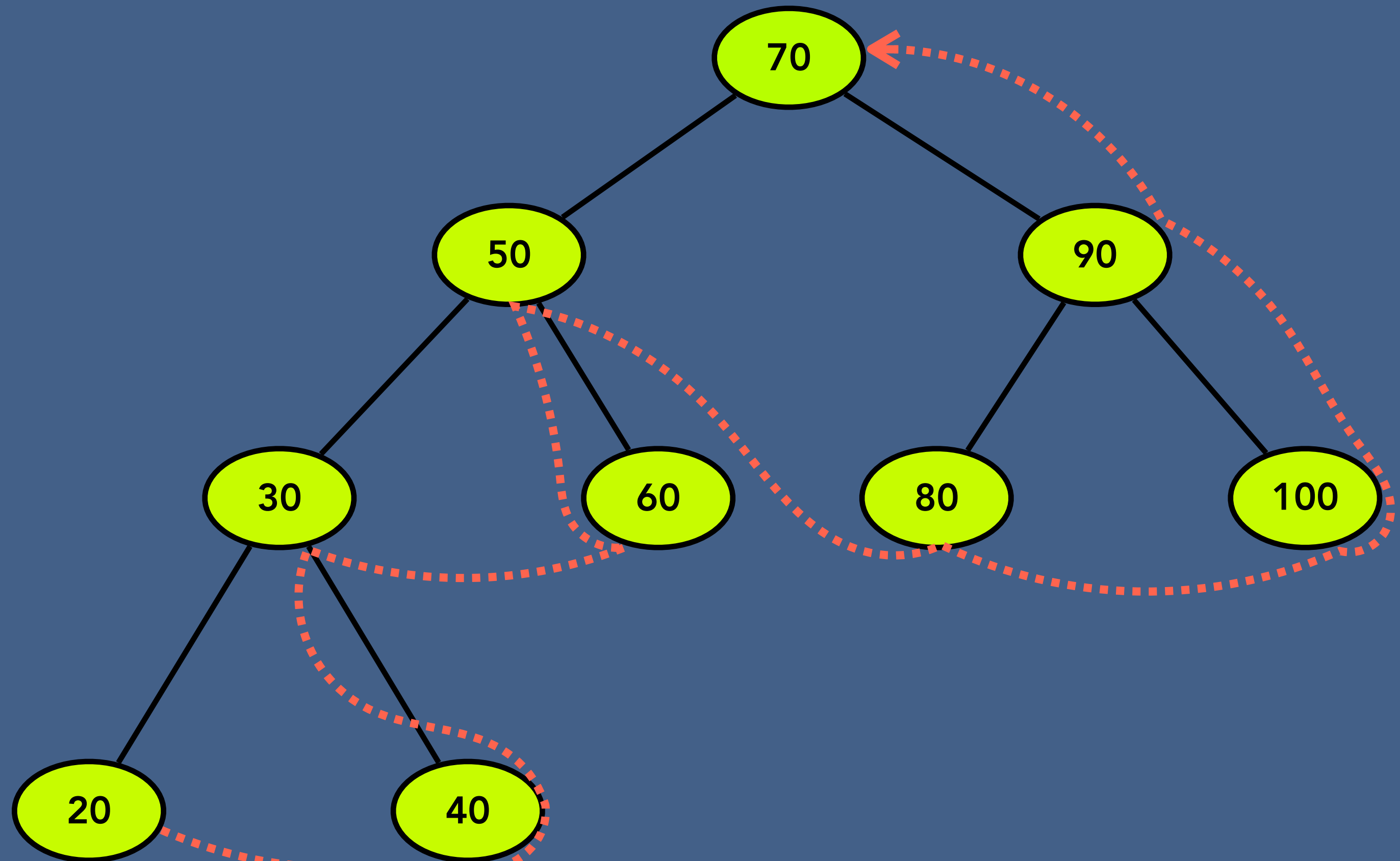
Time complexity : $O(N)$
Space complexity : $O(N)$



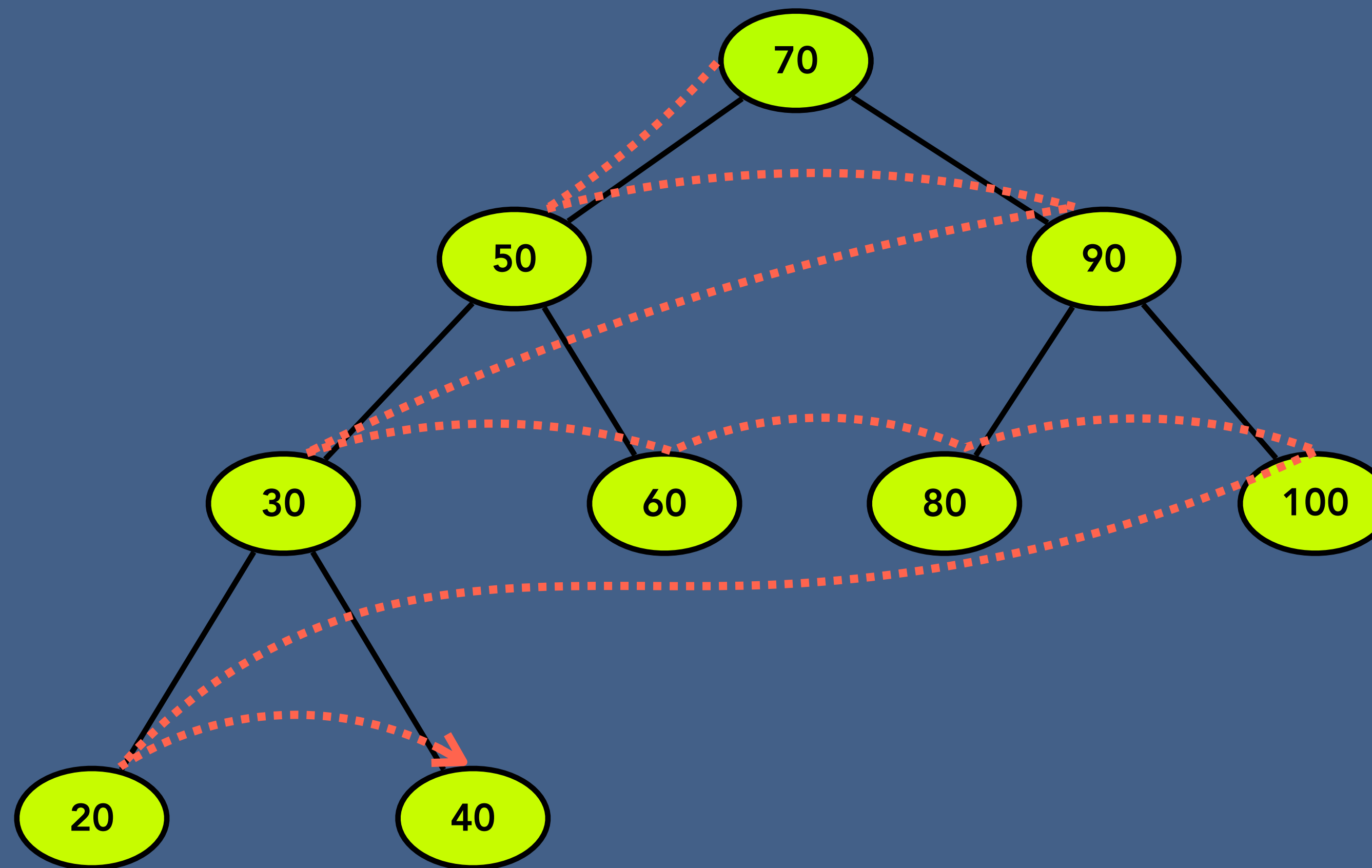
AVL Tree - Post Traversal



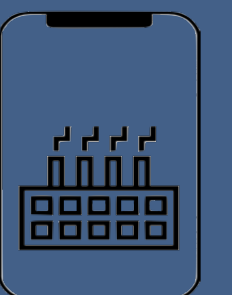
Time complexity : $O(N)$
Space complexity : $O(N)$



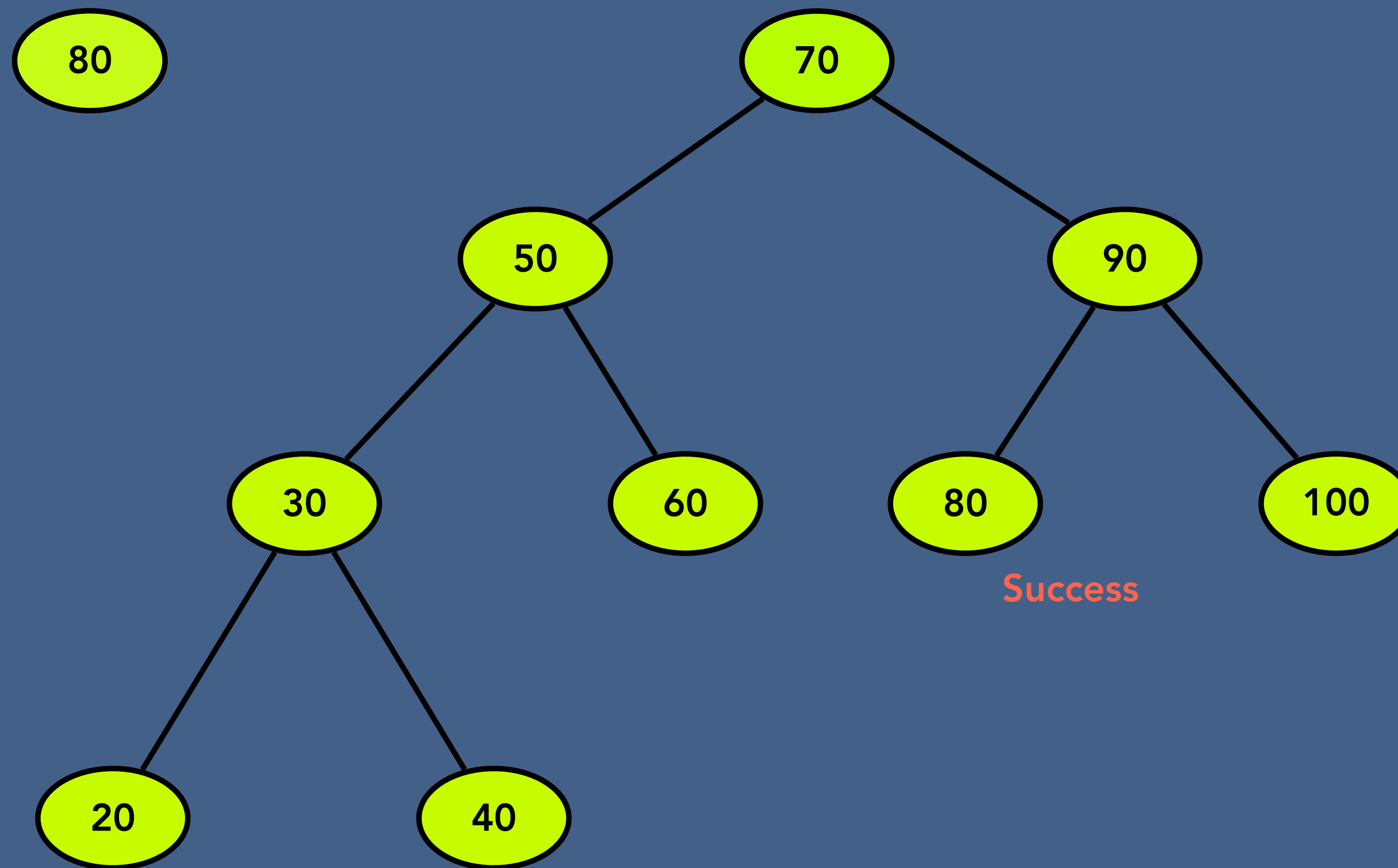
AVL Tree - LevelOrder Traversal



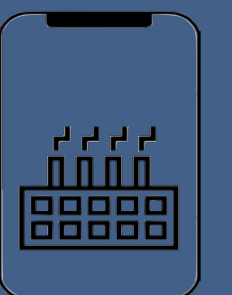
Time complexity : $O(N)$
Space complexity : $O(N)$



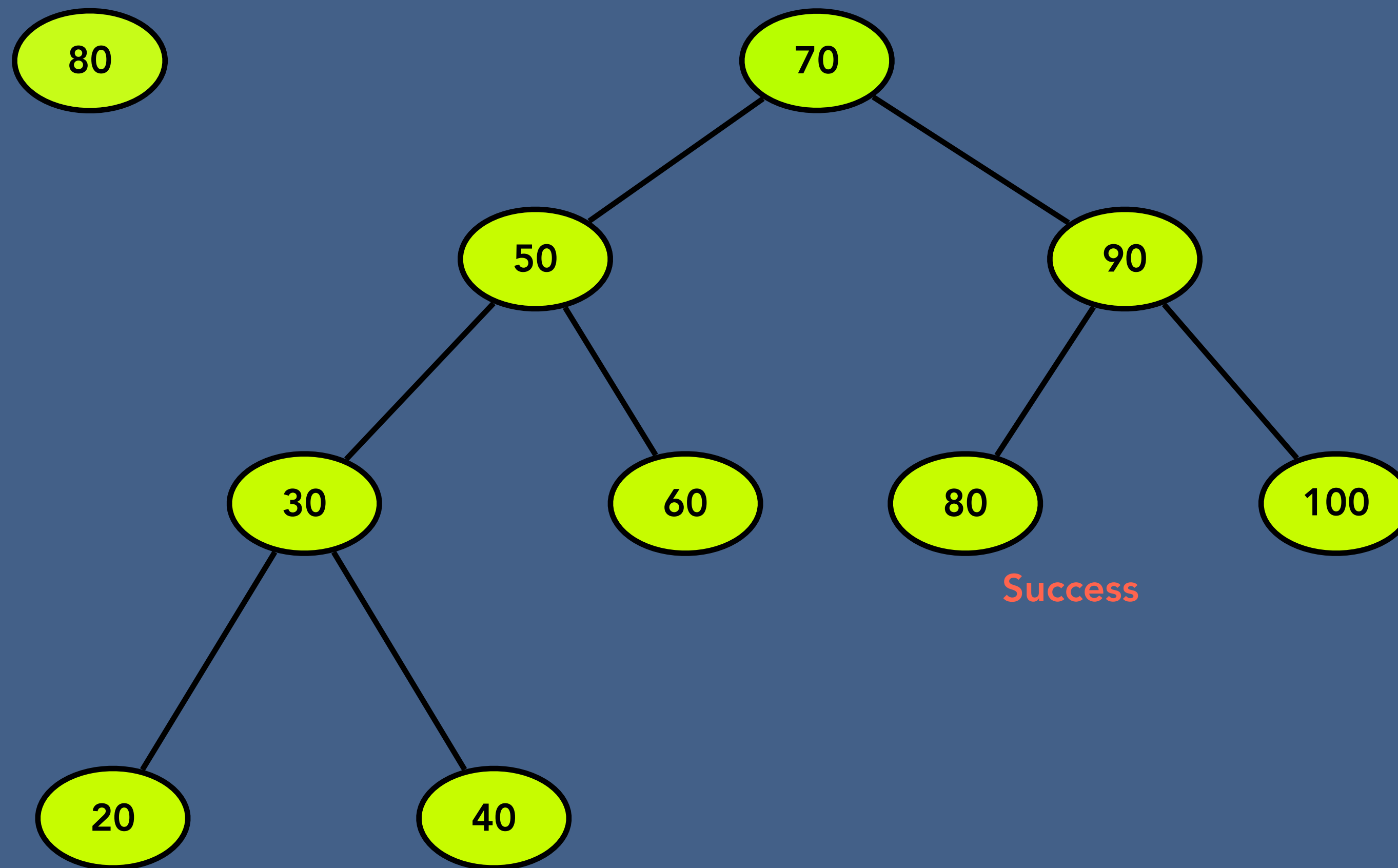
AVL Tree - Search



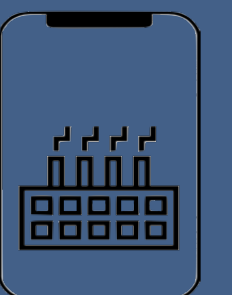
Time complexity : $O(\log N)$
Space complexity : $O(\log N)$



AVL Tree - Search



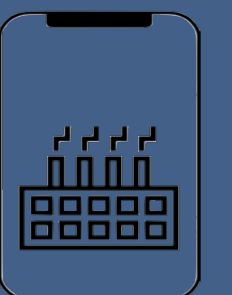
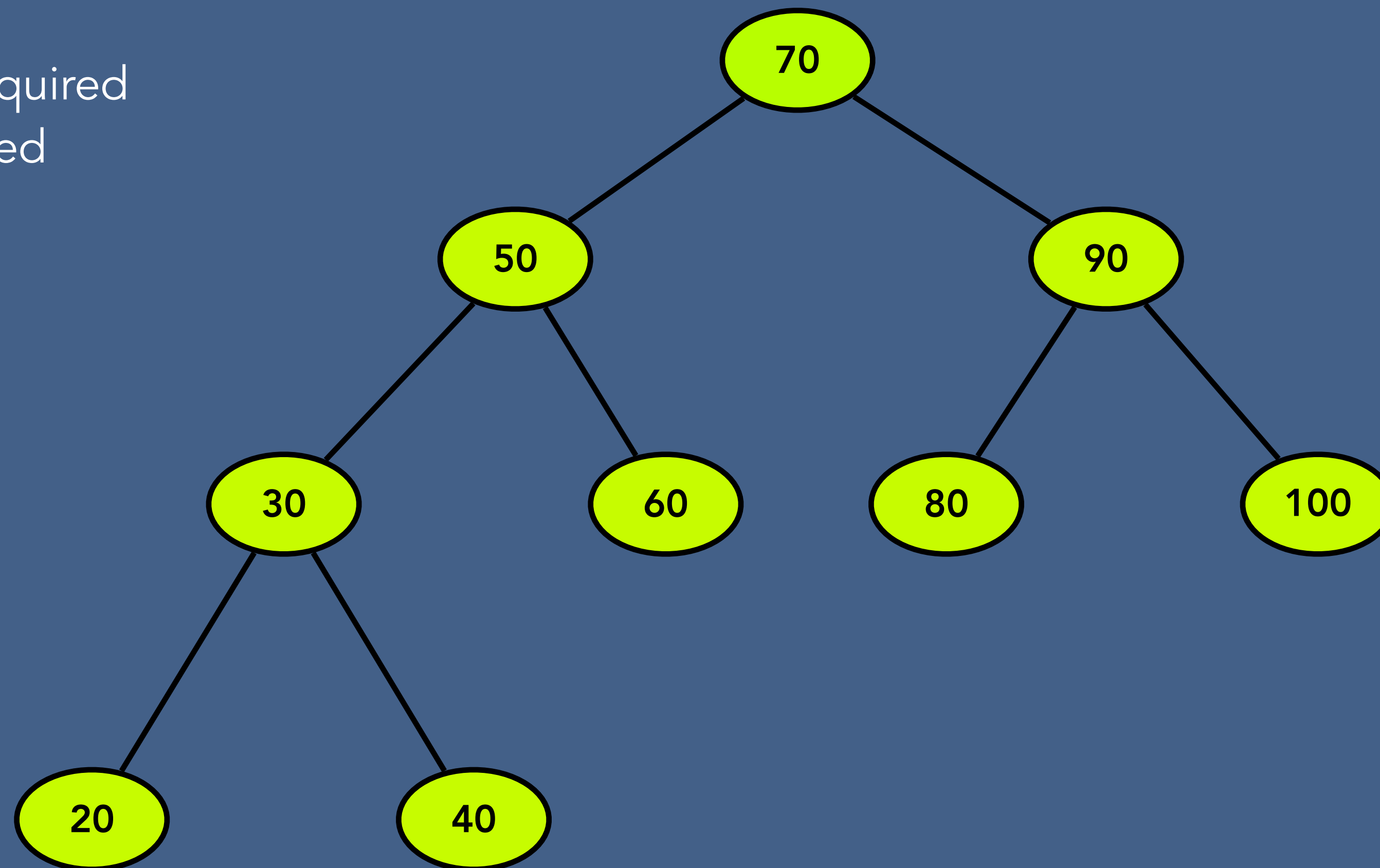
Time complexity : $O(\log N)$
Space complexity : $O(\log N)$



AVL Tree - Insert a Node

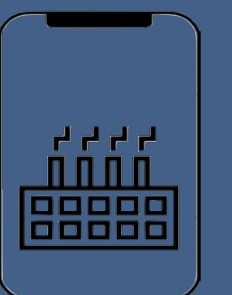
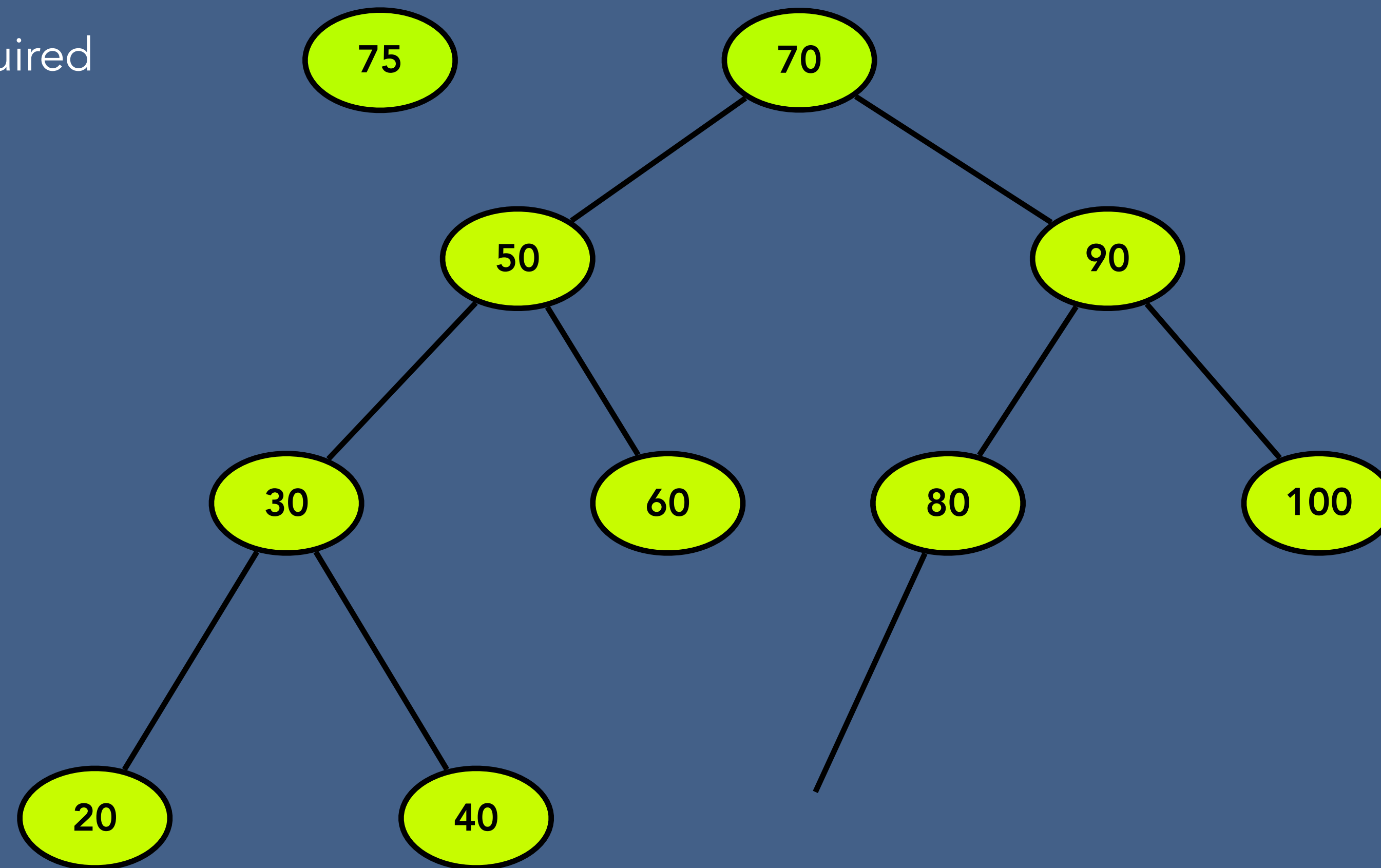
Case 1 : Rotation is not required

Case 2 : Rotation is required



AVL Tree - Insert a Node

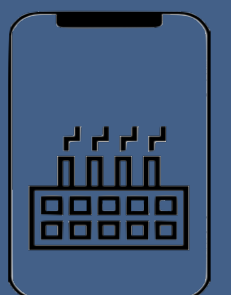
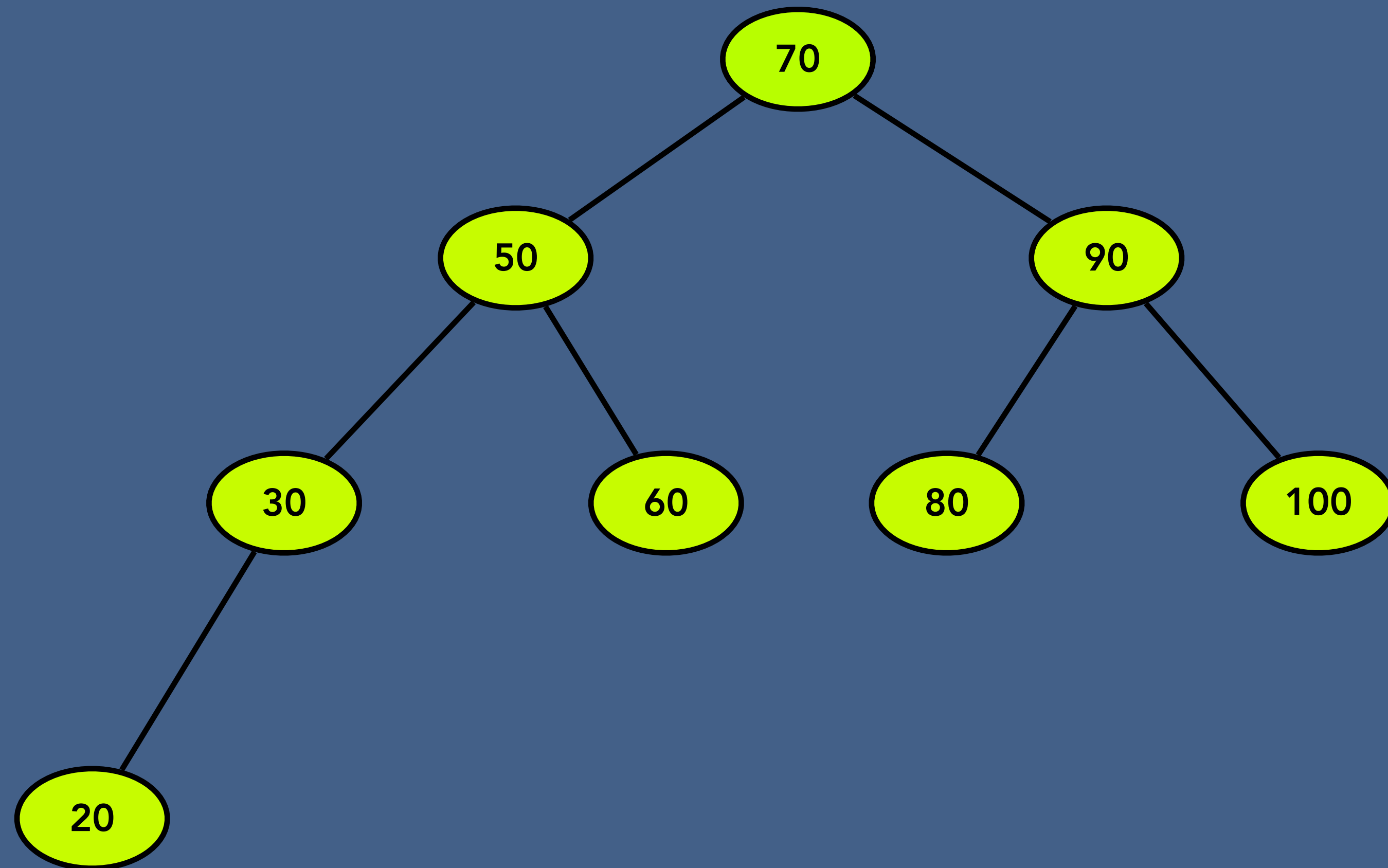
Case 1 : Rotation is not required



AVL Tree - Insert a Node

Case 2 : Rotation is required

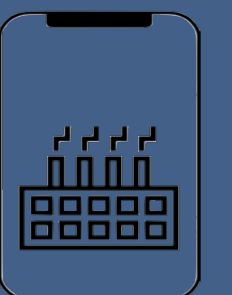
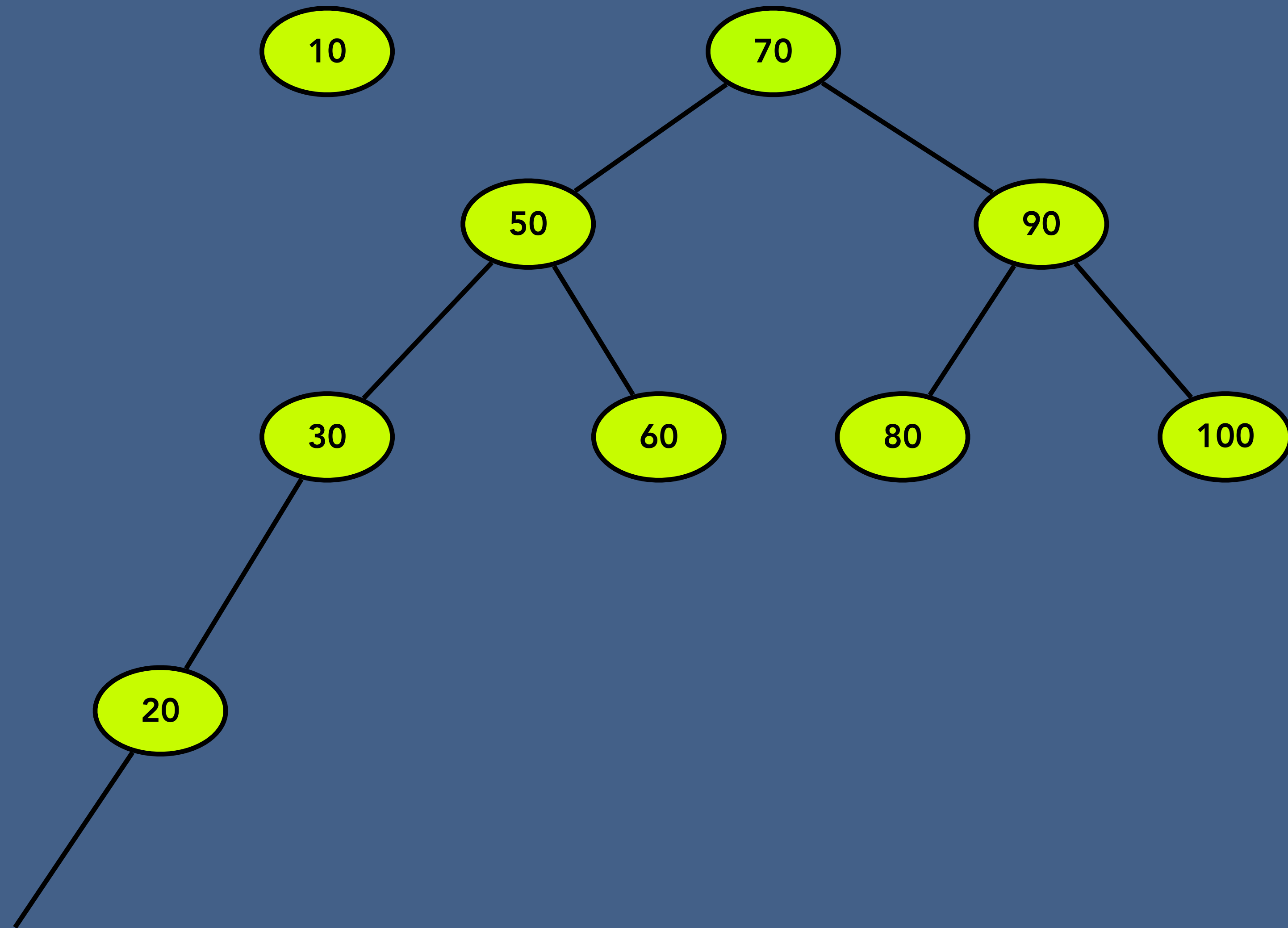
LL - left left condition
LR - left right condition
RR - right right condition
RL - right left condition



AVL Tree - Insert a Node

Case 2 : Rotation is required

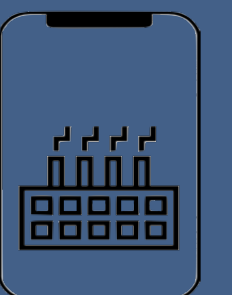
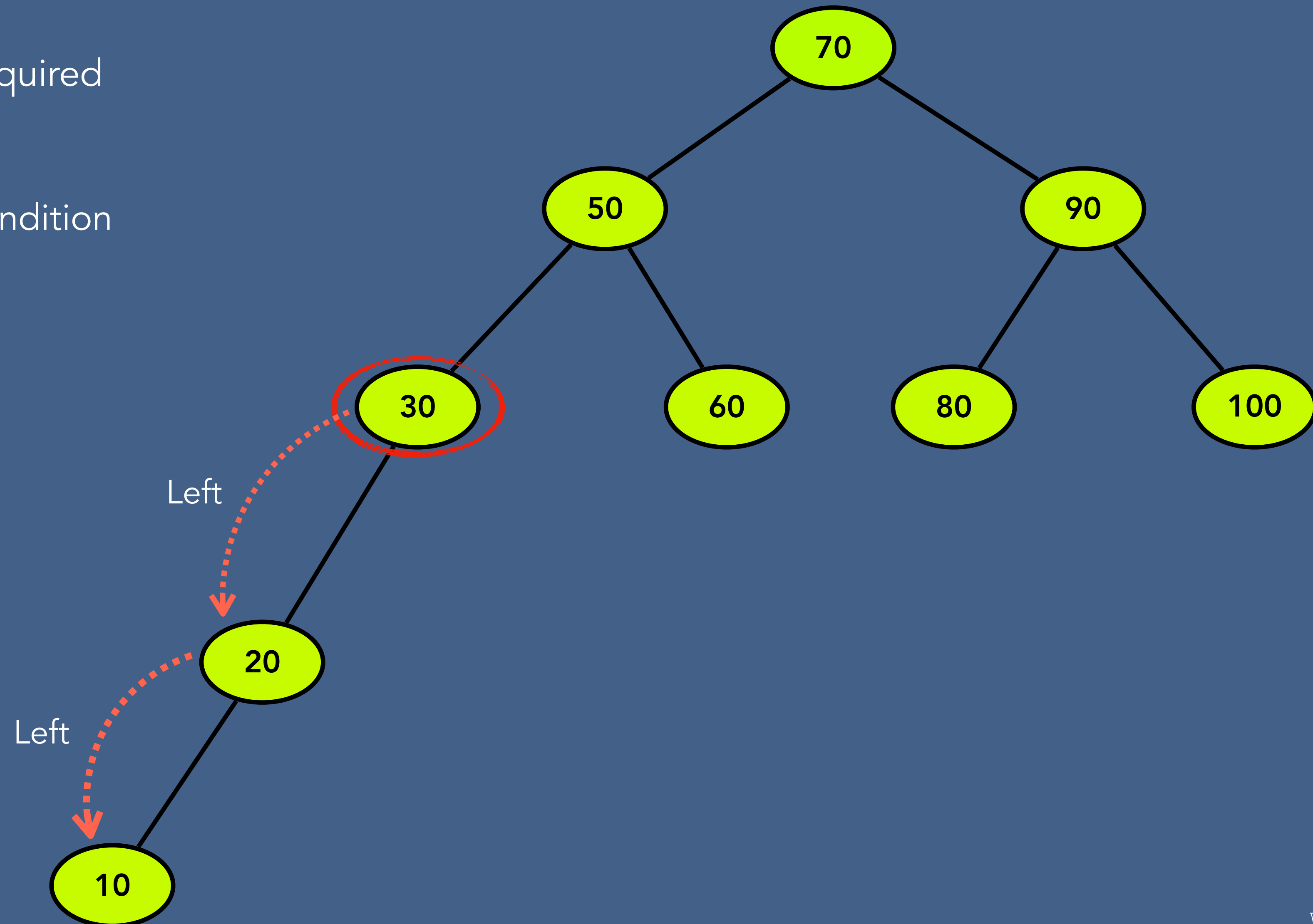
LL - left left condition



AVL Tree - Insert a Node

Case 2 : Rotation is required

LL - left left condition

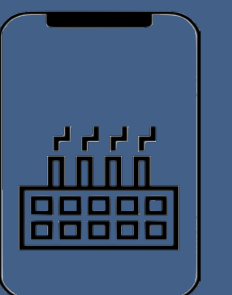
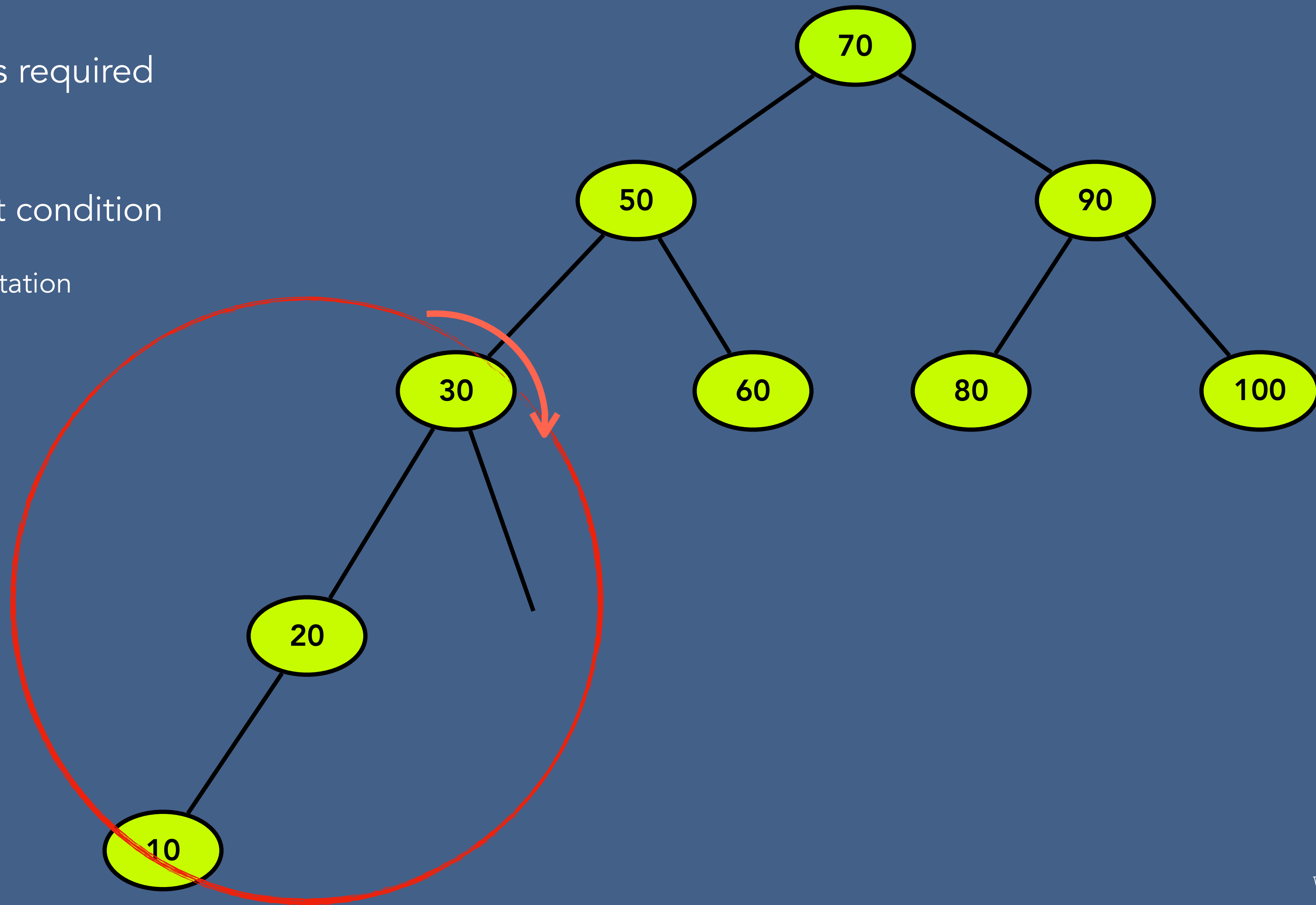


AVL Tree - Insert a Node

Case 2 : Rotation is required

LL - left left condition

Right rotation

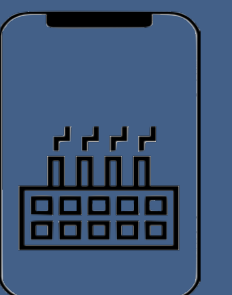
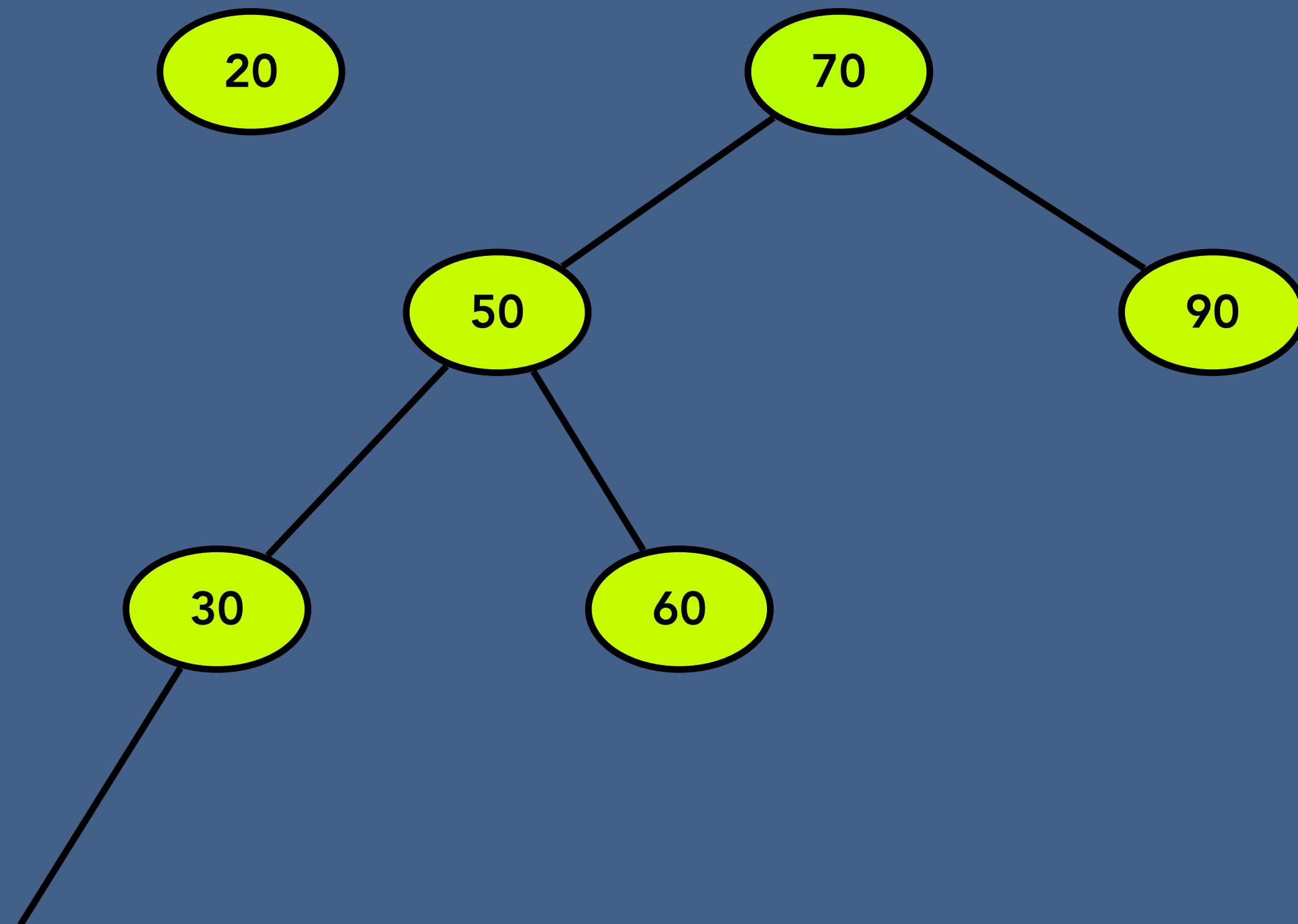


AVL Tree - Insert a Node

Case 2 : Rotation is required

LL - left left condition

Right rotation - example 2

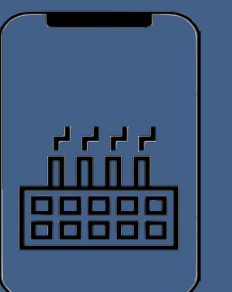
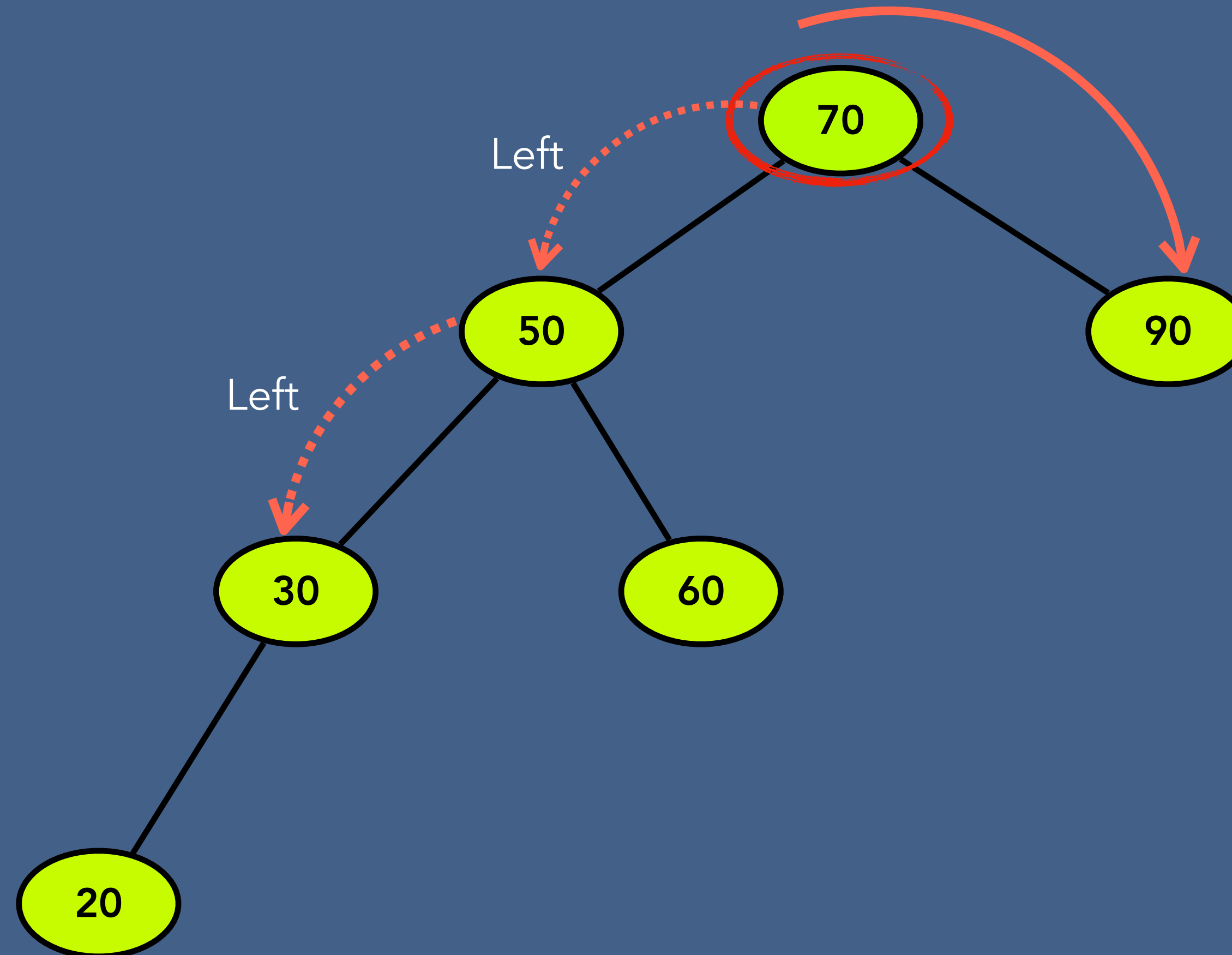


AVL Tree - Insert a Node

Case 2 : Rotation is required

LL - left left condition

Right rotation - example 2

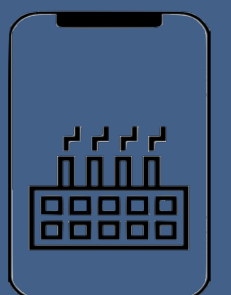
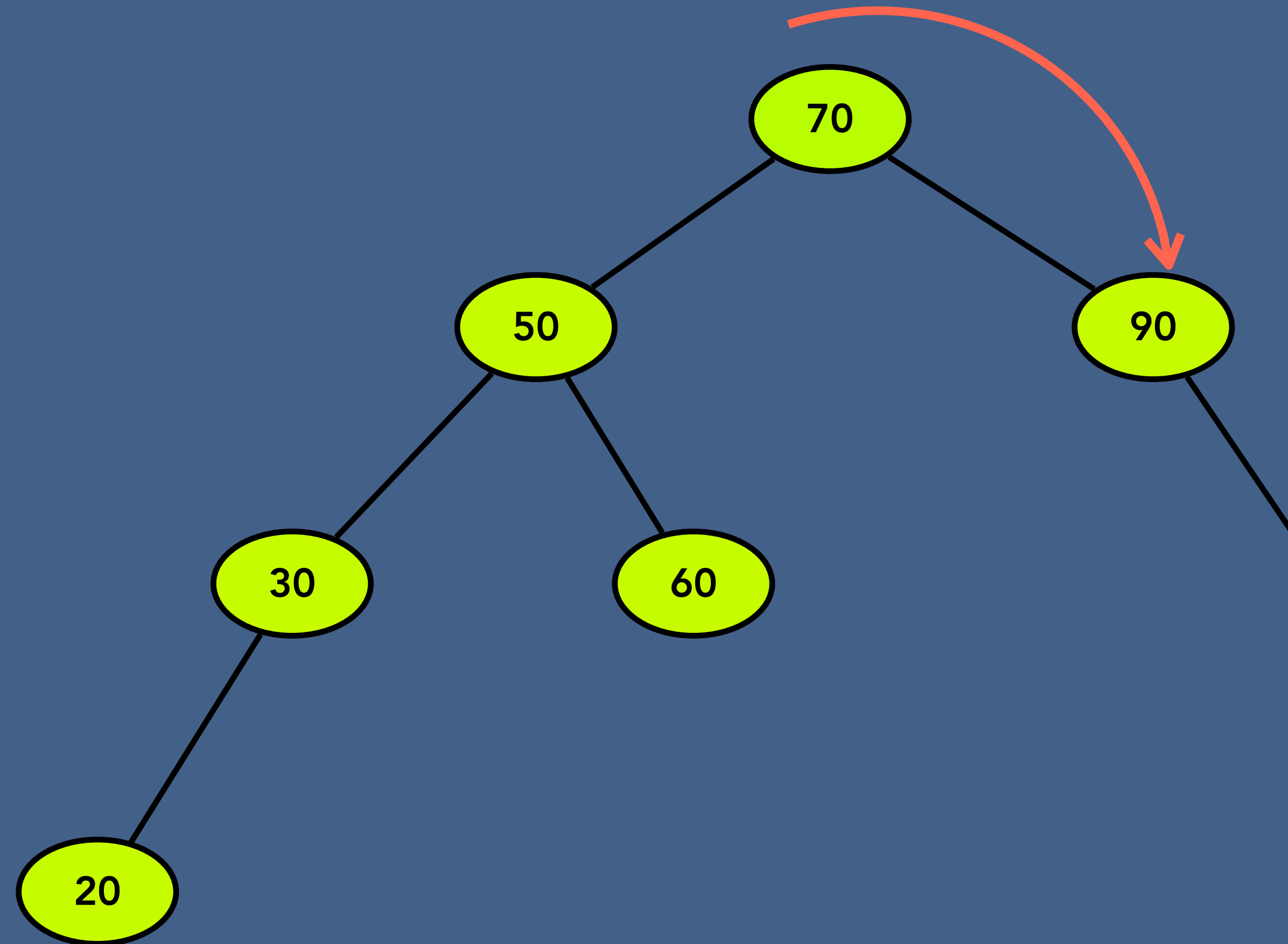


AVL Tree - Insert a Node

Case 2 : Rotation is required

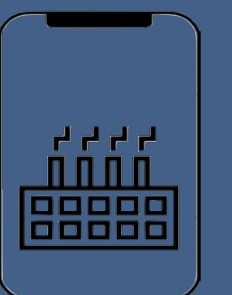
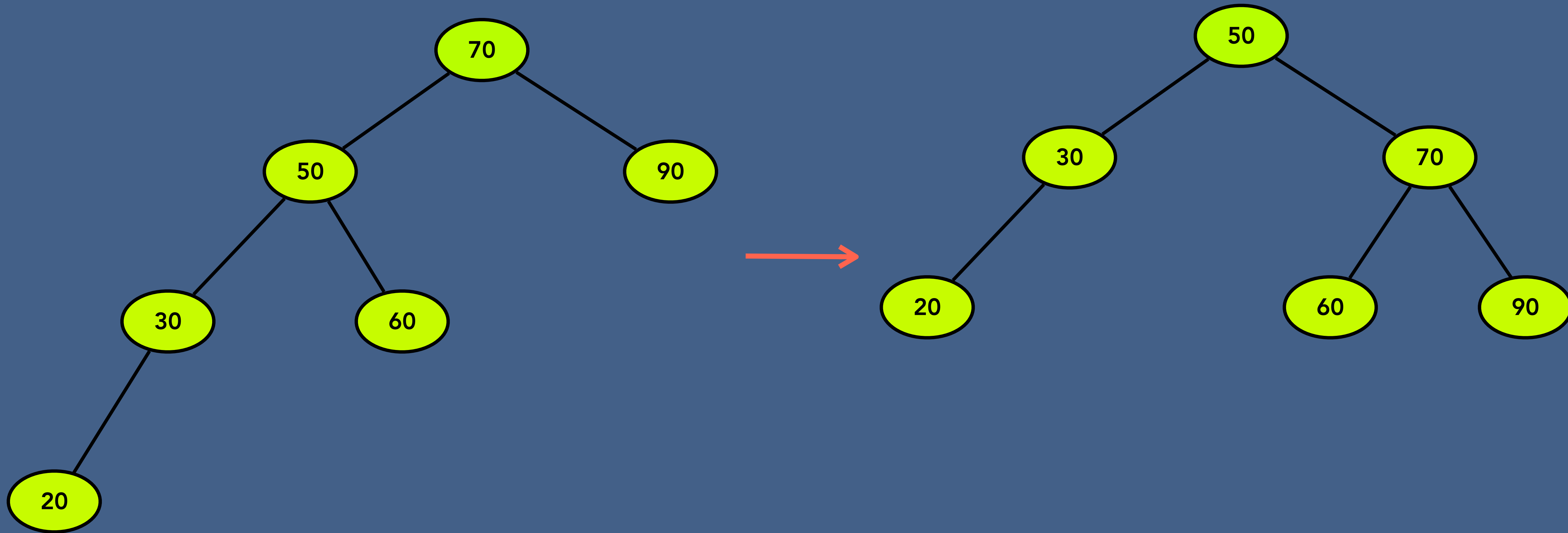
LL - left left condition

Right rotation - example 2



AVL Tree - Insert a Node

Right Rotation - example 2

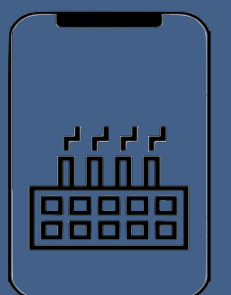
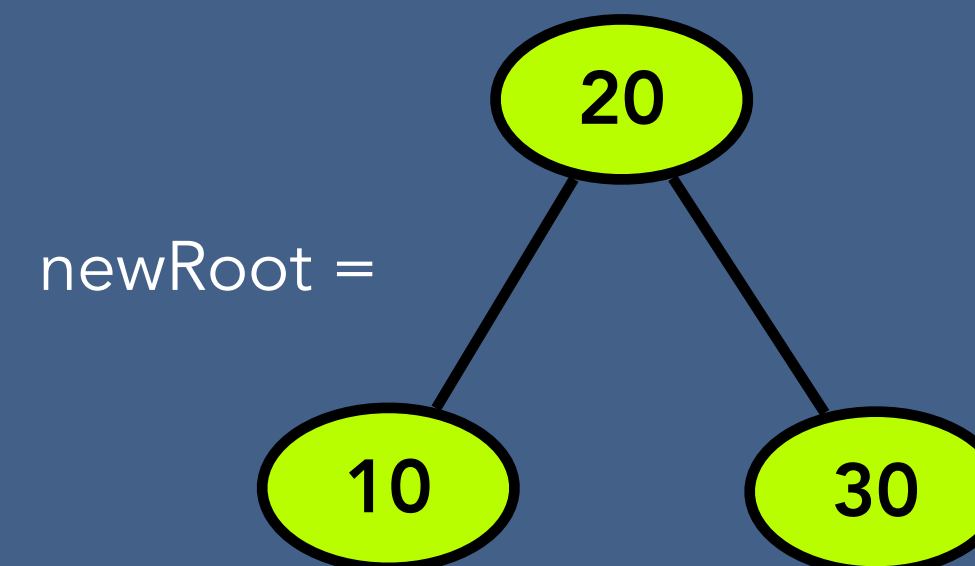
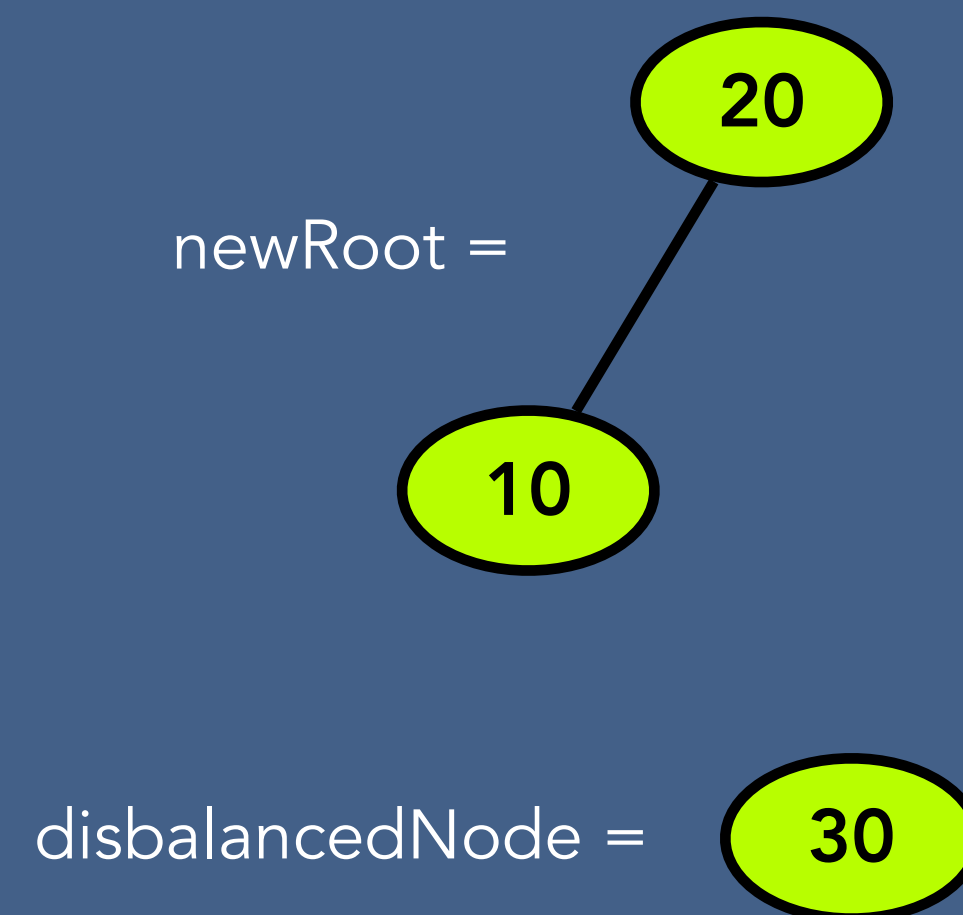
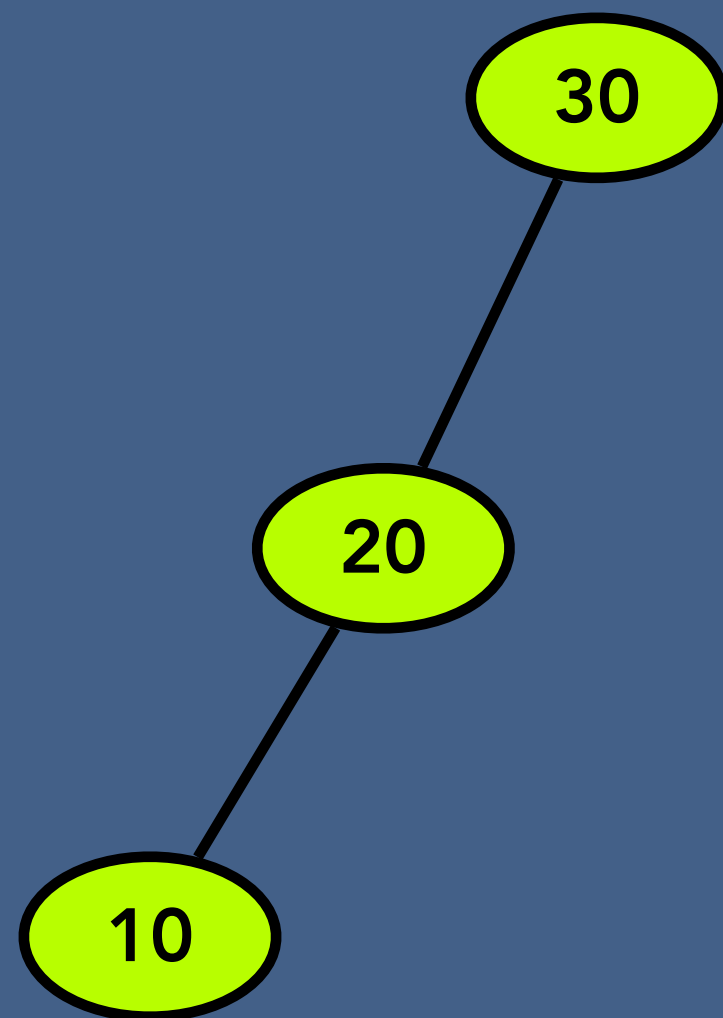


AVL Tree - Insert a Node

Algorithm of Left Left (LL) Condition

```
rotateRight(disbalancedNode) {  
  newRoot = disbalancedNode.leftChild ← Null  
  disbalancedNode.leftChild = disbalancedNode.leftChild.rightChild  
  newRoot.rightChild = disbalancedNode  
  update height of disbalancedNode and newRoot  
  return newRoot  
}
```

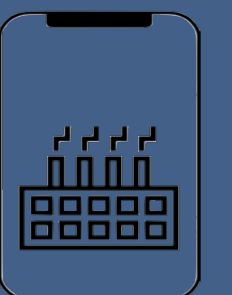
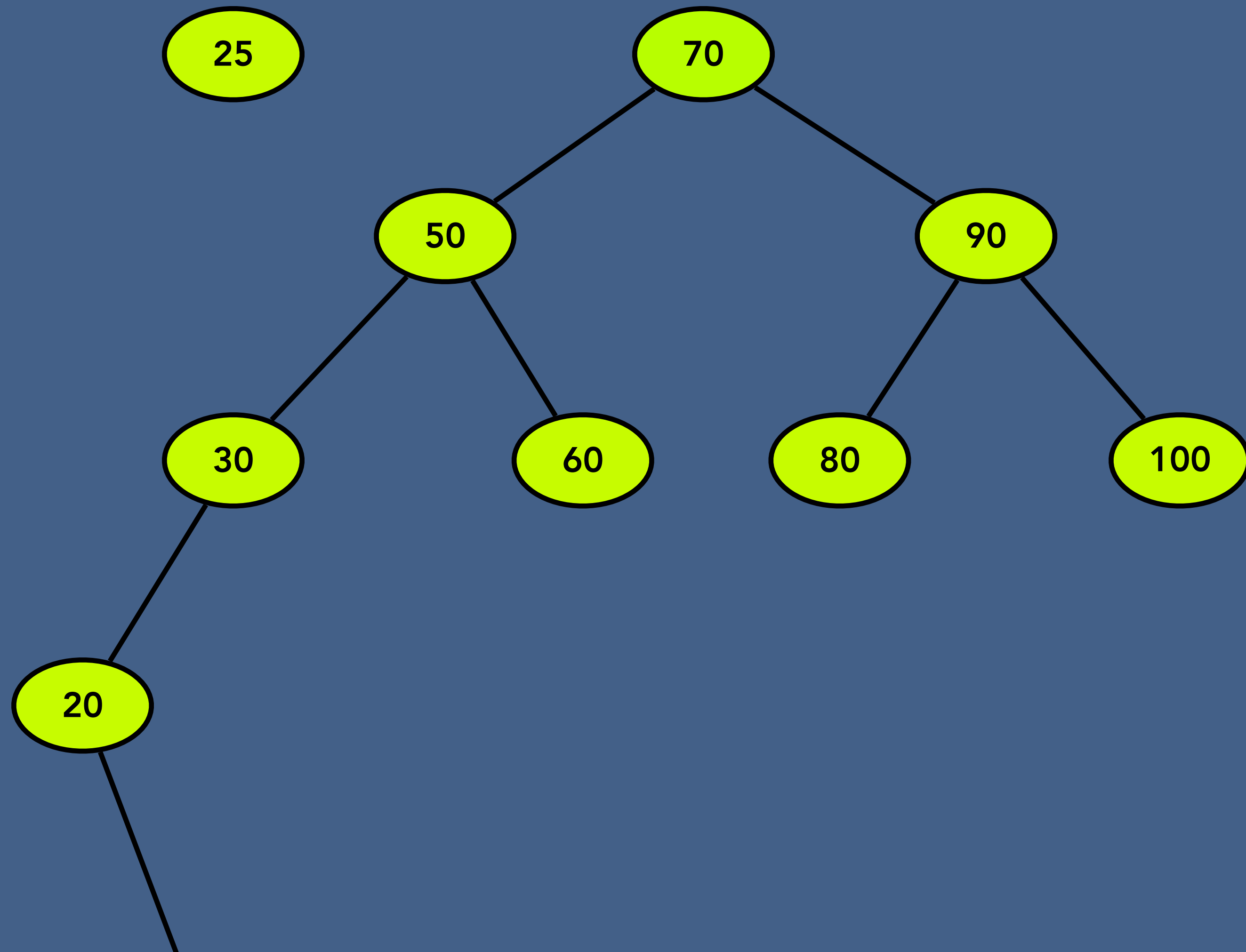
Time complexity : $O(1)$
Space complexity : $O(1)$



AVL Tree - Insert a Node

Case 2 : Rotation is required

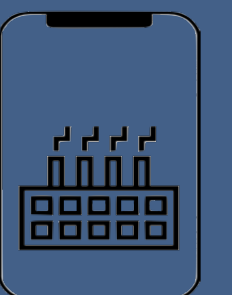
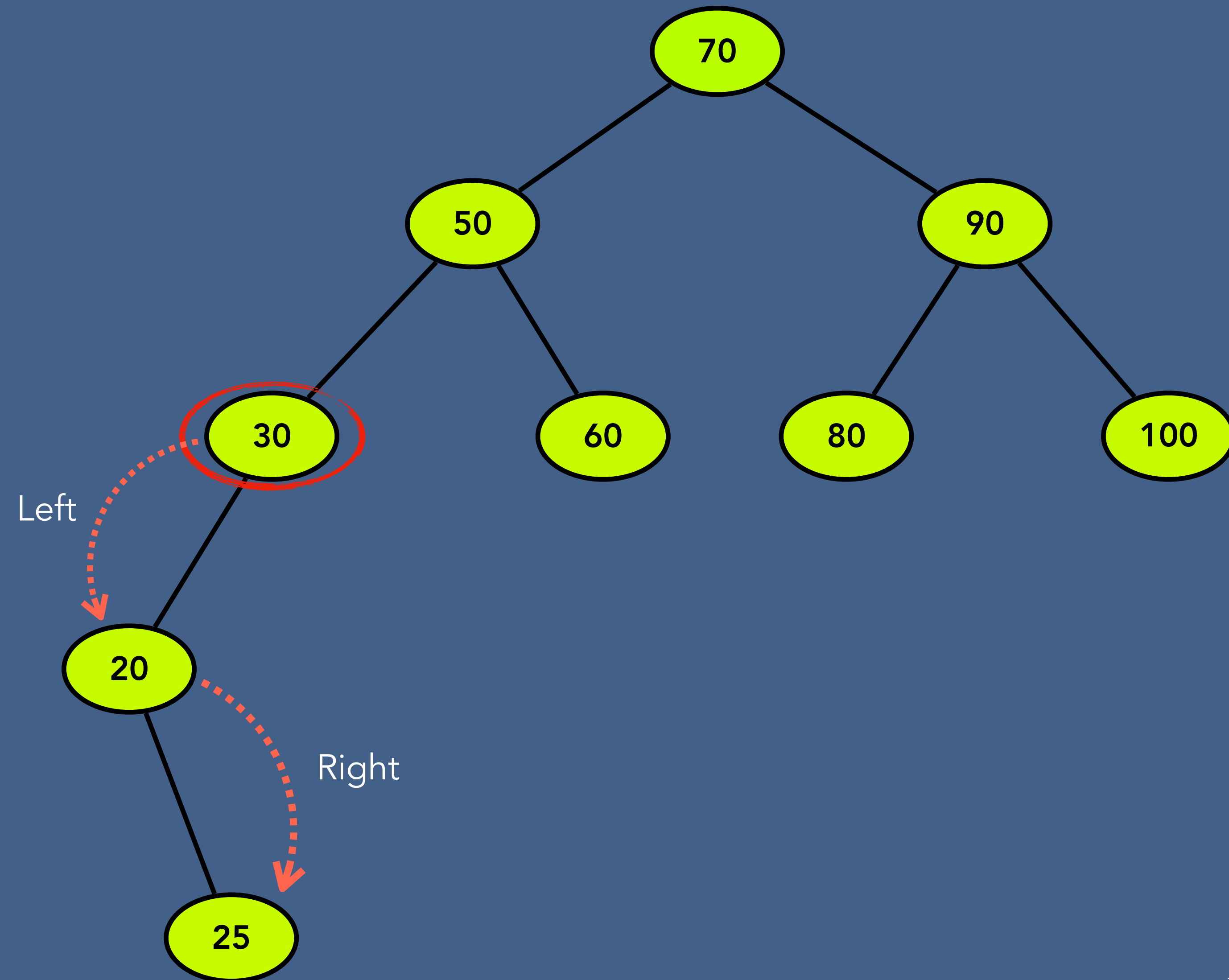
LR - left right condition



AVL Tree - Insert a Node

Case 2 : Rotation is required

LR - left right condition

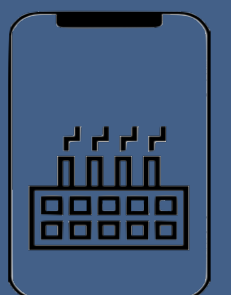
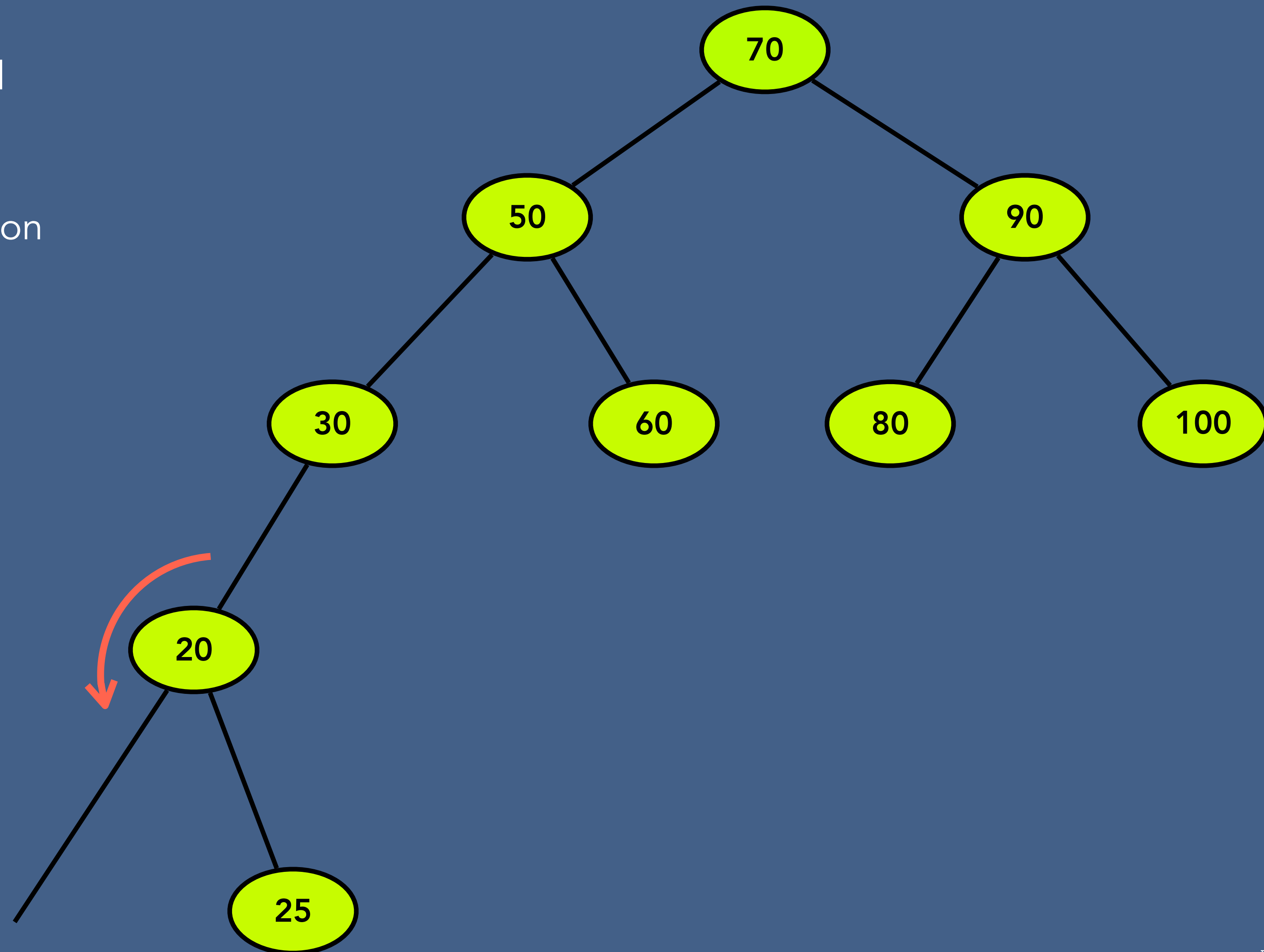


AVL Tree - Insert a Node

Case 2 : Rotation is required

LR - left right condition

1. Left rotation
2. Right rotation

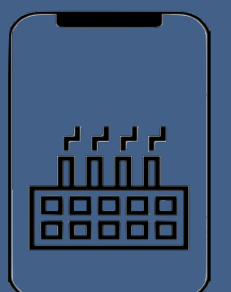
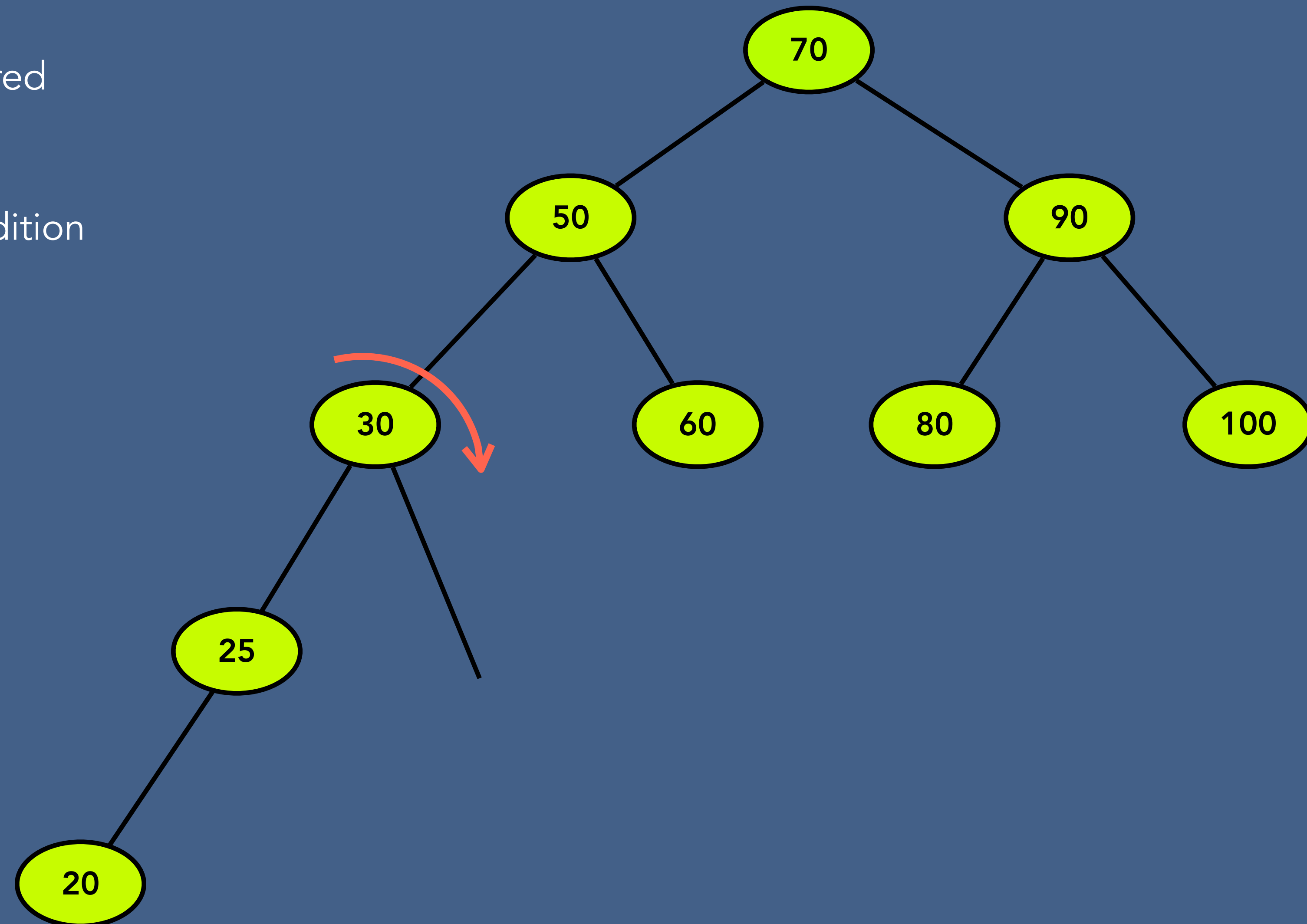


AVL Tree - Insert a Node

Case 2 : Rotation is required

LR - left right condition

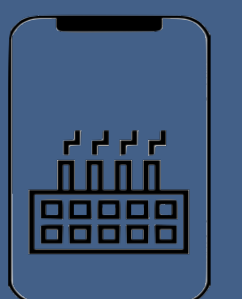
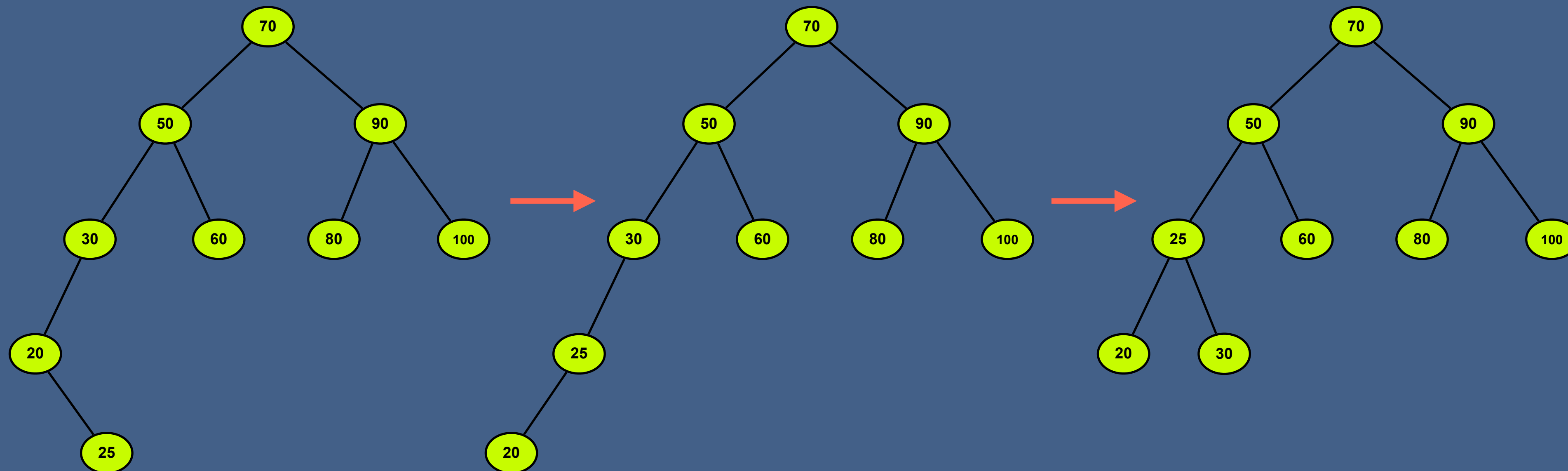
1. Left rotation
2. Right rotation



AVL Tree - Insert a Node

Case 2 : Left Right Condition

1. Left Rotation
2. Right Rotation



AVL Tree - Insert a Node

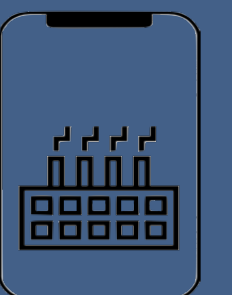
Case 2 : Left Right Condition

Step 1 : rotate Left disbalancedNode.leftChild

Step 2 : rotate Right disbalancedNode

```
rotateLeft(disbalancedNode)
  newRoot = disbalancedNode.rightChild
  disbalancedNode.rightChild = disbalancedNode.rightChild.leftChild
  newRoot.leftChild = disbalancedNode
  update height of disbalancedNode and newRoot
  return newRoot
```

```
rotateRight(disbalancedNode)
  newRoot = disbalancedNode.leftChild
  disbalancedNode.leftChild = disbalancedNode.leftChild.rightChild
  newRoot.rightChild = disbalancedNode
  update height of disbalancedNode and newRoot
  return newRoot
```



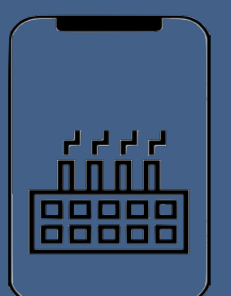
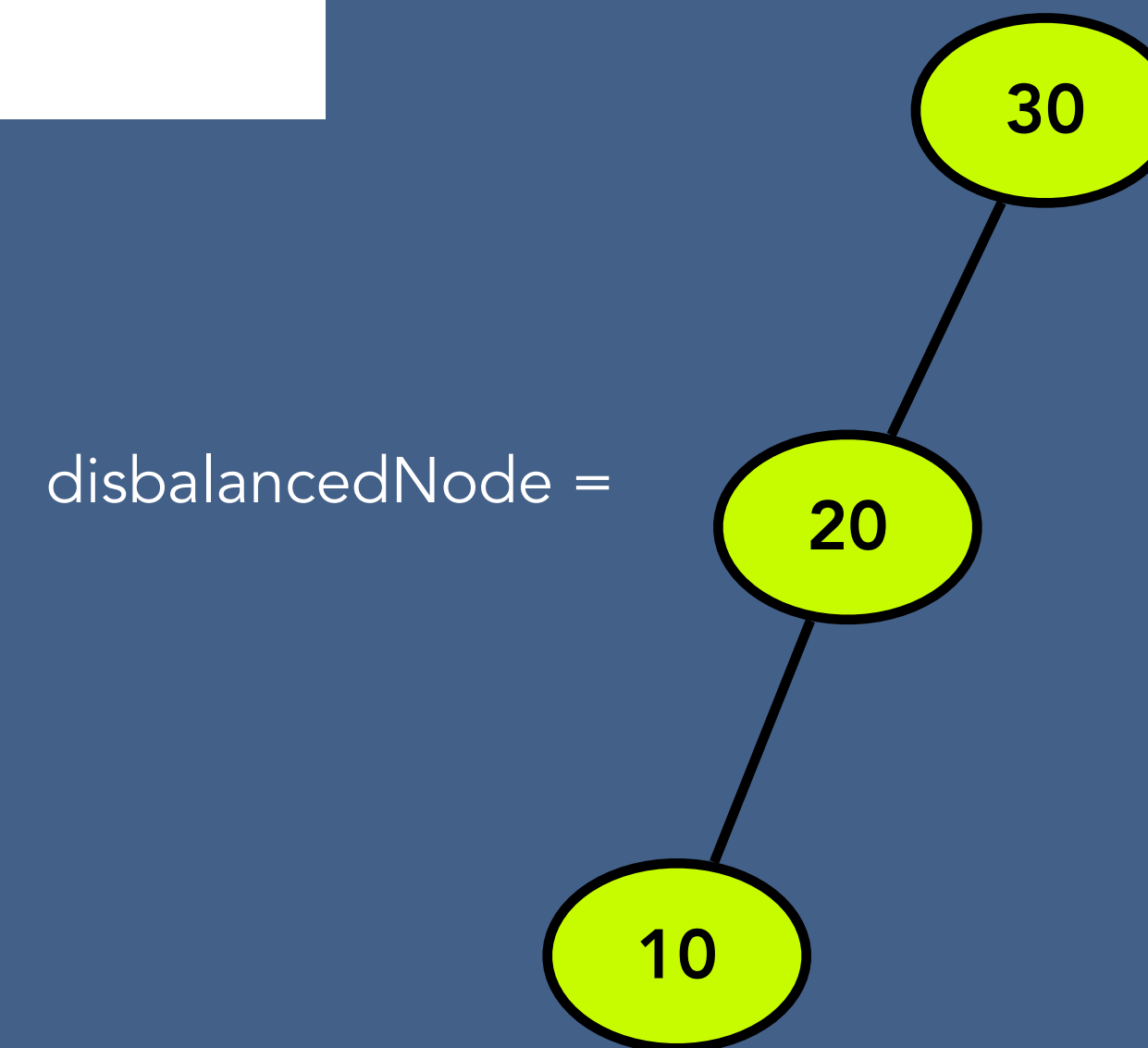
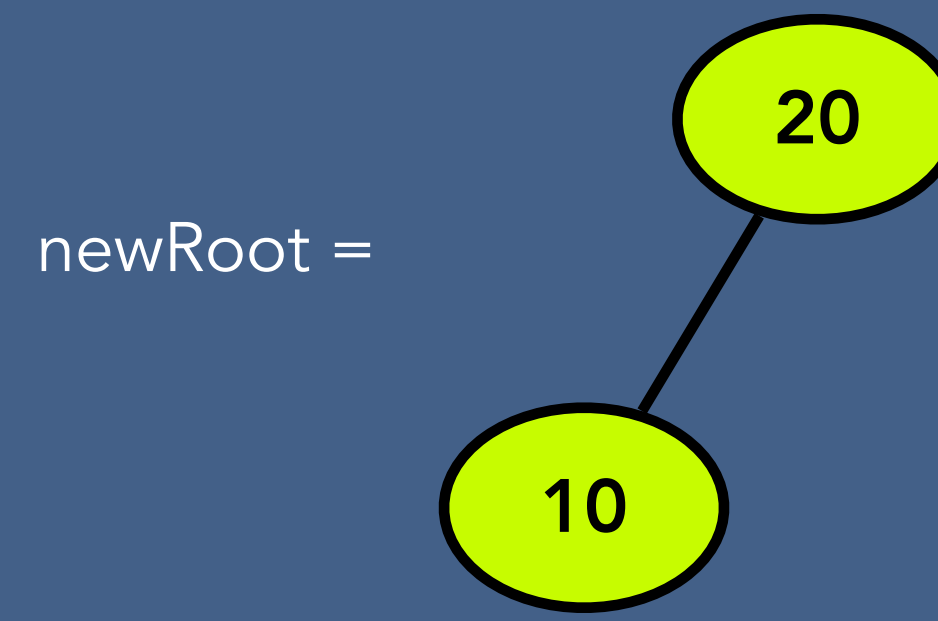
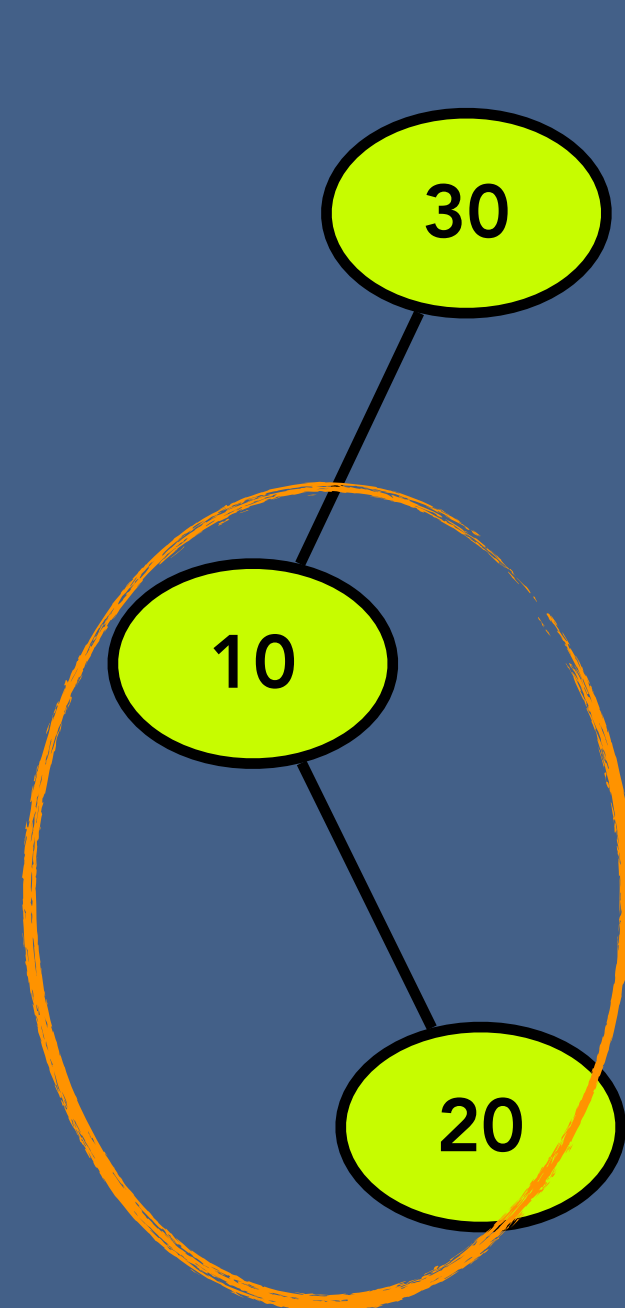
AVL Tree - Insert a Node

Case 2 : Left Right Condition

Step 1 : rotate Left disbalancedNode.leftChild

Step 2 : rotate Right disbalancedNode

```
rotateLeft(disbalancedNode)
  newRoot = disbalancedNode.rightChild
  disbalancedNode.rightChild = disbalancedNode.rightChild.leftChild
  newRoot.leftChild = disbalancedNode
  update height of disbalancedNode and newRoot
  return newRoot
```



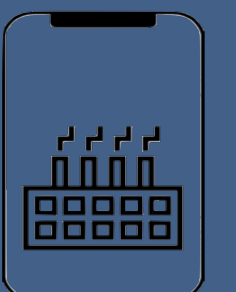
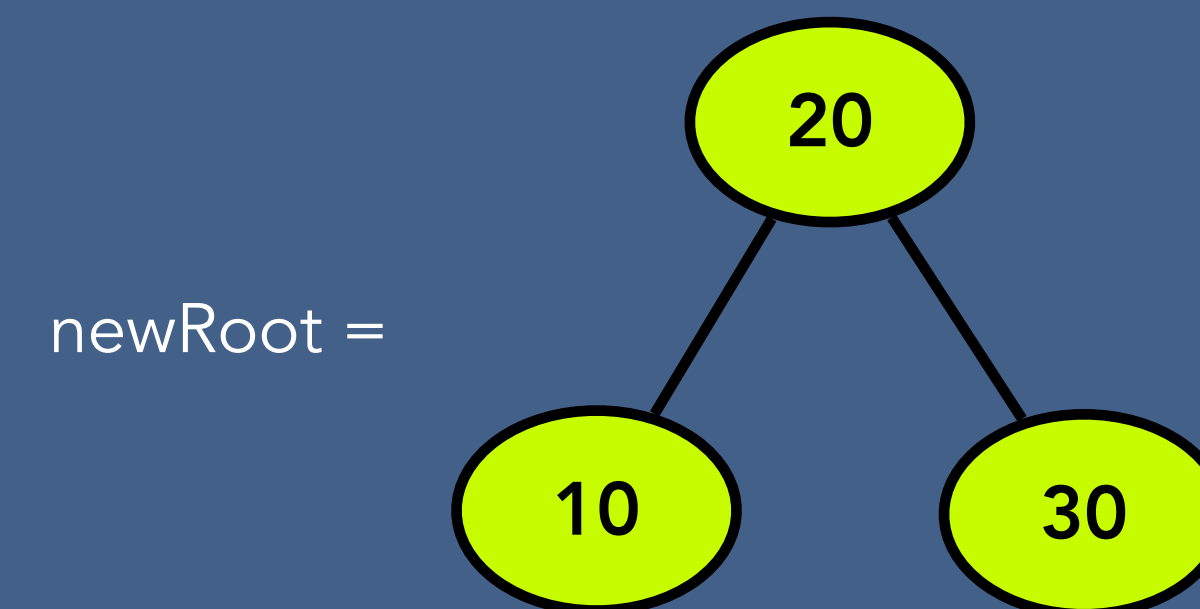
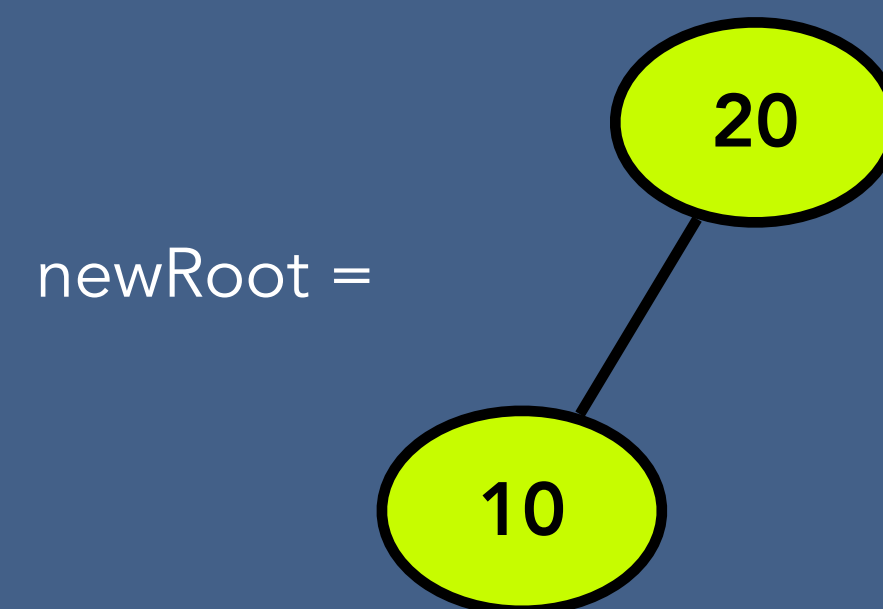
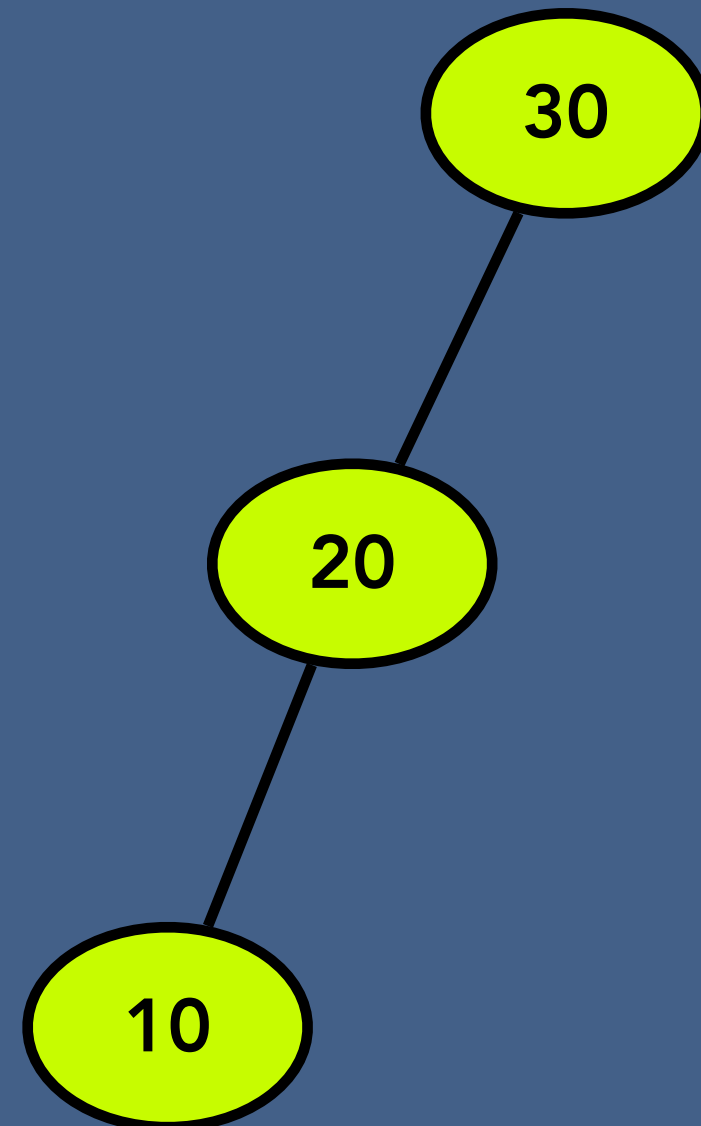
AVL Tree - Insert a Node

Case 2 : Left Right Condition

Step 1 : rotate Left disbalancedNode.leftChild

Step 2 : rotate Right disbalancedNode

```
rotateRight(disbalancedNode)
  newRoot = disbalancedNode.leftChild
  disbalancedNode.leftChild = disbalancedNode.leftChild.rightChild
  newRoot.rightChild = disbalancedNode
  update height of disbalancedNode and newRoot
  return newRoot
```



AVL Tree - Insert a Node

Case 2 : Left Right Condition

Step 1 : rotate Left `disbalancedNode.leftChild`

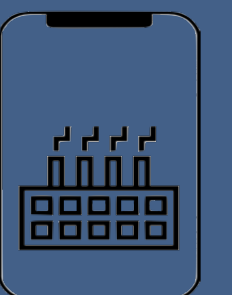
Step 2 : rotate Right `disbalancedNode`

```
rotateLeft(disbalancedNode)
  newRoot = disbalancedNode.rightChild
  disbalancedNode.rightChild = disbalancedNode.rightChild.leftChild
  newRoot.leftChild = disbalancedNode
  update height of disbalancedNode and newRoot
  return newRoot
```

```
rotateRight(disbalancedNode)
  newRoot = disbalancedNode.leftChild
  disbalancedNode.leftChild = disbalancedNode.leftChild.rightChild
  newRoot.rightChild = disbalancedNode
  update height of disbalancedNode and newRoot
  return newRoot
```

Time complexity : $O(1)$

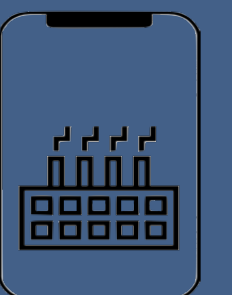
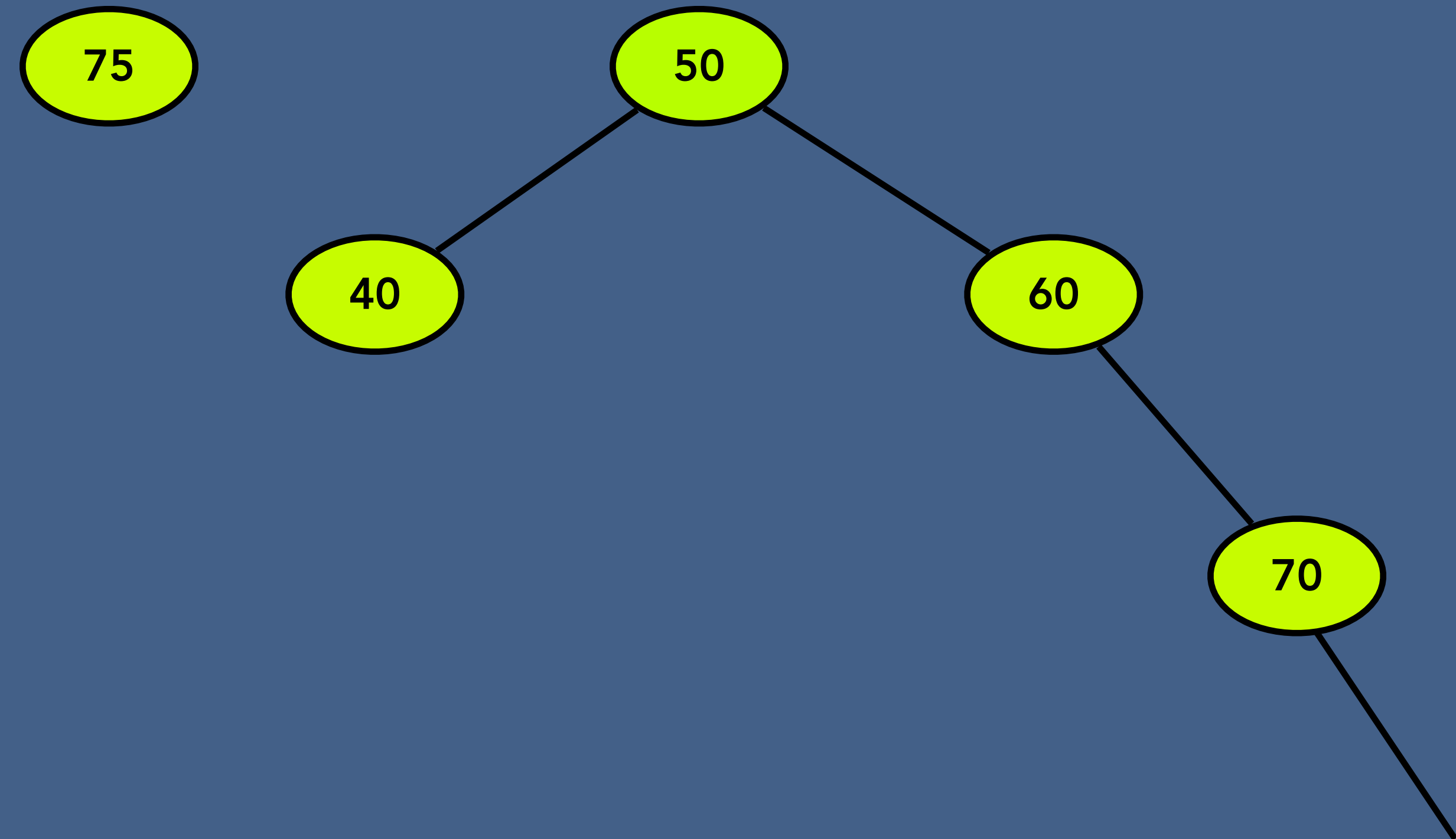
Space complexity : $O(1)$



AVL Tree - Insert a Node

Case 2 : Rotation is required

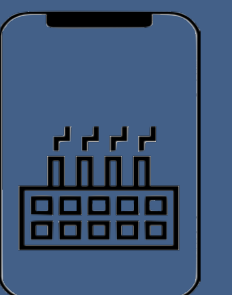
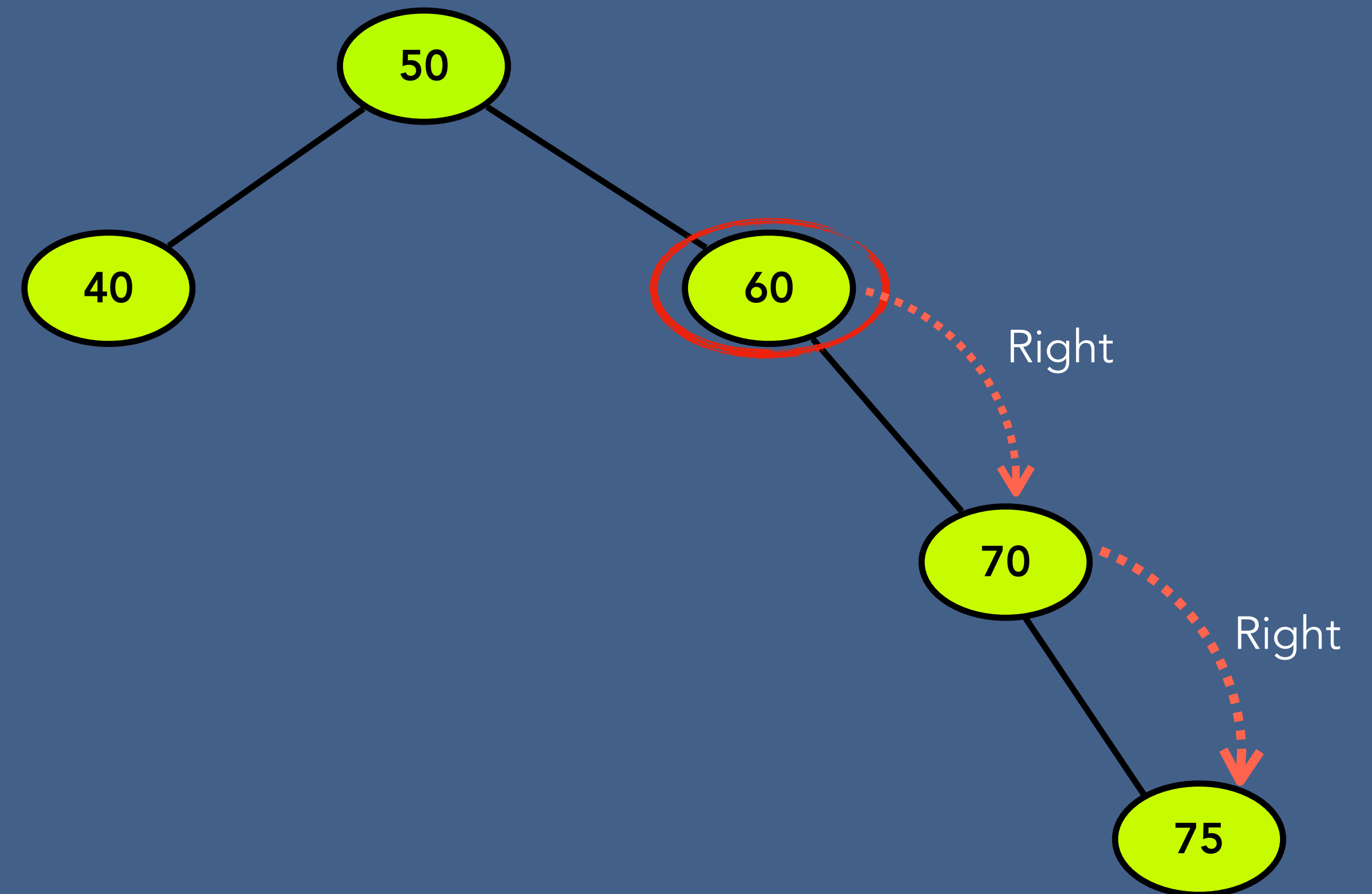
RR - right right condition



AVL Tree - Insert a Node

Case 2 : Rotation is required

RR - right right condition

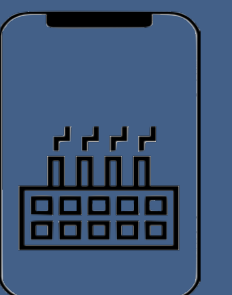
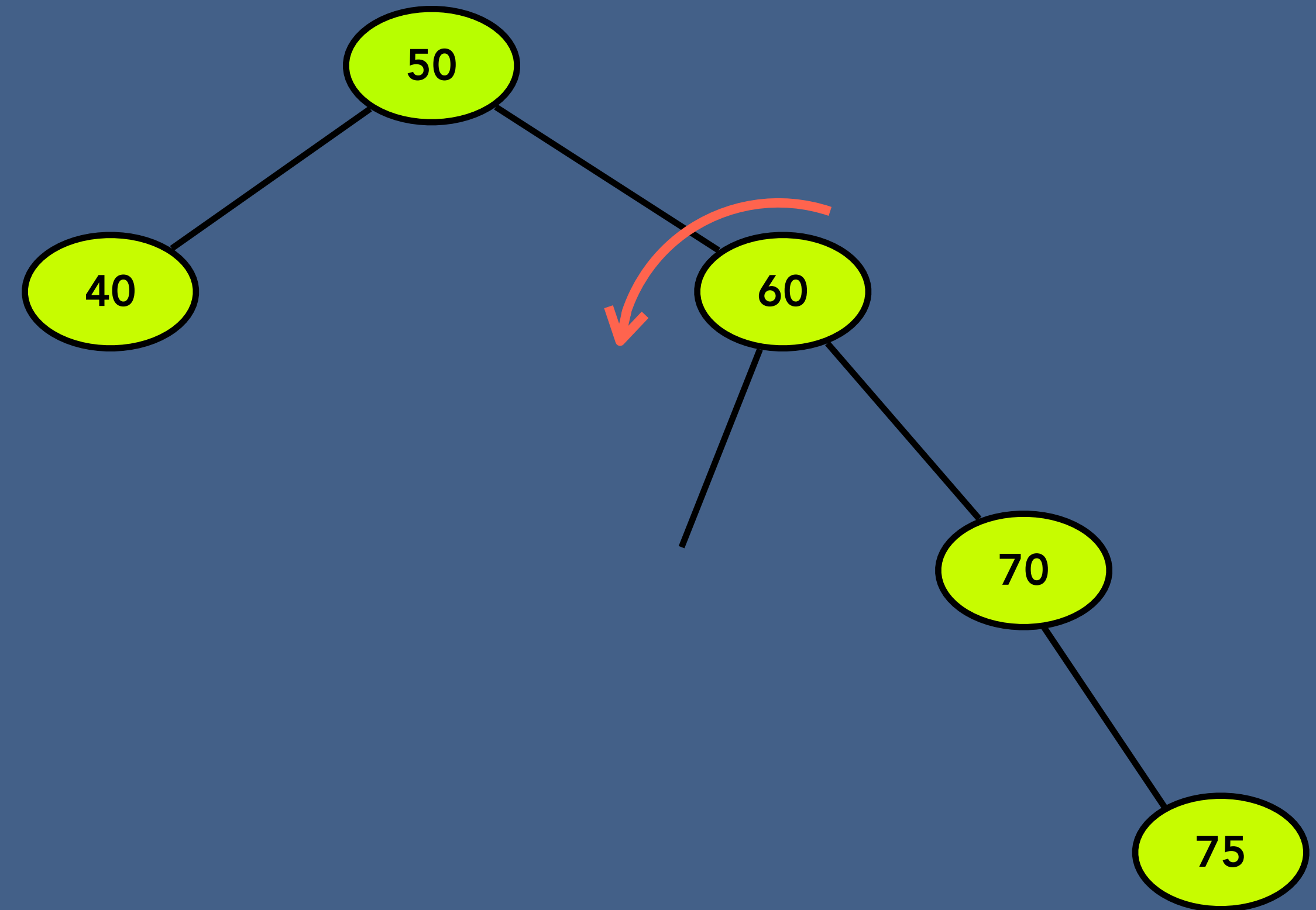


AVL Tree - Insert a Node

Case 2 : Rotation is required

RR - right right condition

Left Rotation

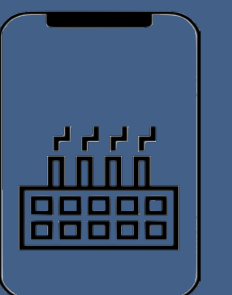
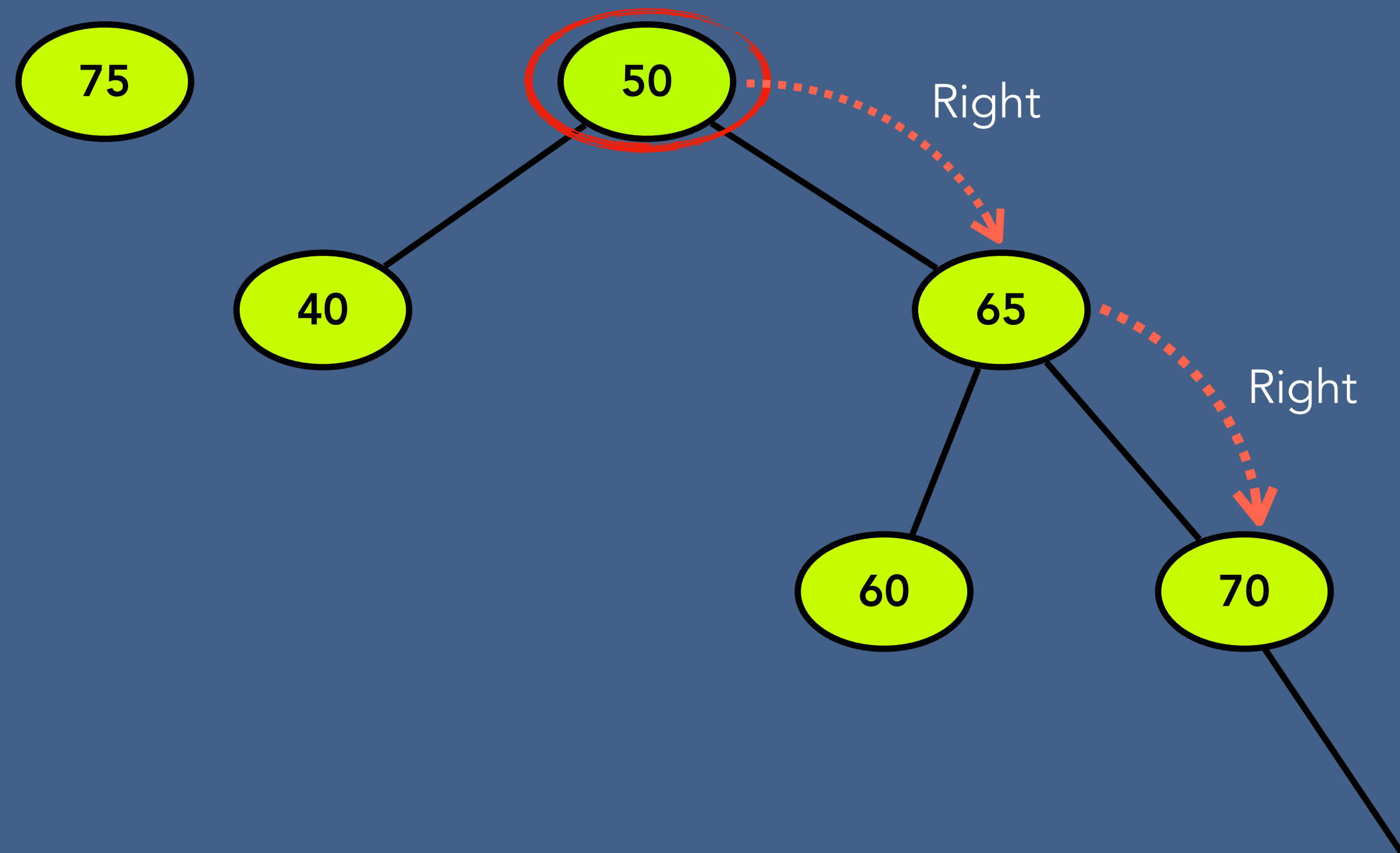


AVL Tree - Insert a Node

Case 2 : Rotation is required

RR - right right condition

Left Rotation - example 2

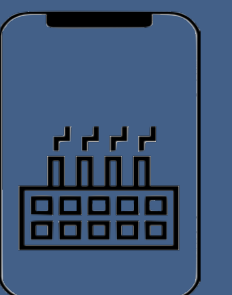
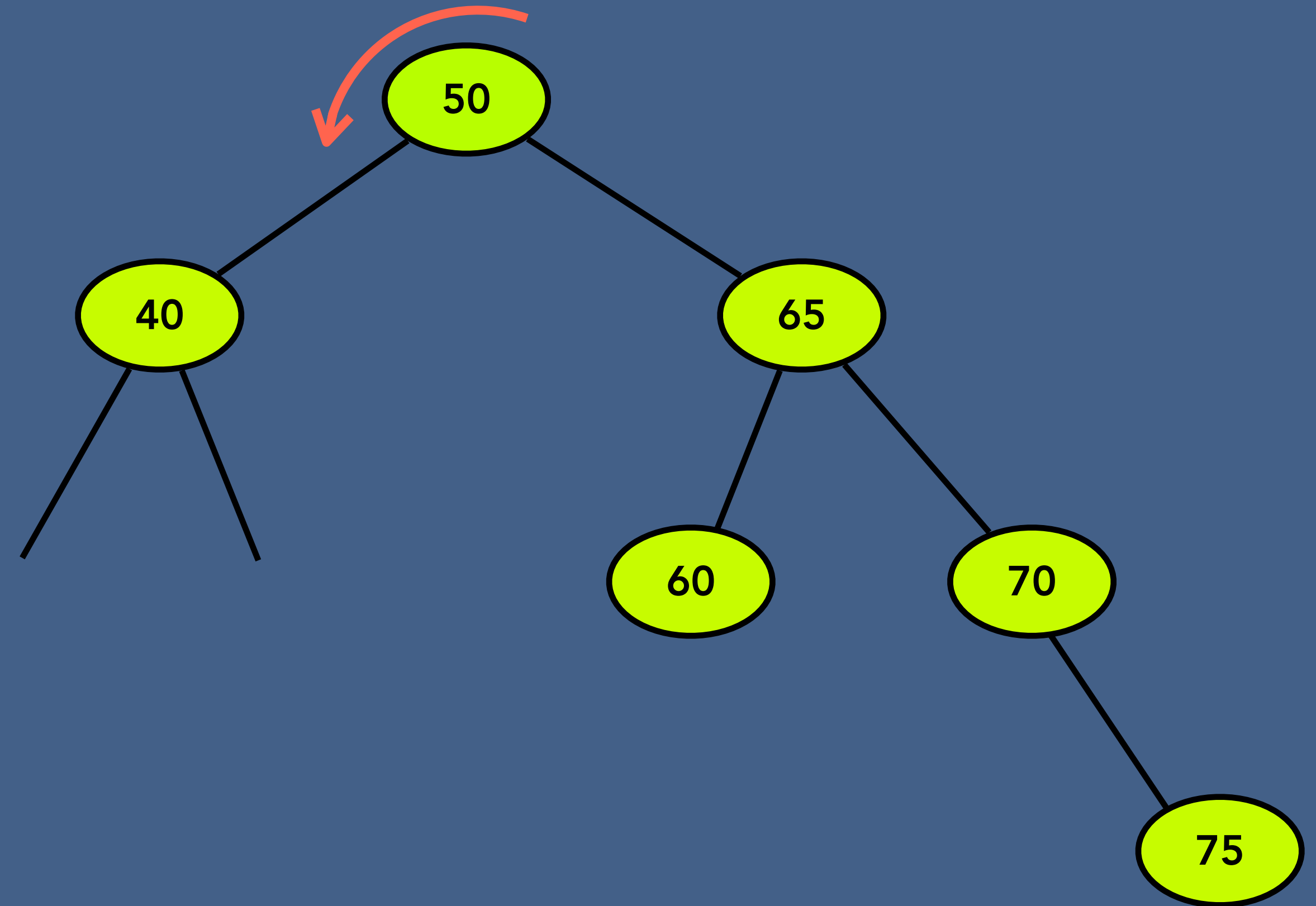


AVL Tree - Insert a Node

Case 2 : Rotation is required

RR - right right condition

Left Rotation - example 2

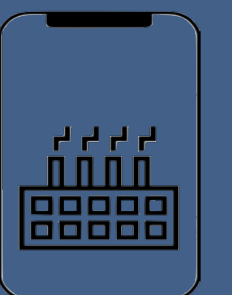
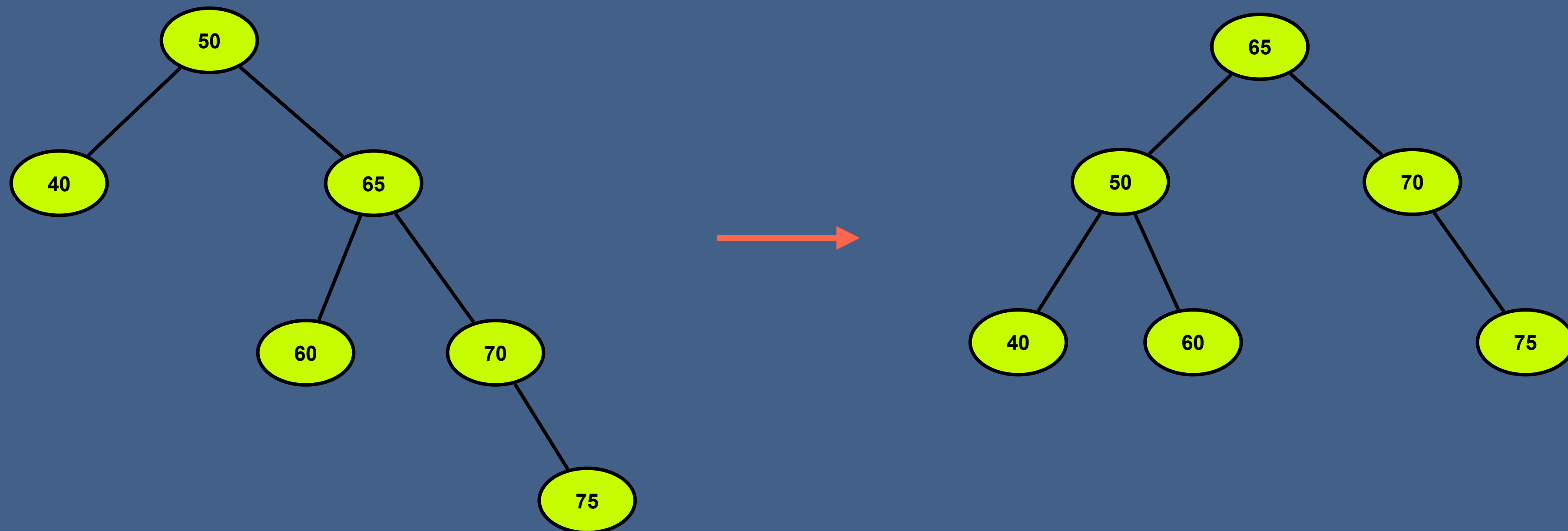


AVL Tree - Insert a Node

Case 2 : Rotation is required

RR - right right condition

Left Rotation



AVL Tree - Insert a Node

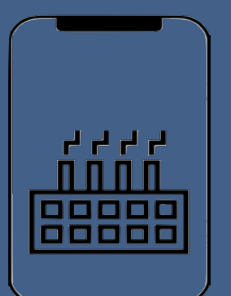
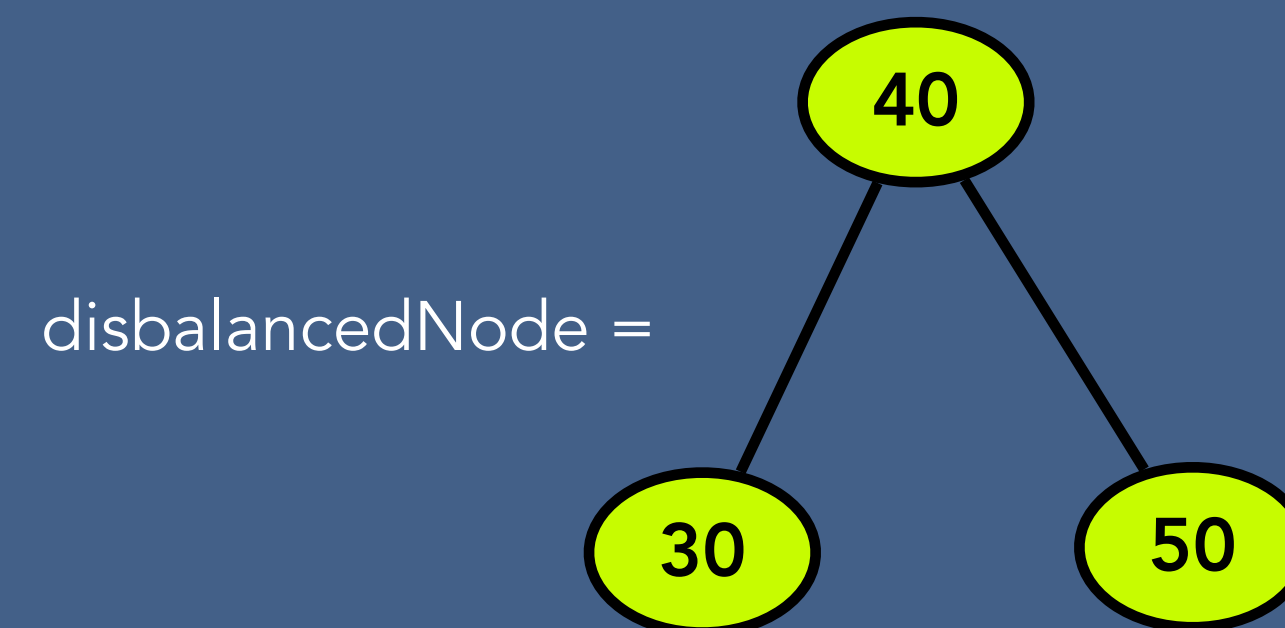
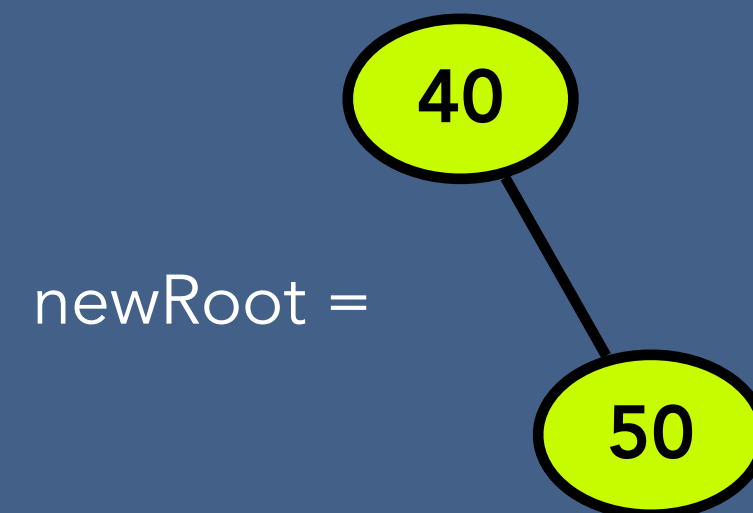
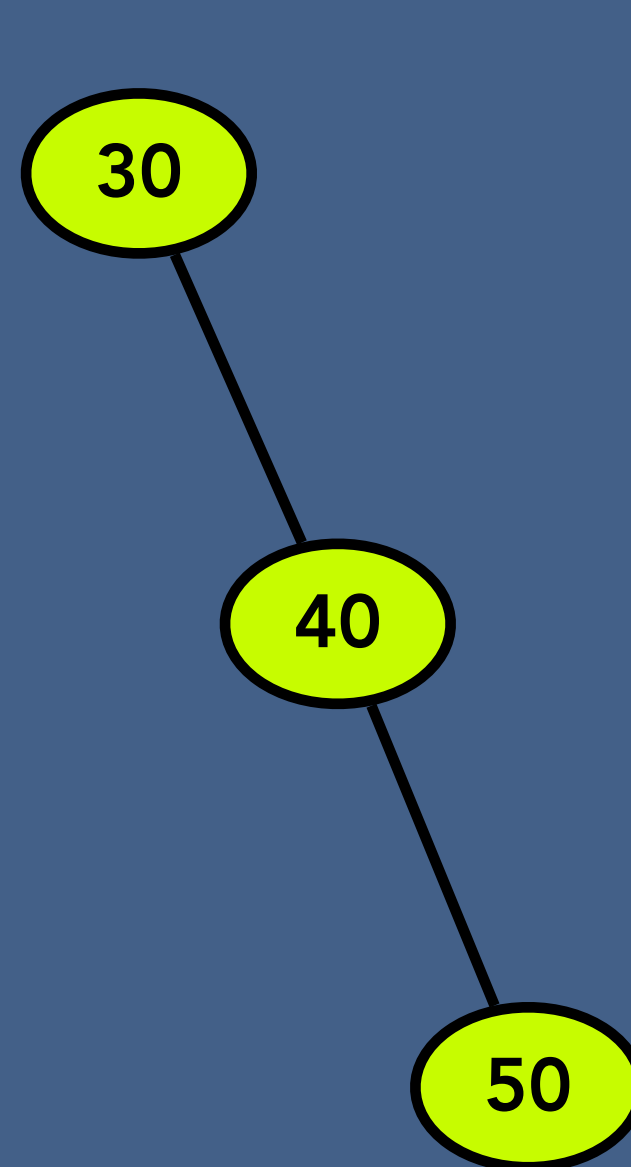
RR - right right condition

- rotate Left disbalancedNode

```
rotateLeft(disbalancedNode)
  newRoot = disbalancedNode.rightChild
  disbalancedNode.rightChild = disbalancedNode.rightChild.leftChild
  newRoot.leftChild = disbalancedNode
  update height of disbalancedNode and newRoot
  return newRoot
```

Time complexity : $O(1)$

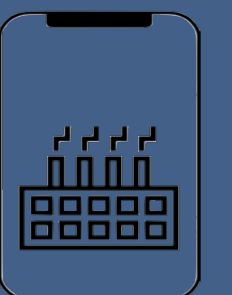
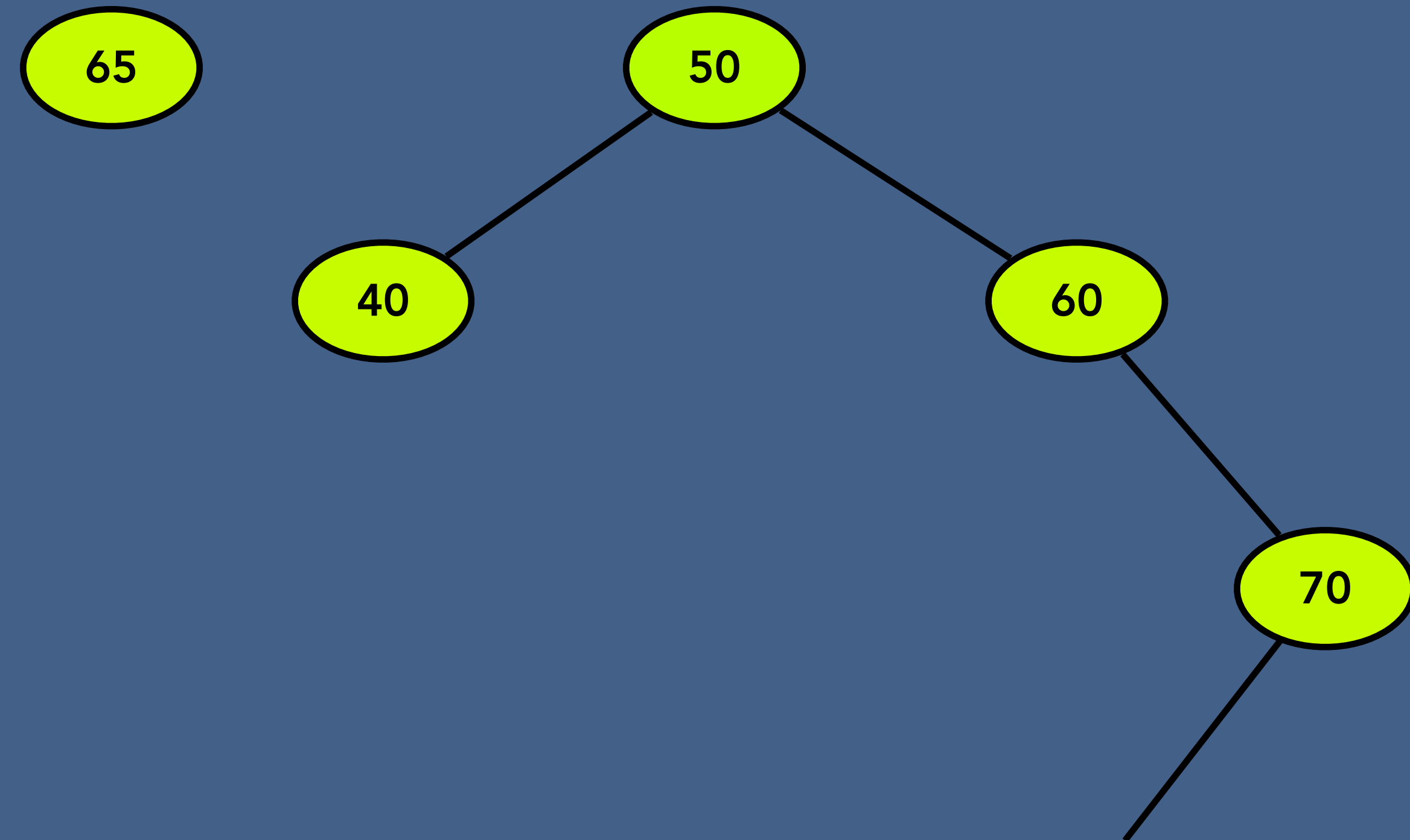
Space complexity : $O(1)$



AVL Tree - Insert a Node

Case 2 : Rotation is required

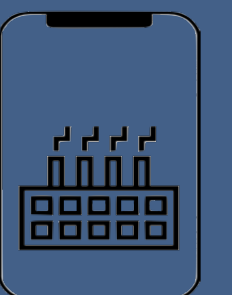
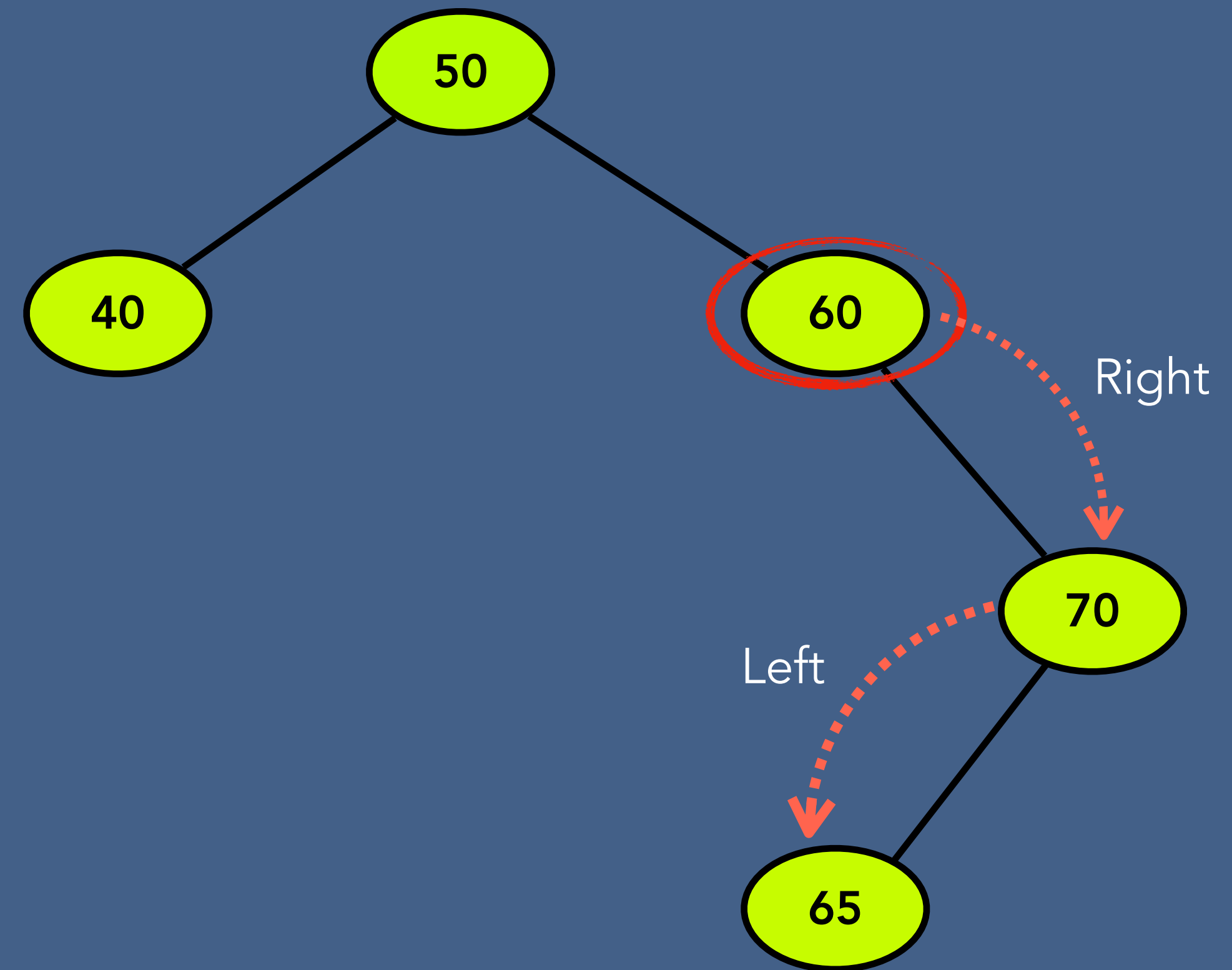
RL - right left condition



AVL Tree - Insert a Node

Case 2 : Rotation is required

RL - right left condition

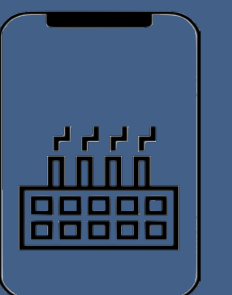
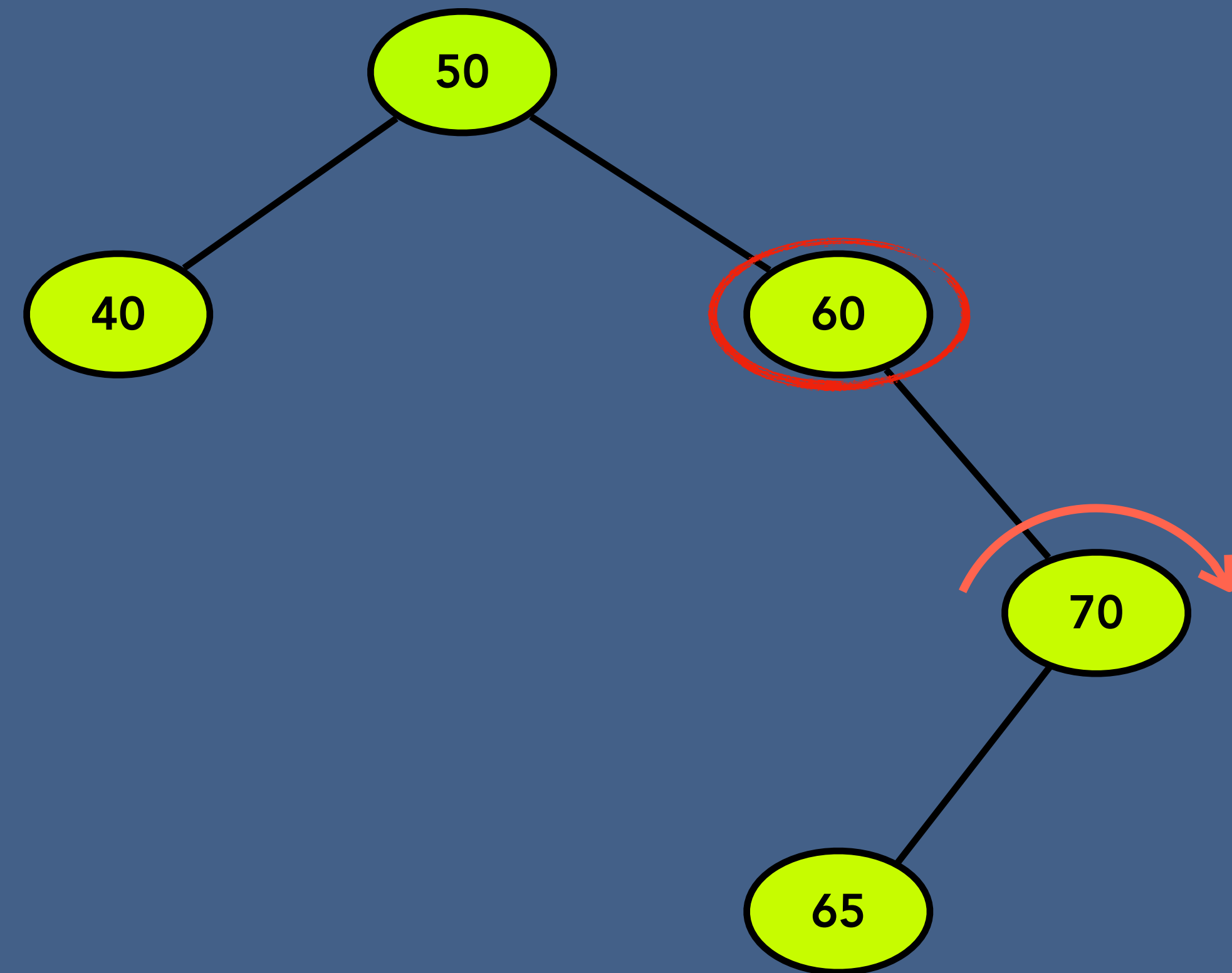


AVL Tree - Insert a Node

Case 2 : Rotation is required

RL - right left condition

1. Right rotation
2. Left rotation

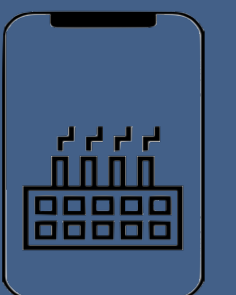
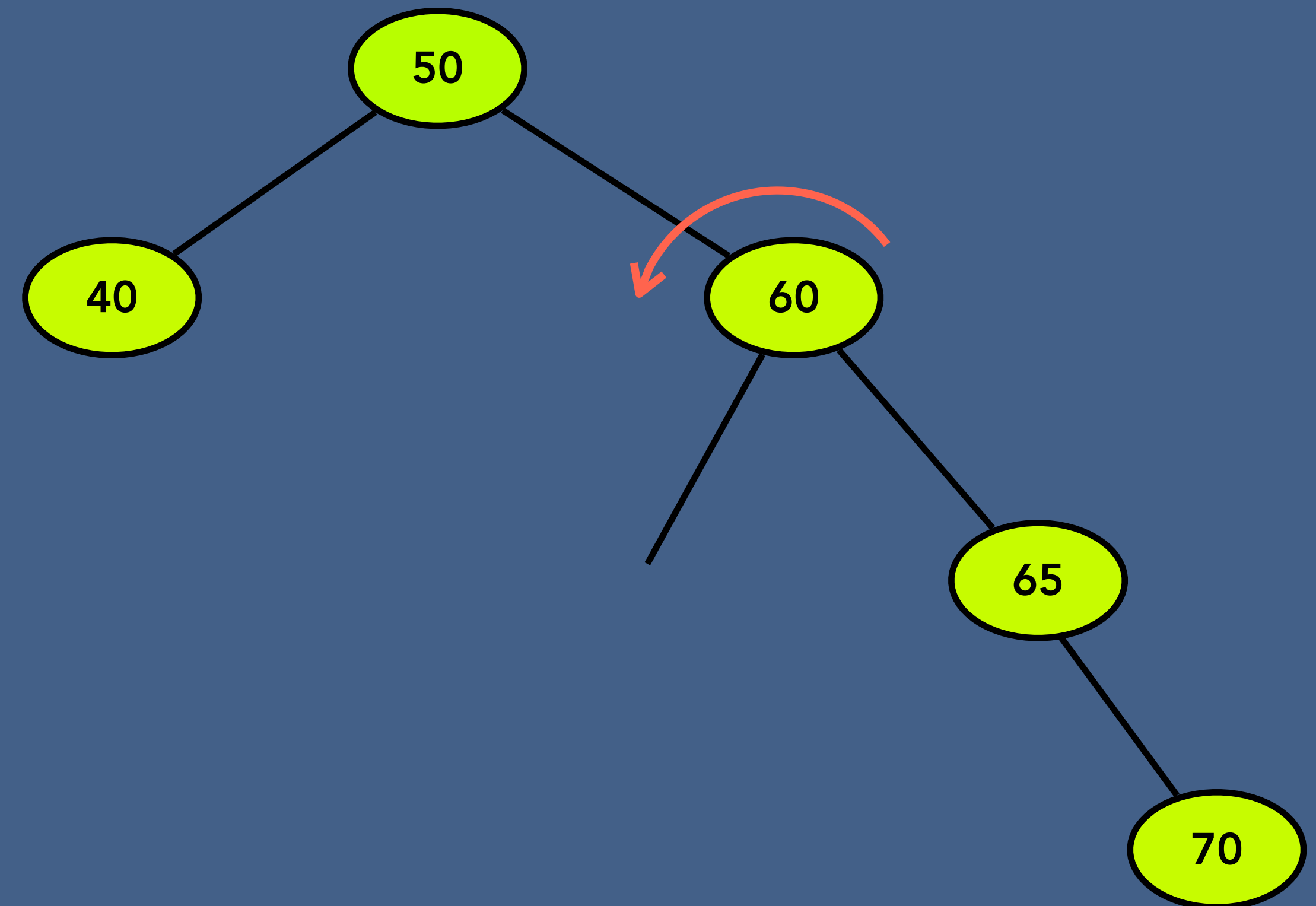


AVL Tree - Insert a Node

Case 2 : Rotation is required

RL - right left condition

1. Right rotation
2. Left rotation



AVL Tree - Insert a Node

RL - right left condition

Step 1 : rotate Right disbalancedNode.rightChild

Step 2 : rotate Left disbalancedNode

```
rotateRight(disbalancedNode)
```

```
    newRoot = disbalancedNode.leftChild
```

```
    disbalancedNode.leftChild = disbalancedNode.leftChild.rightChild
```

```
    newRoot.rightChild = disbalancedNode
```

```
    update height of disbalancedNode and newRoot
```

```
    return newRoot
```

```
rotateLeft(disbalancedNode)
```

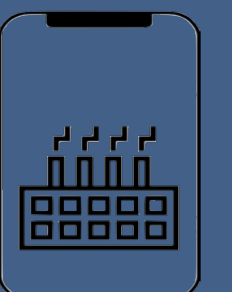
```
    newRoot = disbalancedNode.rightChild
```

```
    disbalancedNode.rightChild = disbalancedNode.rightChild.leftChild
```

```
    newRoot.leftChild = disbalancedNode
```

```
    update height of disbalancedNode and newRoot
```

```
    return newRoot
```



AVL Tree - Insert a Node

RL - right left condition

Step 1 : rotate Right disbalancedNode.rightChild

Step 2 : rotate Left disbalancedNode

```
rotateRight(disbalancedNode)
```

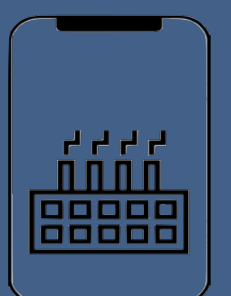
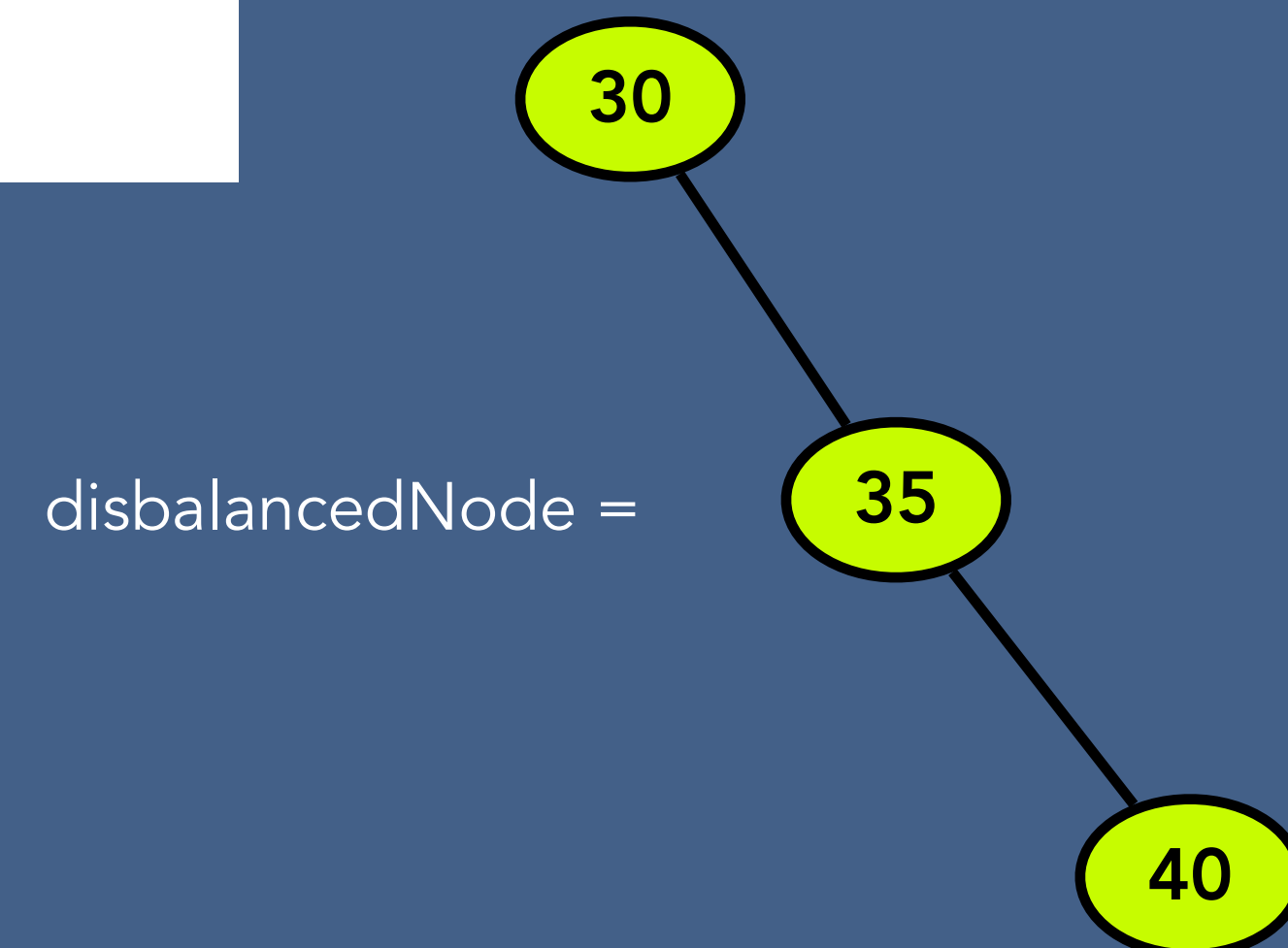
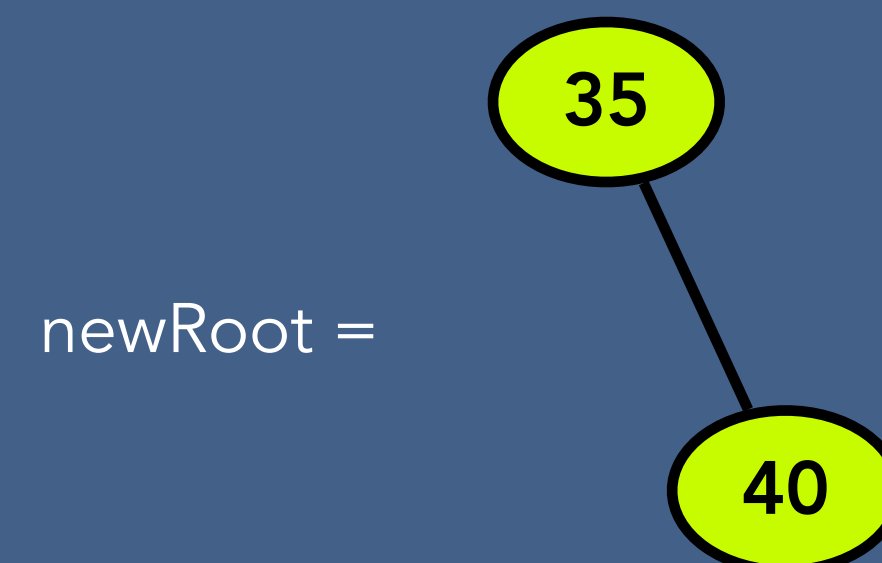
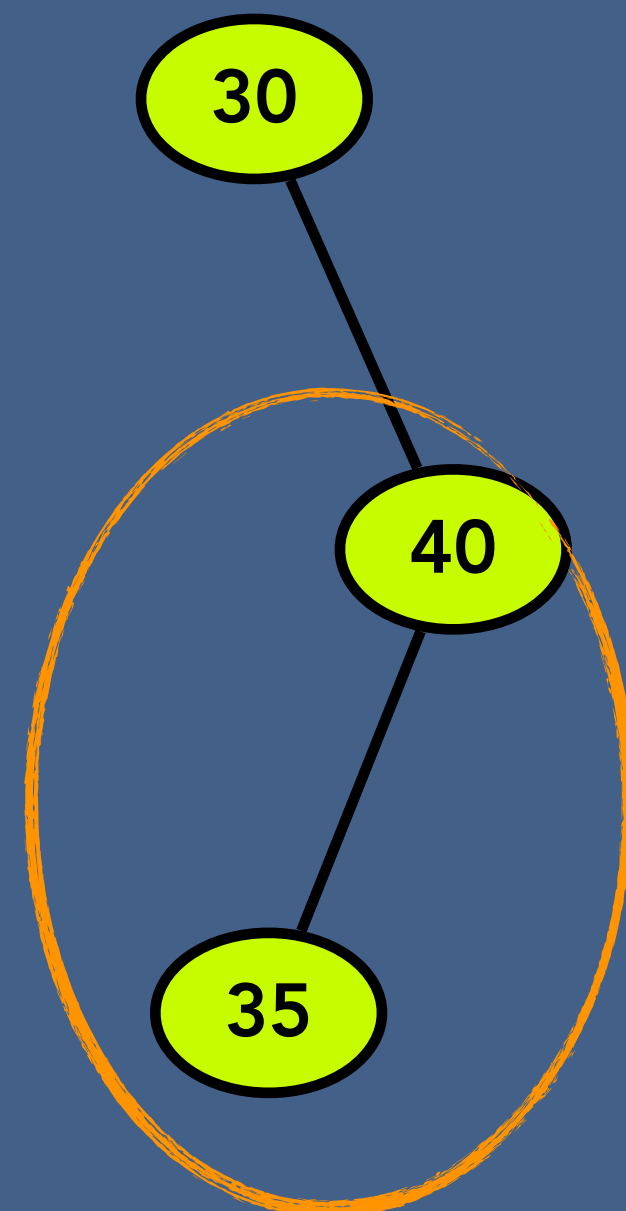
```
newRoot = disbalancedNode.leftChild
```

```
disbalancedNode.leftChild = disbalancedNode.leftChild.rightChild
```

```
newRoot.rightChild = disbalancedNode
```

```
update height of disbalancedNode and newRoot
```

```
return newRoot
```



AVL Tree - Insert a Node

RL - right left condition

Step 1 : rotate Right disbalancedNode.rightChild

Step 2 : rotate Left disbalancedNode

```
rotateLeft(disbalancedNode)
```

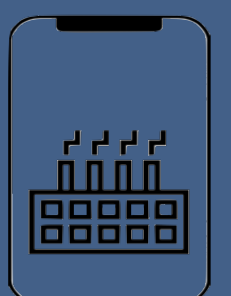
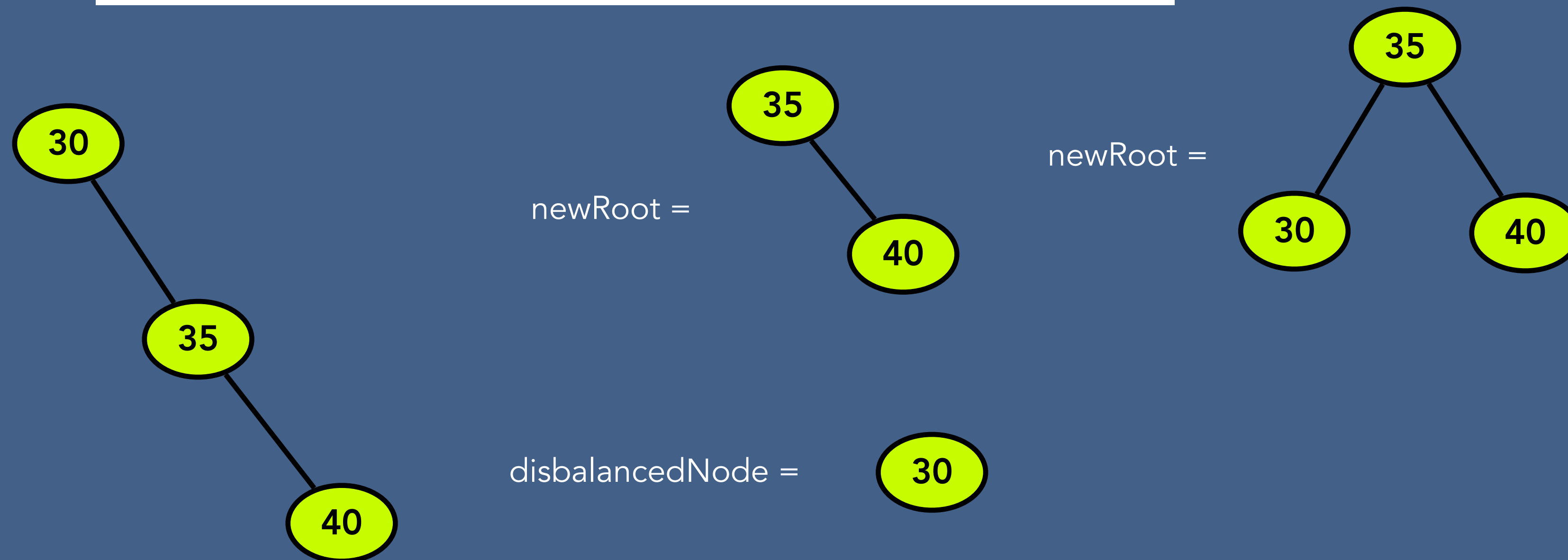
```
newRoot = disbalancedNode.rightChild
```

```
disbalancedNode.rightChild = disbalancedNode.rightChild.leftChild
```

```
newRoot.leftChild = disbalancedNode
```

```
update height of disbalancedNode and newRoot
```

```
return newRoot
```



AVL Tree - Insert a Node

RL - right left condition

Step 1 : rotate Right disbalancedNode.rightChild

Step 2 : rotate Left disbalancedNode

```
rotateRight(disbalancedNode)
```

```
newRoot = disbalancedNode.leftChild
```

```
disbalancedNode.leftChild = disbalancedNode.leftChild.rightChild
```

```
newRoot.rightChild = disbalancedNode
```

```
update height of disbalancedNode and newRoot
```

```
return newRoot
```

```
rotateLeft(disbalancedNode)
```

```
newRoot = disbalancedNode.rightChild
```

```
disbalancedNode.rightChild = disbalancedNode.rightChild.leftChild
```

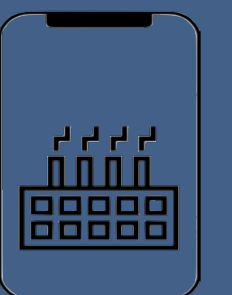
```
newRoot.leftChild = disbalancedNode
```

```
update height of disbalancedNode and newRoot
```

```
return newRoot
```

Time complexity : $O(1)$

Space complexity : $O(1)$

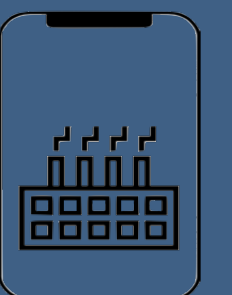


AVL Tree - Insert a Node (All together)

Case 1 : Rotation is not required

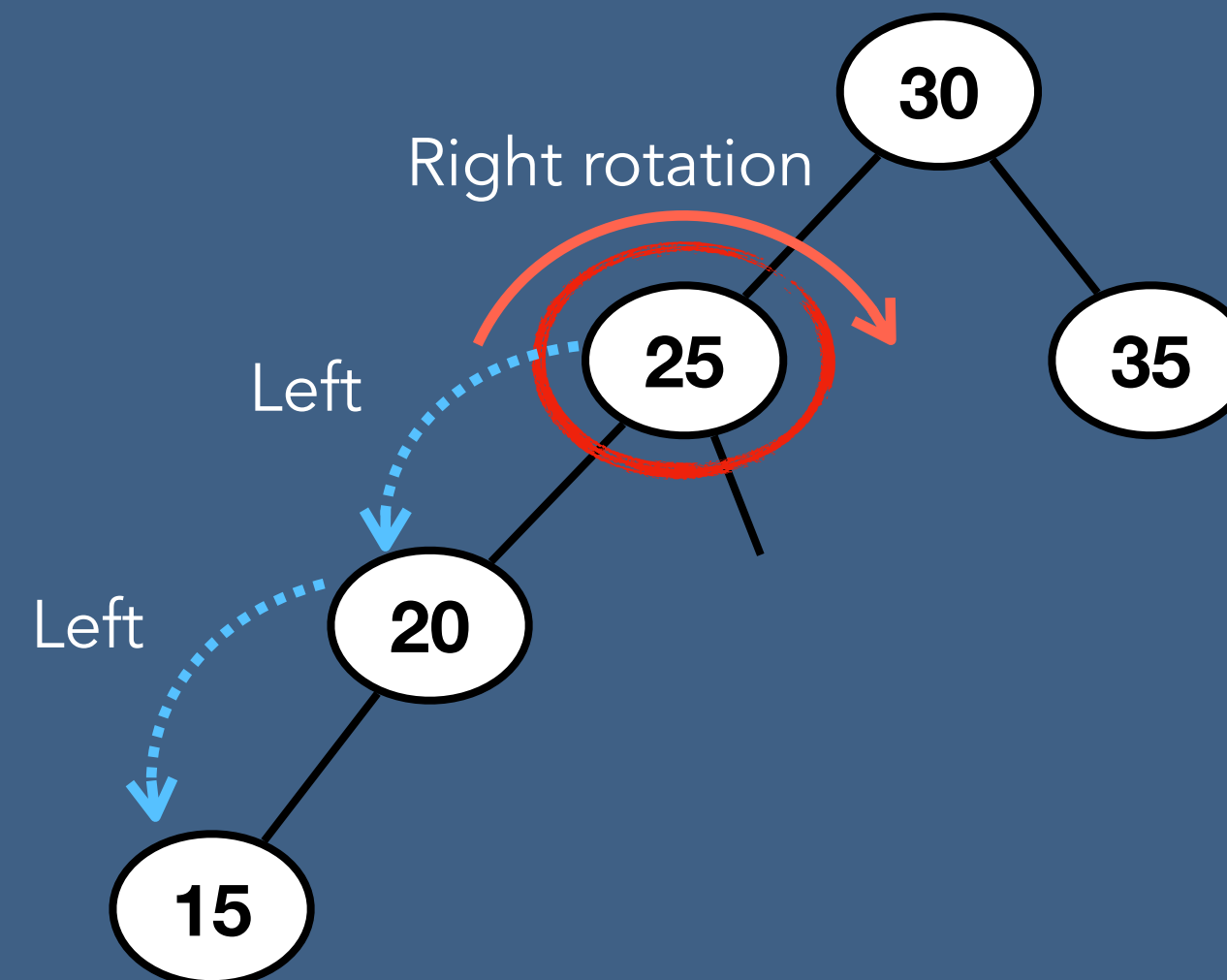
Case 2 : Rotation is required

- Left Left condition (LL)
- Left Right condition (LR)
- Right Right condition (RR)
- Right Left condition (RL)

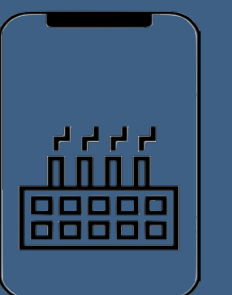


AVL Tree - Insert a Node (All together)

30, 25, 35, 20, 15, 5, 10, 50, 60, 70, 65

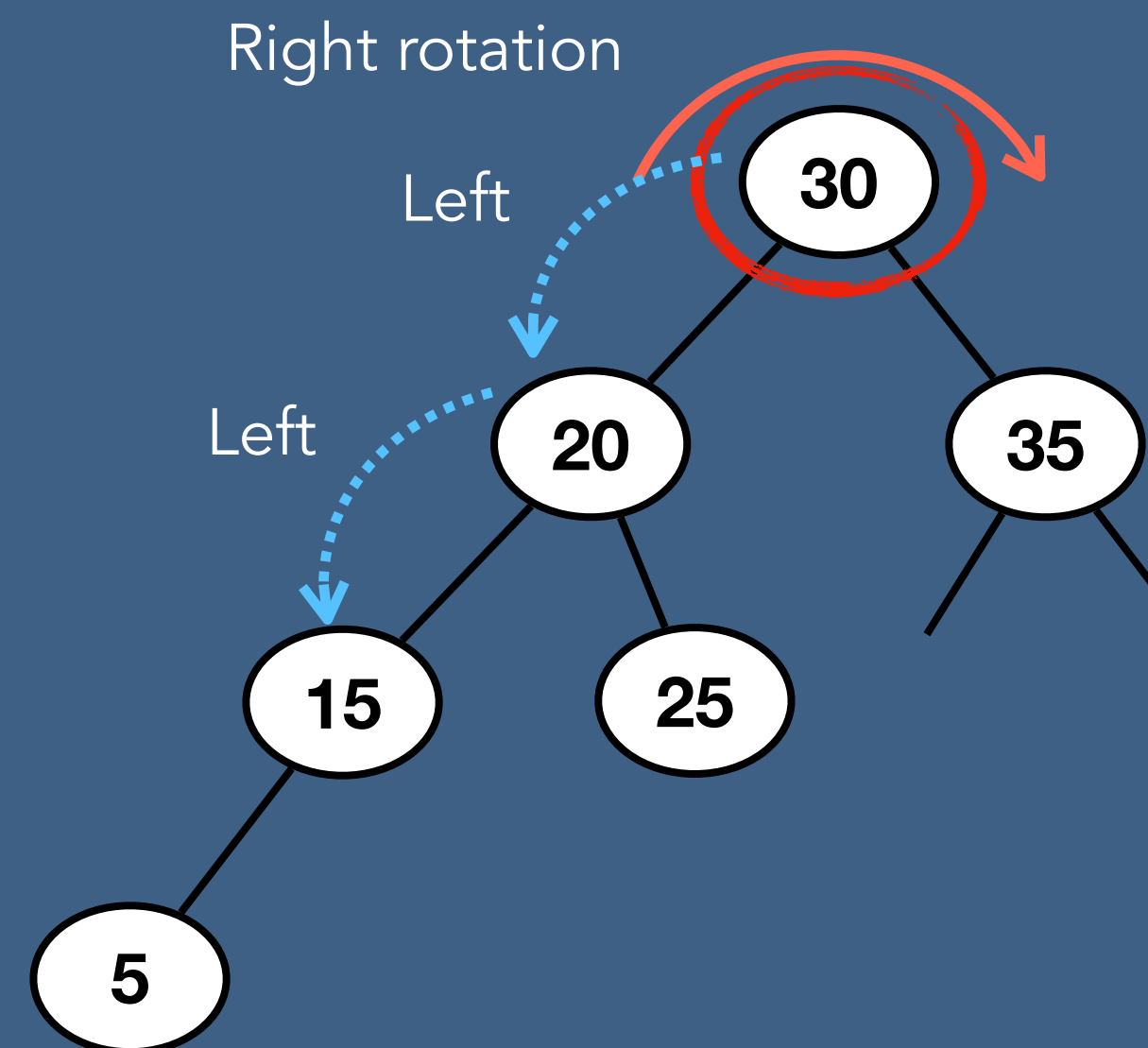


- Left Left condition (LL)
- Left Right condition (LR)
- Right Right condition (RR)
- Right Left condition (RL)

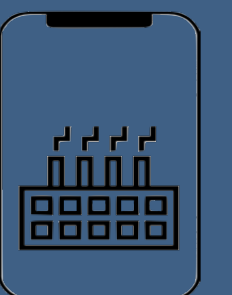


AVL Tree - Insert a Node (All together)

30,25,35,20,15,5,10,50,60,70,65

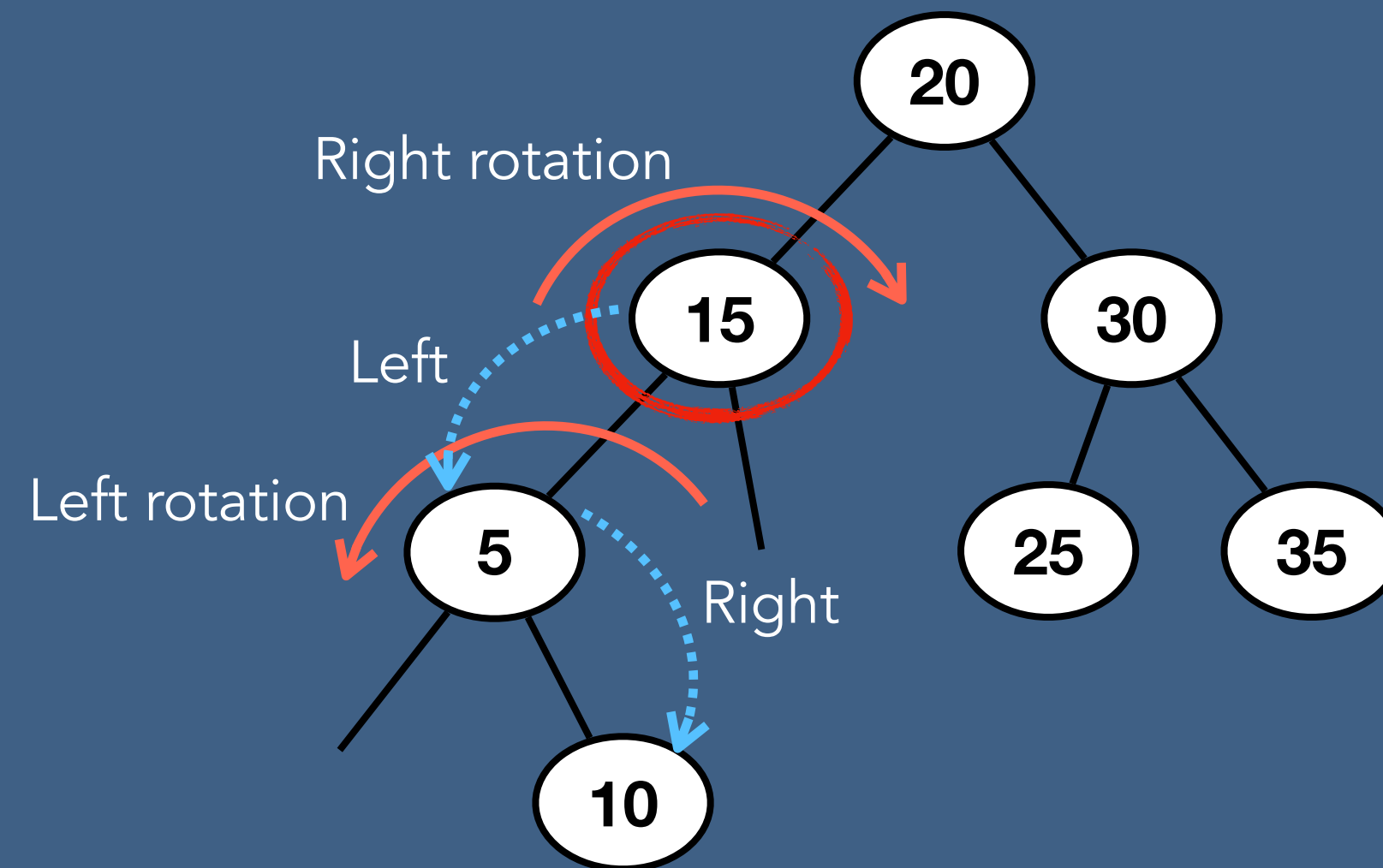


- Left Left condition (LL)
- Left Right condition (LR)
- Right Right condition (RR)
- Right Left condition (RL)

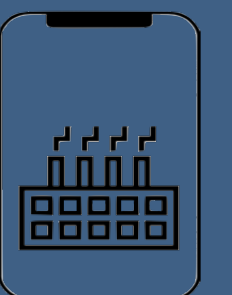


AVL Tree - Insert a Node (All together)

30,25,35,20,15,5,10,50,60,70,65



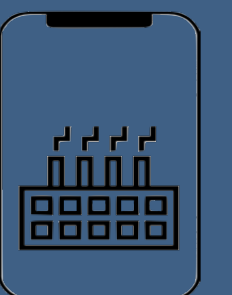
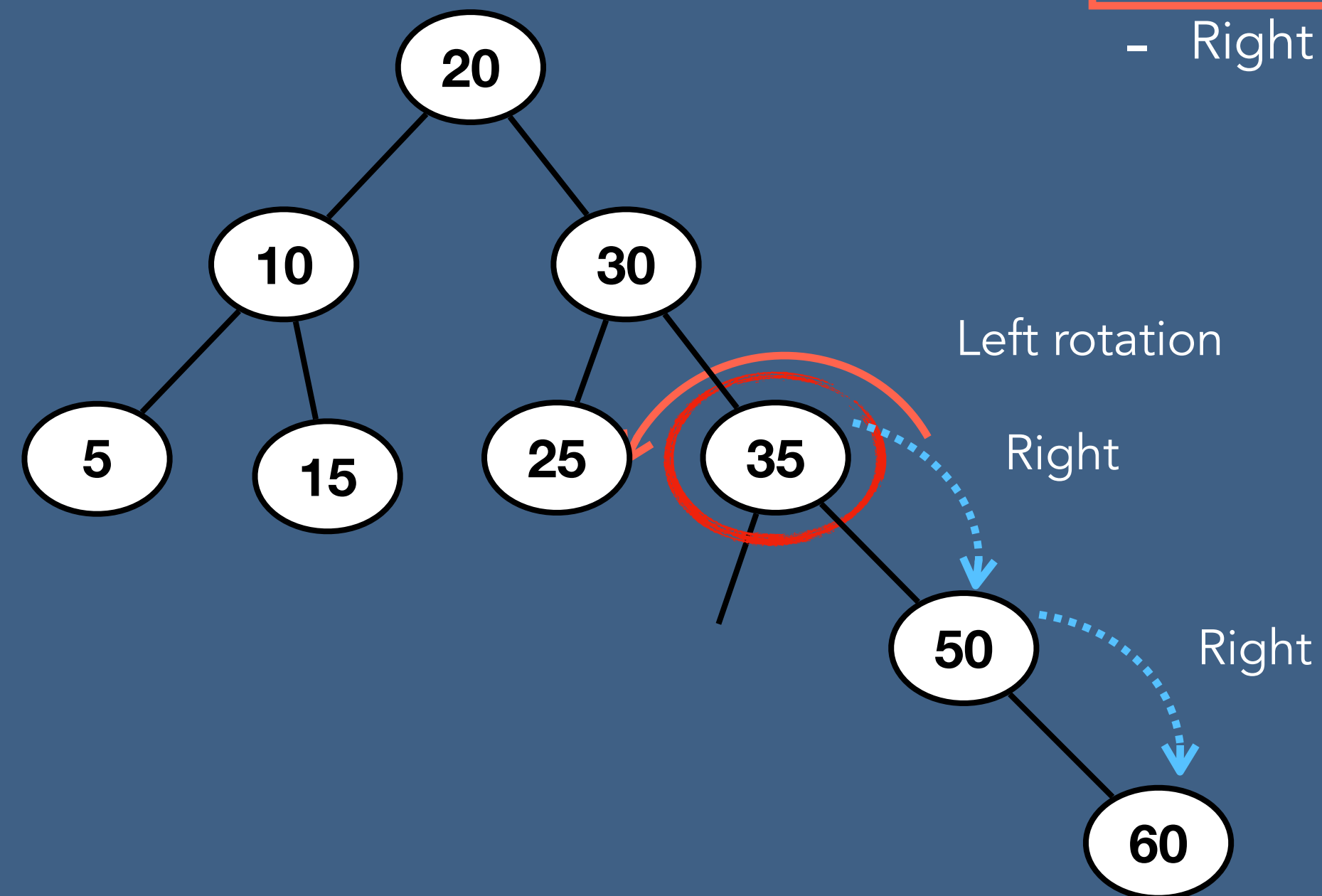
- Left Left condition (LL)
- Left Right condition (LR)
- Right Right condition (RR)
- Right Left condition (RL)



AVL Tree - Insert a Node (All together)

30,25,35,20,15,5,10,50,60,70,65

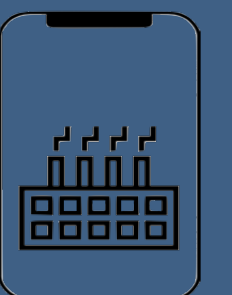
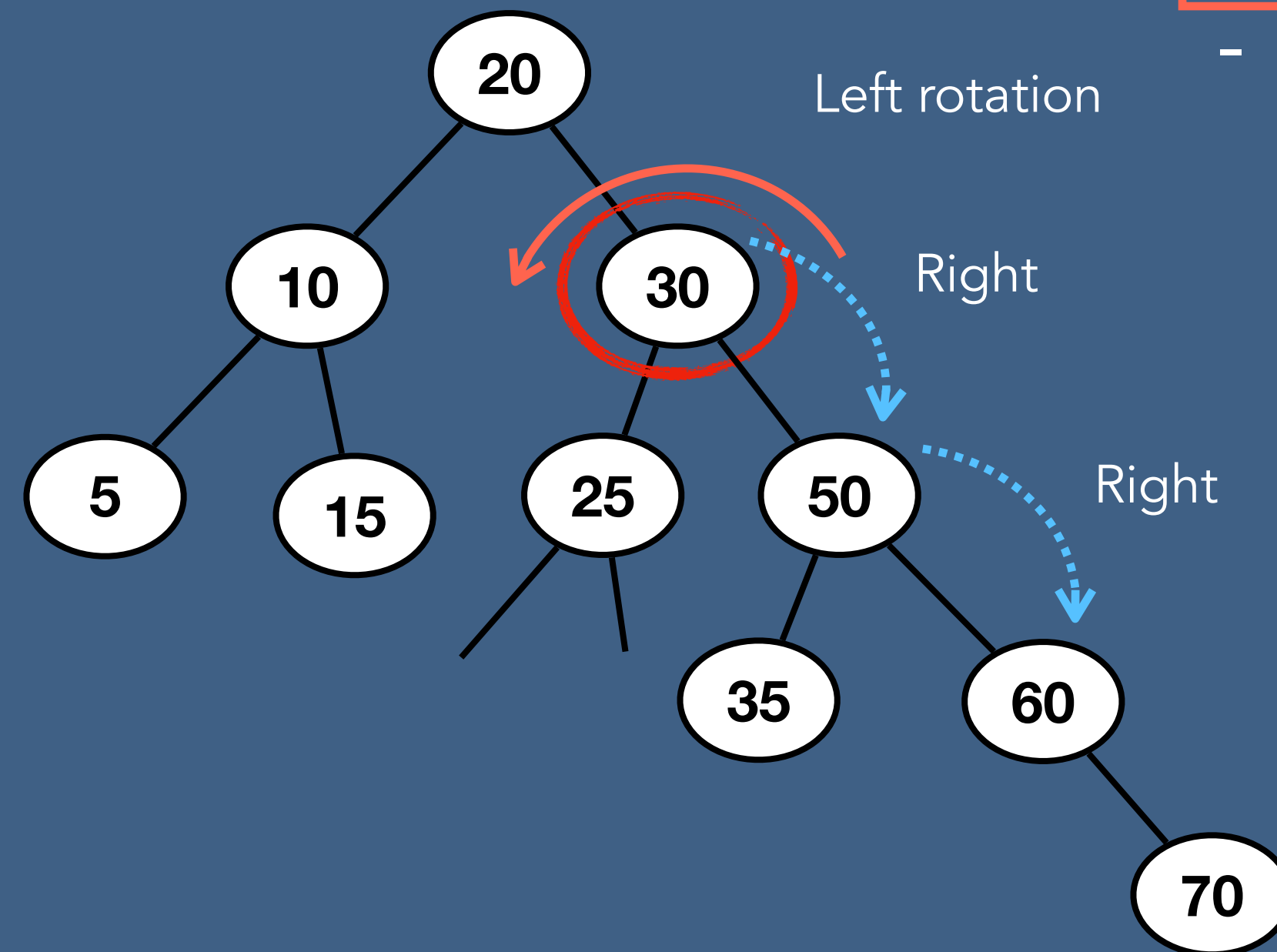
- Left Left condition (LL)
- Left Right condition (LR)
- Right Right condition (RR)
- Right Left condition (RL)



AVL Tree - Insert a Node (All together)

30,25,35,20,15,5,10,50,60,70,65

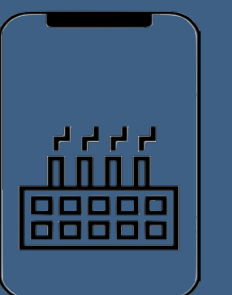
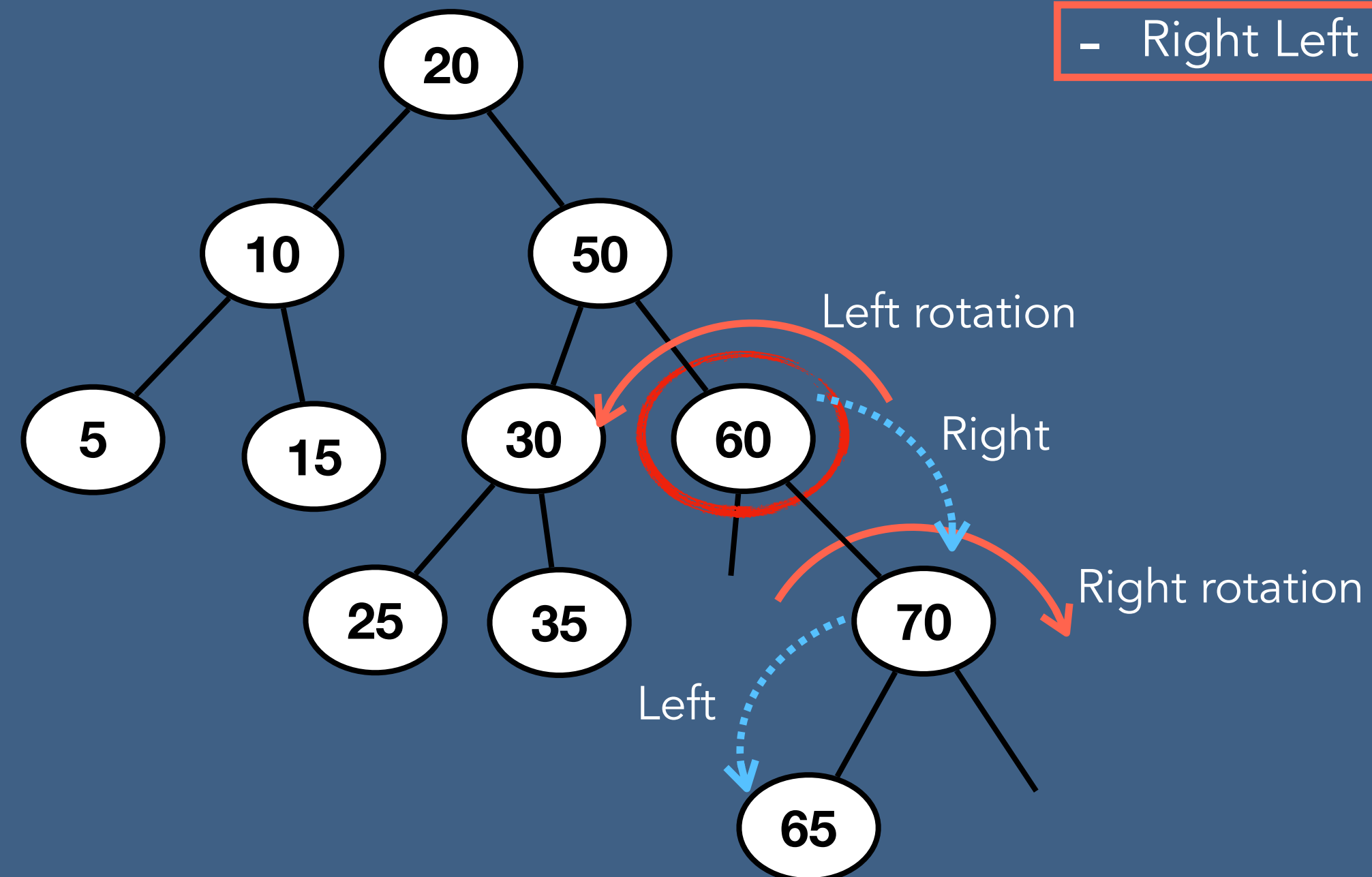
- Left Left condition (LL)
- Left Right condition (LR)
- Right Right condition (RR)
- Right Left condition (RL)



AVL Tree - Insert a Node (All together)

30,25,35,20,15,5,10,50,60,70,65

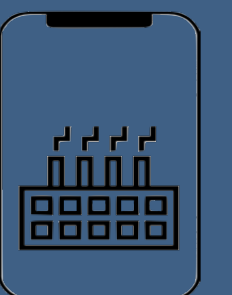
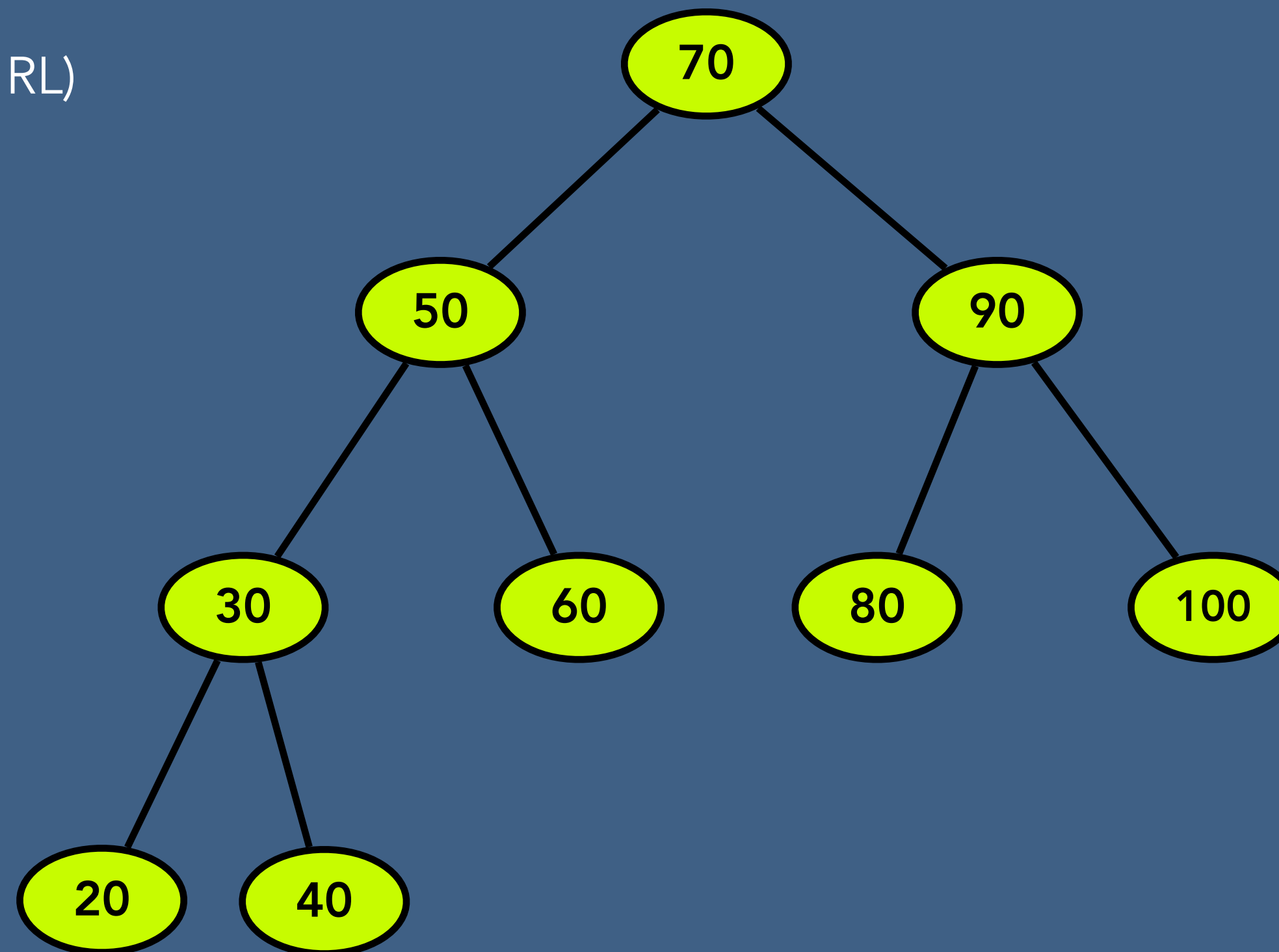
- Left Left condition (LL)
- Left Right condition (LR)
- Right Right condition (RR)
- Right Left condition (RL)



AVL Tree - Delete a Node

Case 1 - Rotation is not required

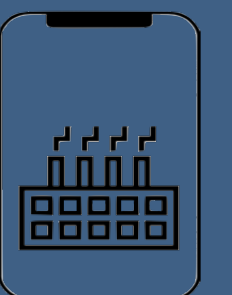
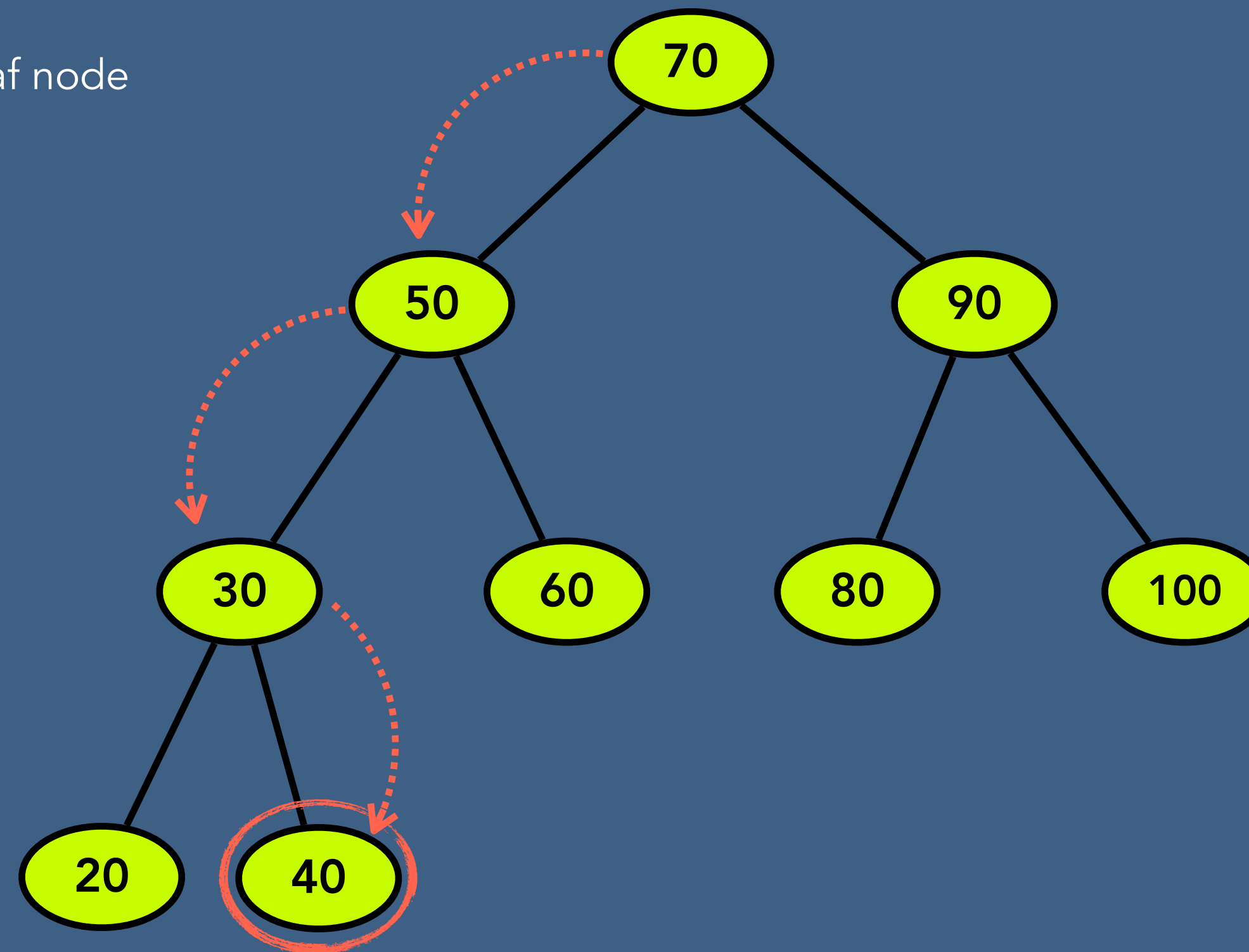
Case 2 - Rotation is required (LL, LR, RR, RL)



AVL Tree - Delete a Node

Case 1 - Rotation is not required

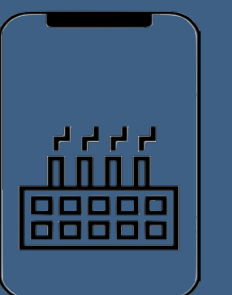
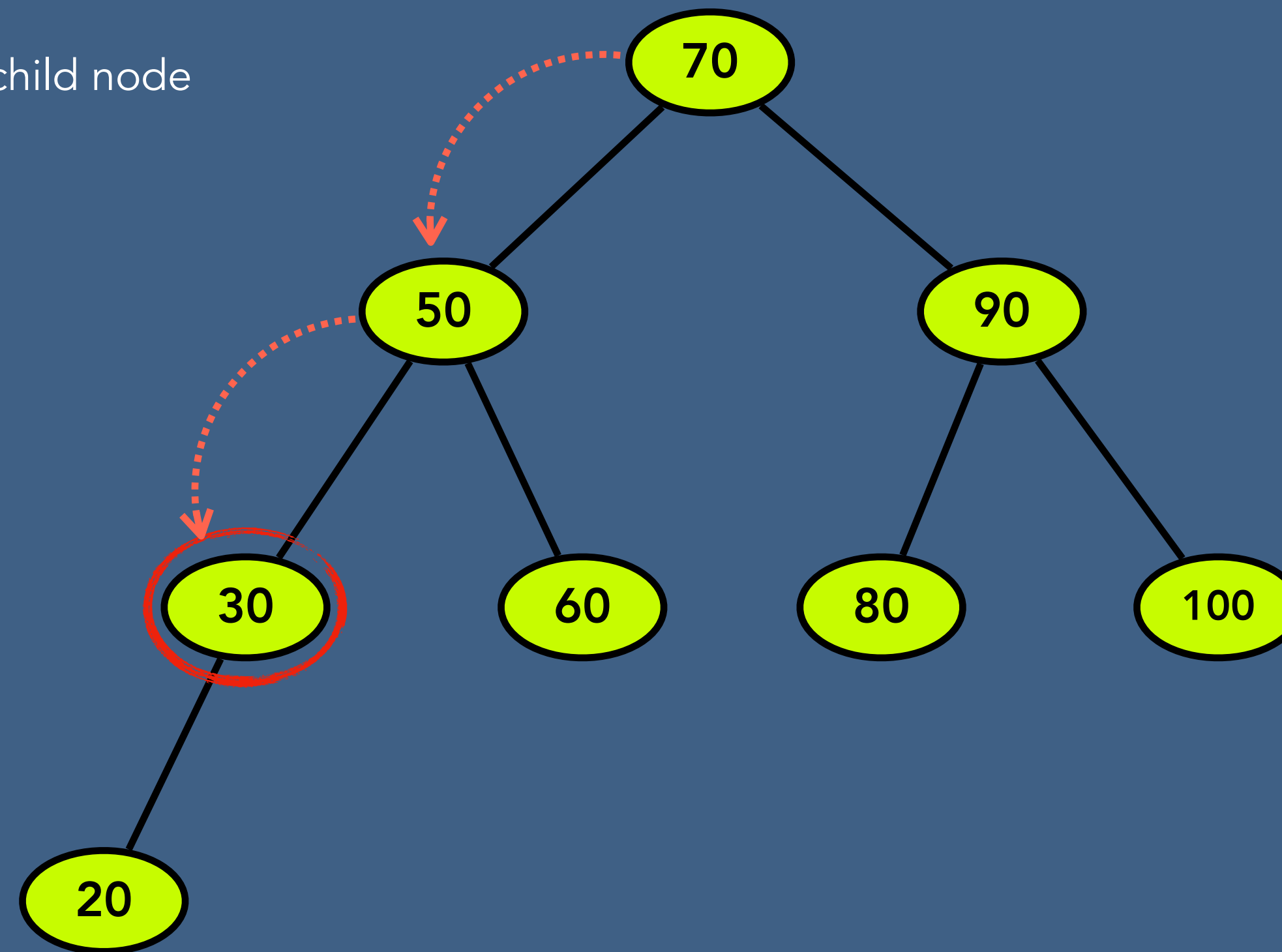
- The node to be deleted is a leaf node



AVL Tree - Delete a Node

Case 1 - Rotation is not required

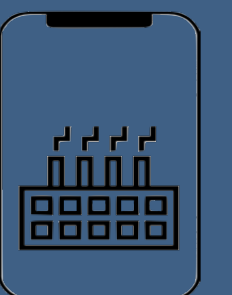
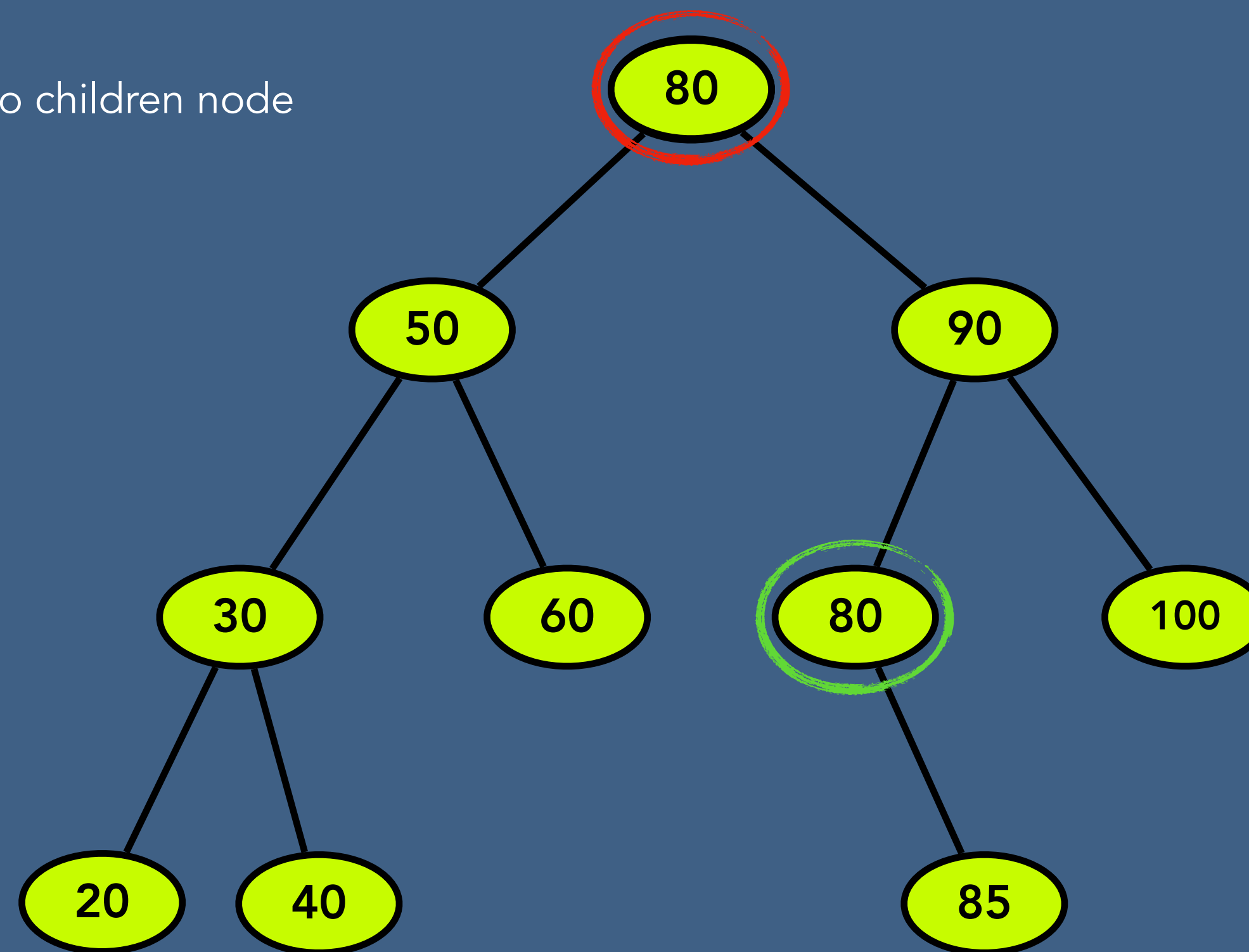
- The node to be deleted has a child node



AVL Tree - Delete a Node

Case 1 - Rotation is not required

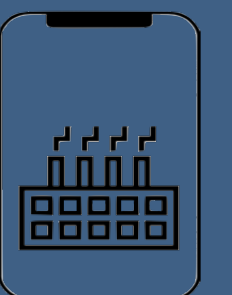
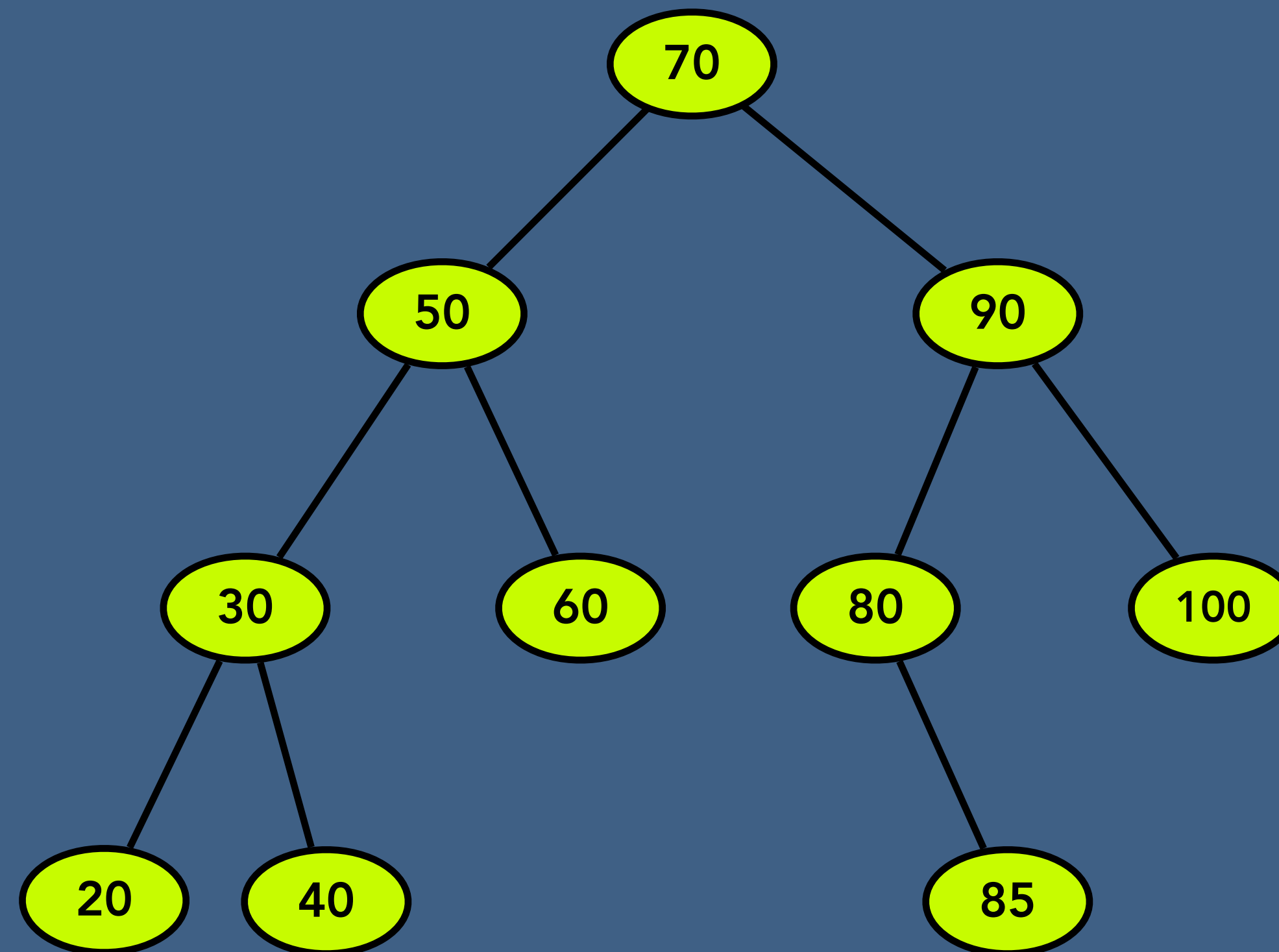
- The node to be deleted has two children node



AVL Tree - Delete a Node

Case 2 - Rotation is required

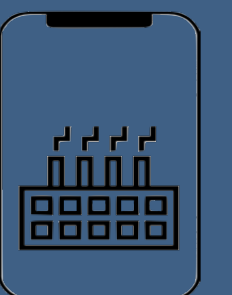
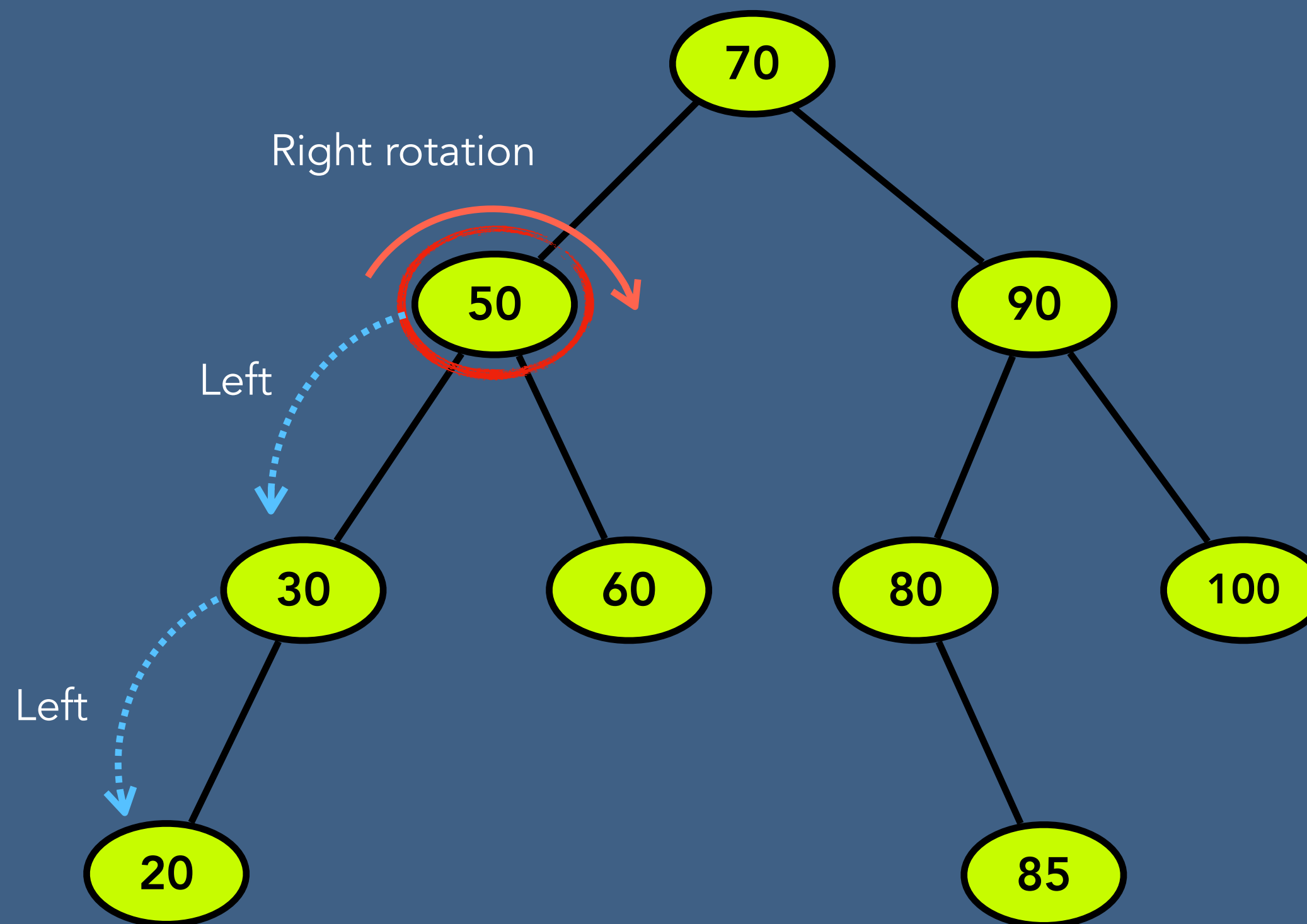
- Left Left Condition (LL)
- Left Right Condition (LR)
- Right Right Condition (RR)
- Right Left Condition (RL)



AVL Tree - Delete a Node

Case 2 - Rotation is required

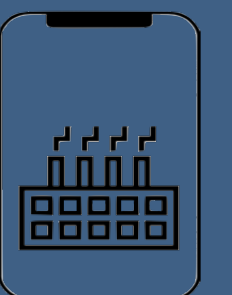
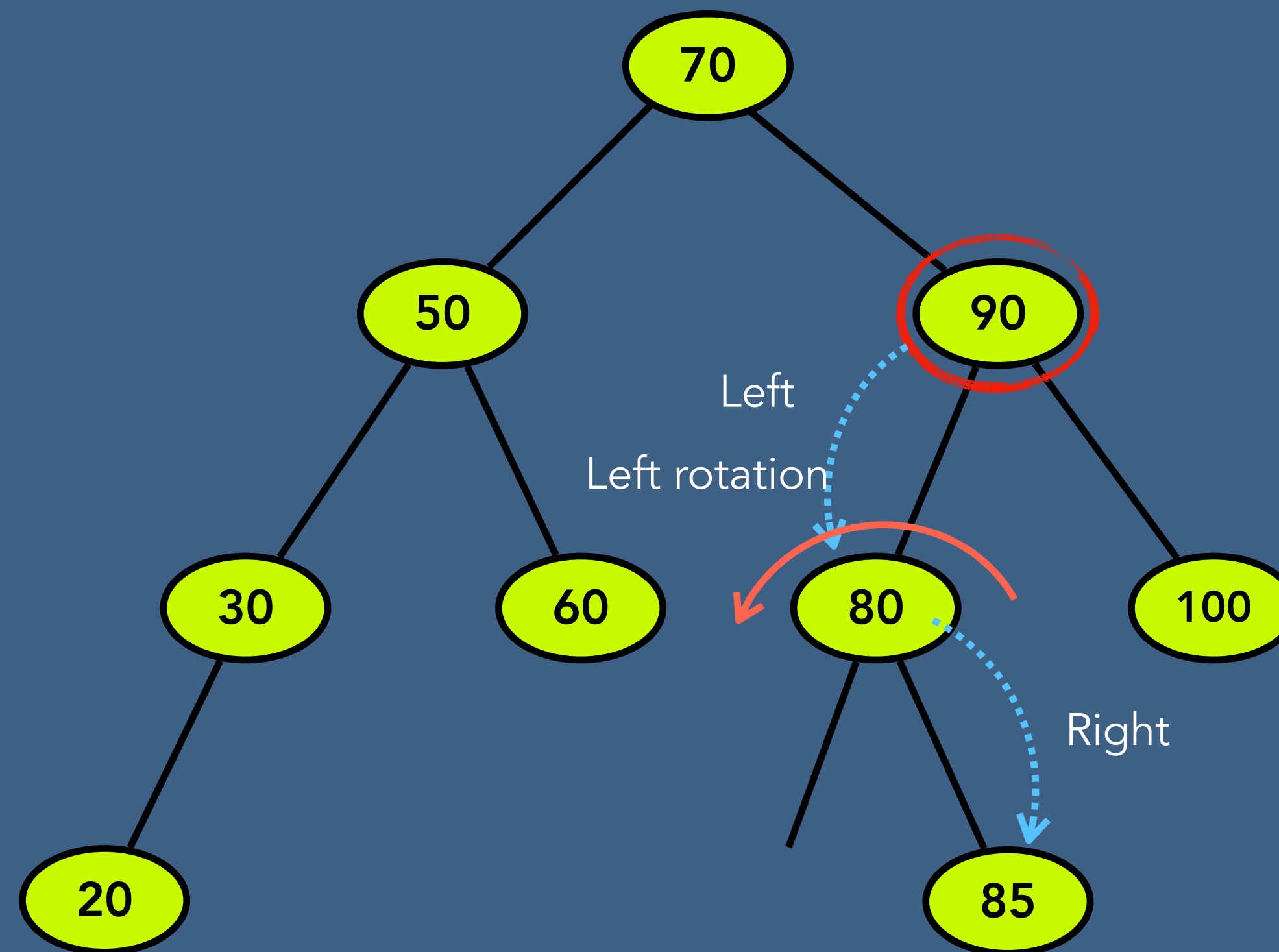
Left Left Condition (LL)



AVL Tree - Delete a Node

Case 2 - Rotation is required

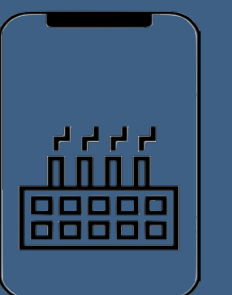
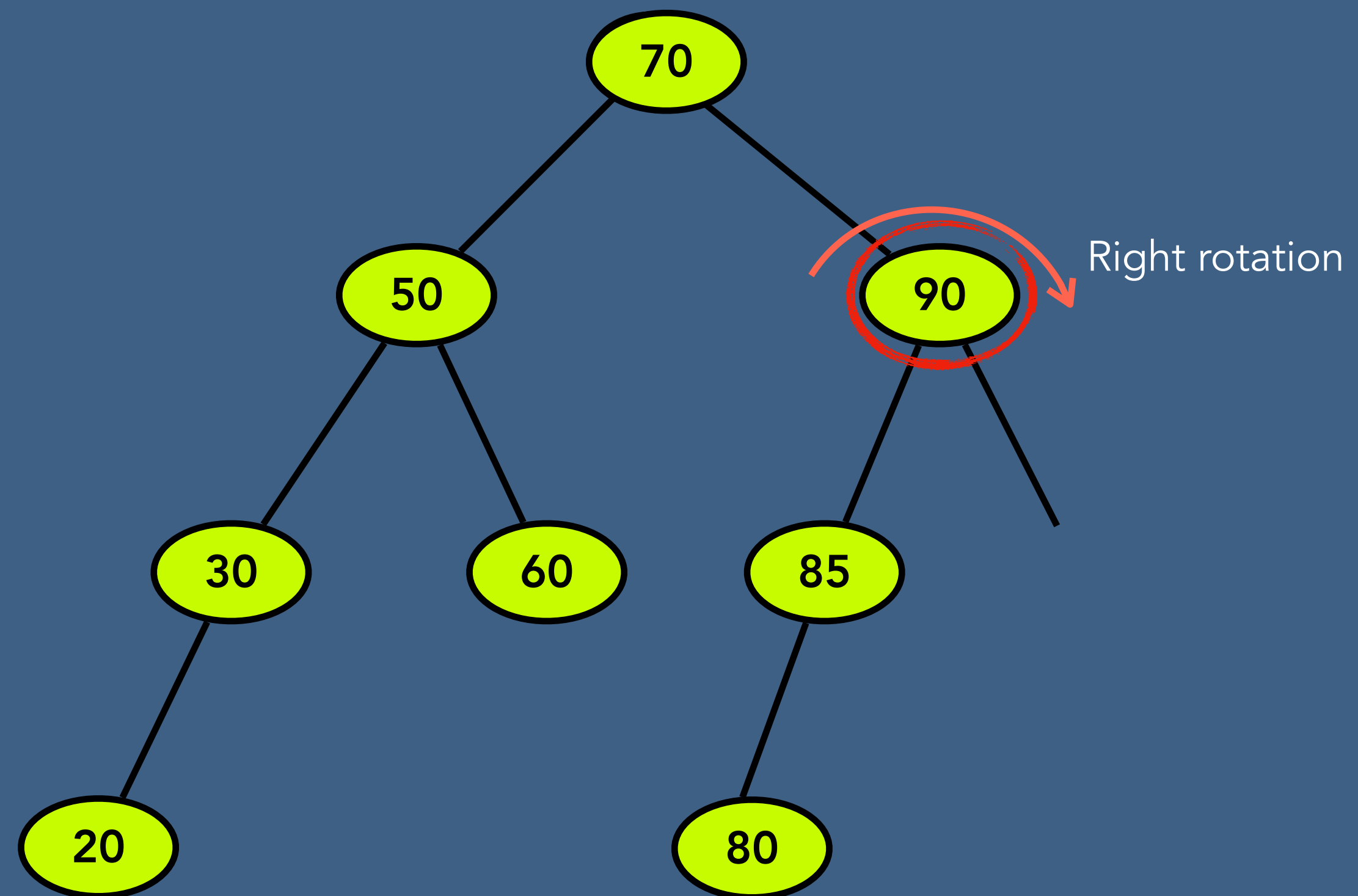
Left Right Condition (LR)



AVL Tree - Delete a Node

Case 2 - Rotation is required

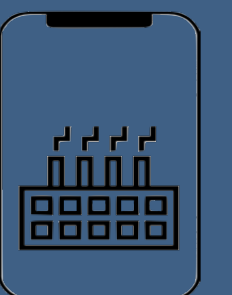
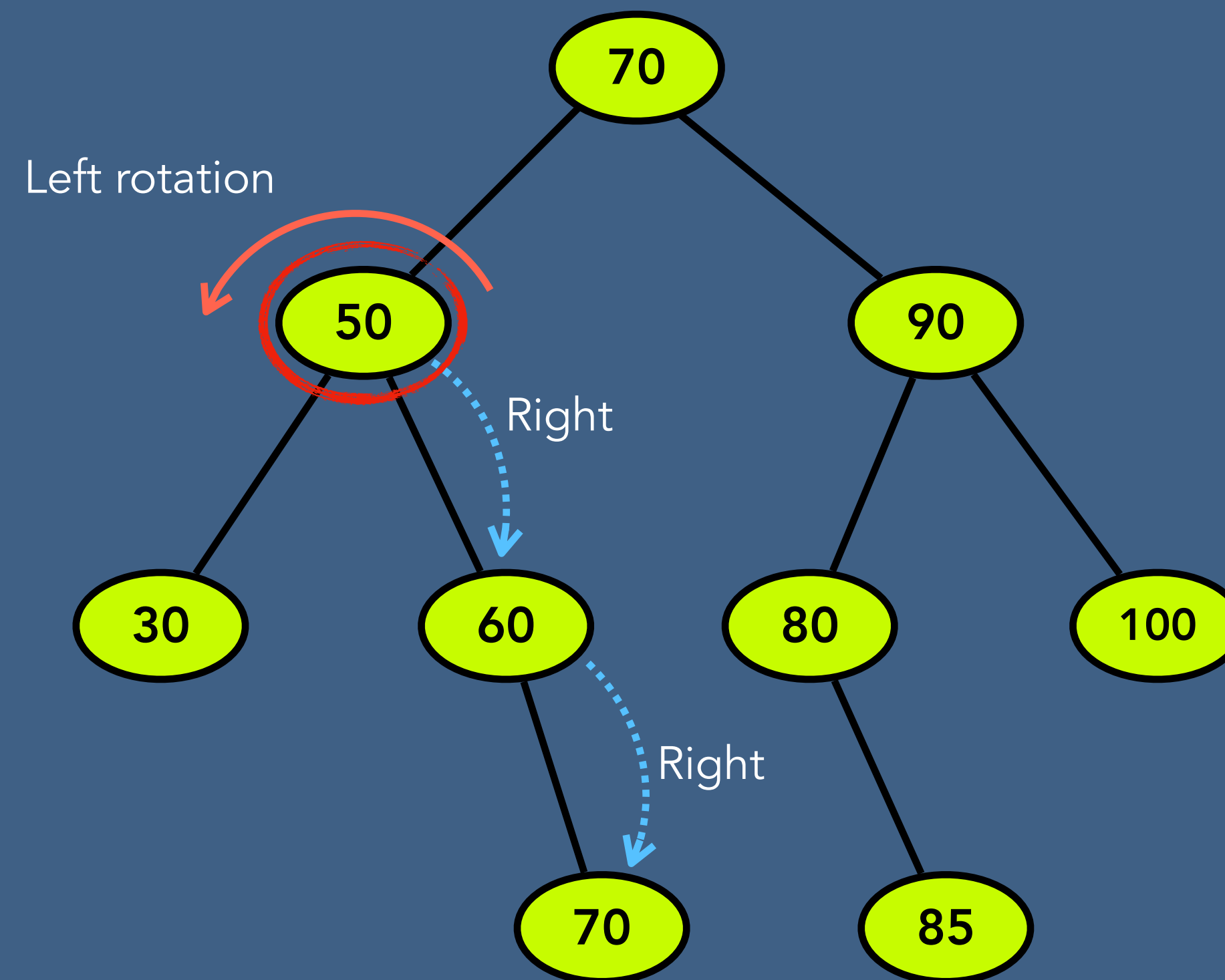
Left Right Condition (LR)



AVL Tree - Delete a Node

Case 2 - Rotation is required

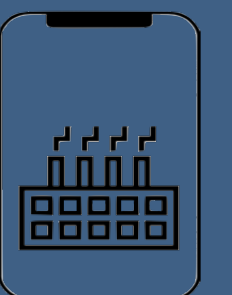
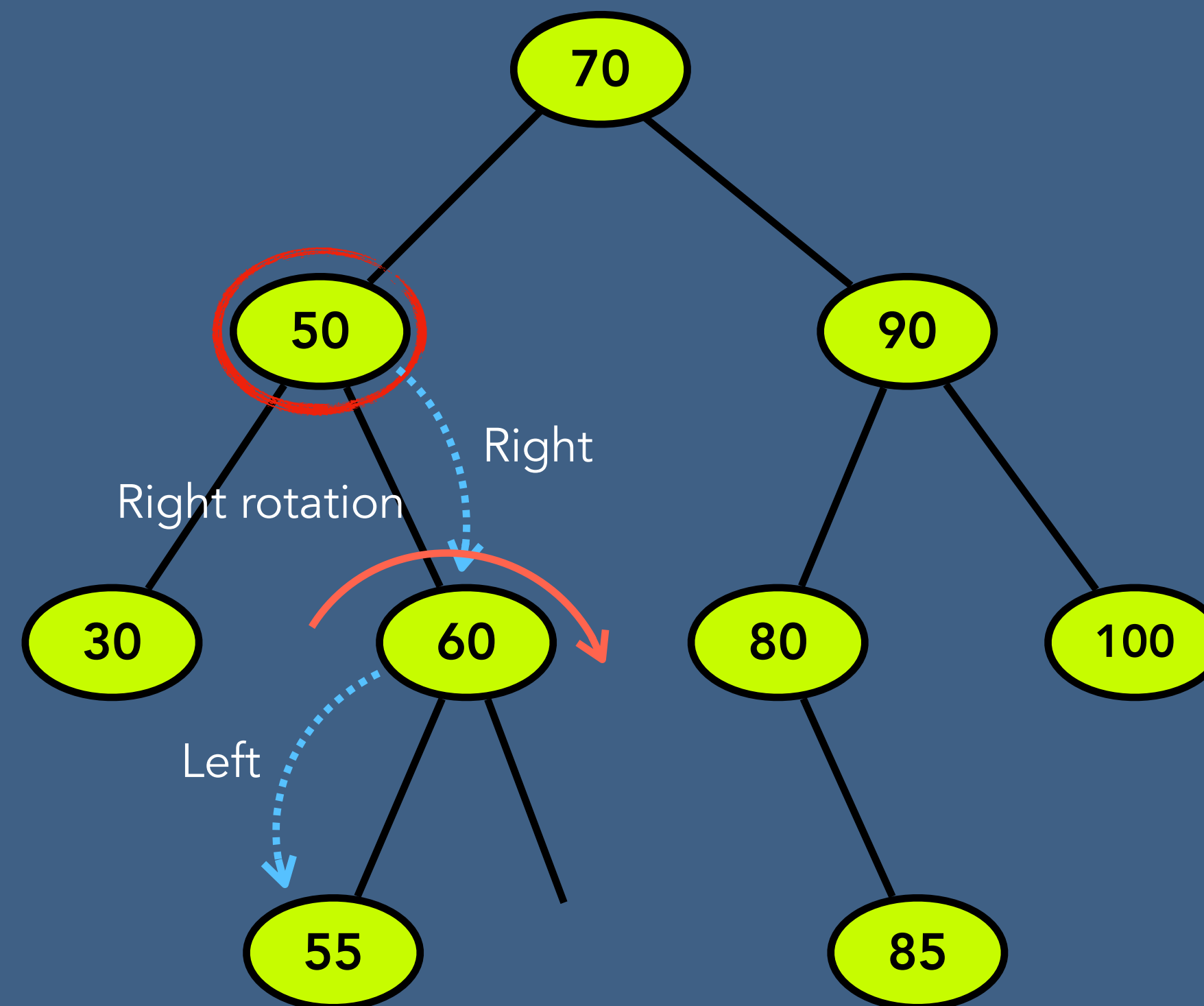
Right Right Condition (RR)



AVL Tree - Delete a Node

Case 2 - Rotation is required

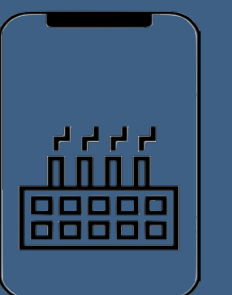
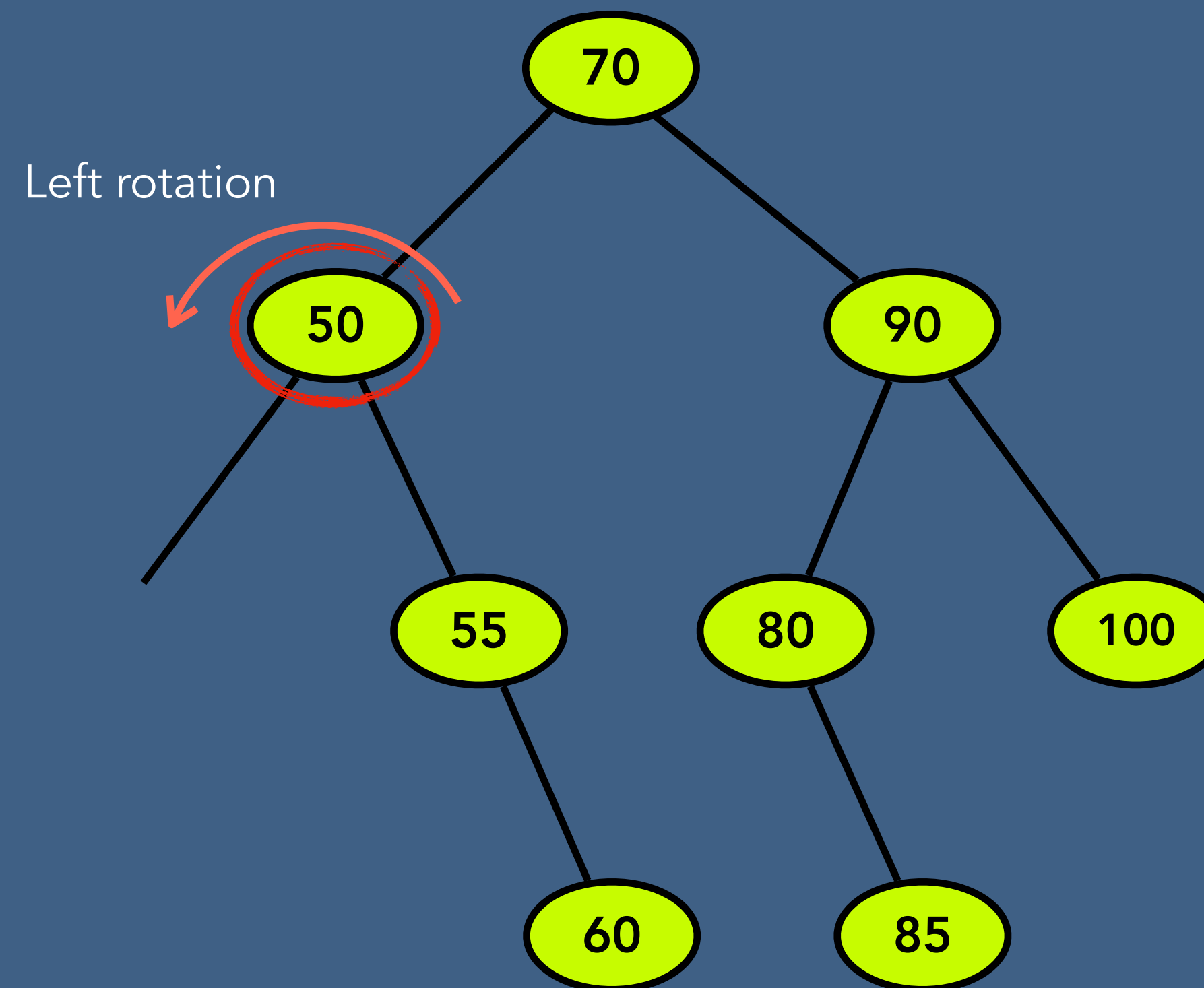
Right Left Condition (RL)



AVL Tree - Delete a Node

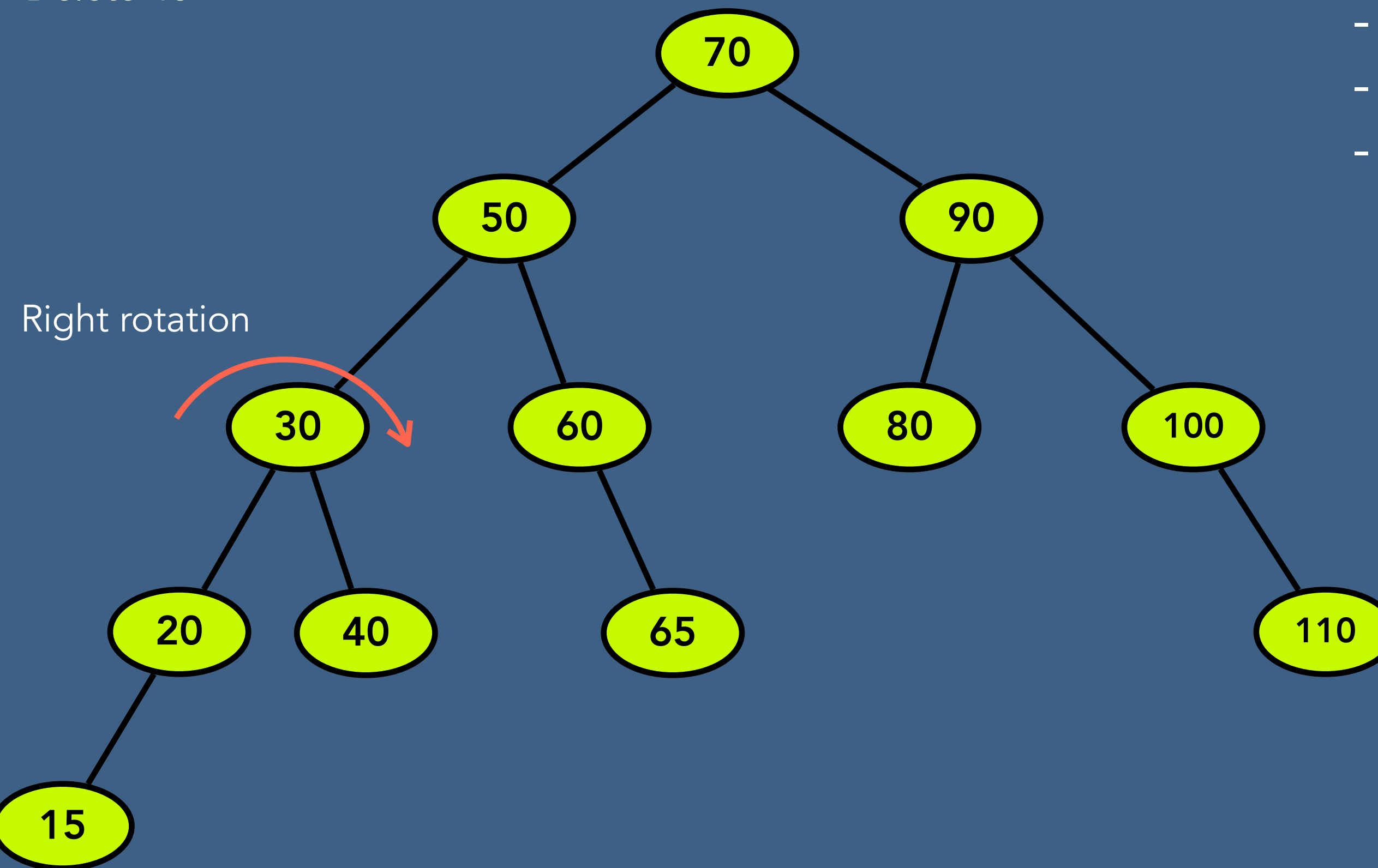
Case 2 - Rotation is required

Right Left Condition (RL)

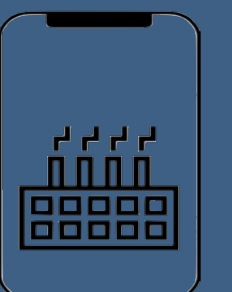


AVL Tree - Delete a Node (all together)

Delete 40



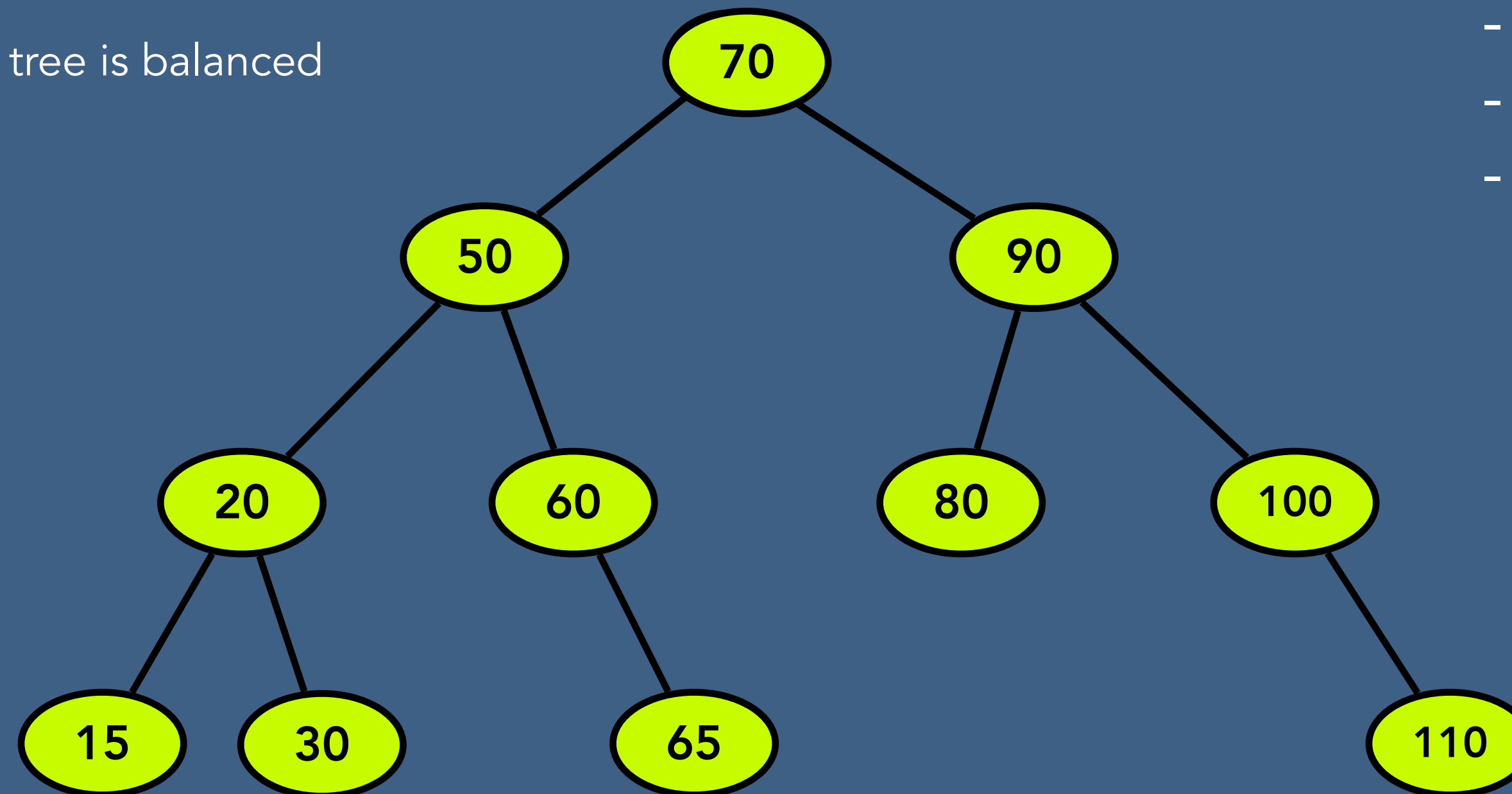
- Left Left Condition (LL)
- Left Right Condition (LR)
- Right Right Condition (RR)
- Right Left Condition (RL)



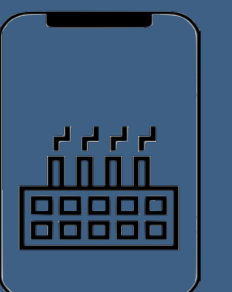
AVL Tree - Delete a Node (all together)

Delete 15

The tree is balanced



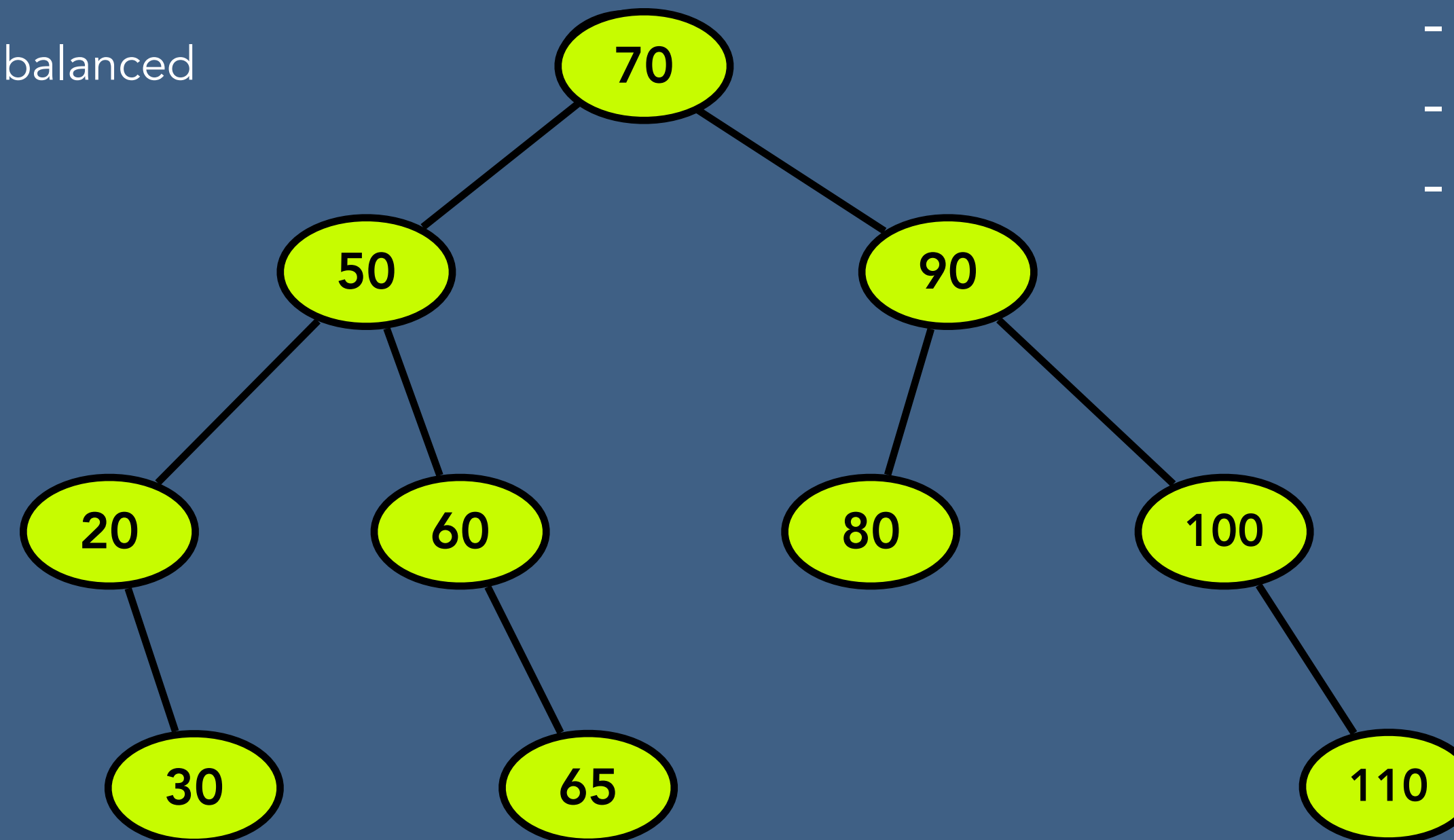
- Left Left Condition (LL)
- Left Right Condition (LR)
- Right Right Condition (RR)
- Right Left Condition (RL)



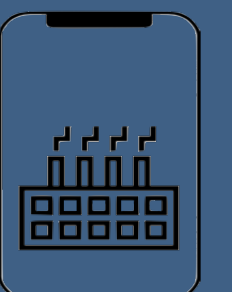
AVL Tree - Delete a Node (all together)

Delete 65

The tree is balanced



- Left Left Condition (LL)
- Left Right Condition (LR)
- Right Right Condition (RR)
- Right Left Condition (RL)



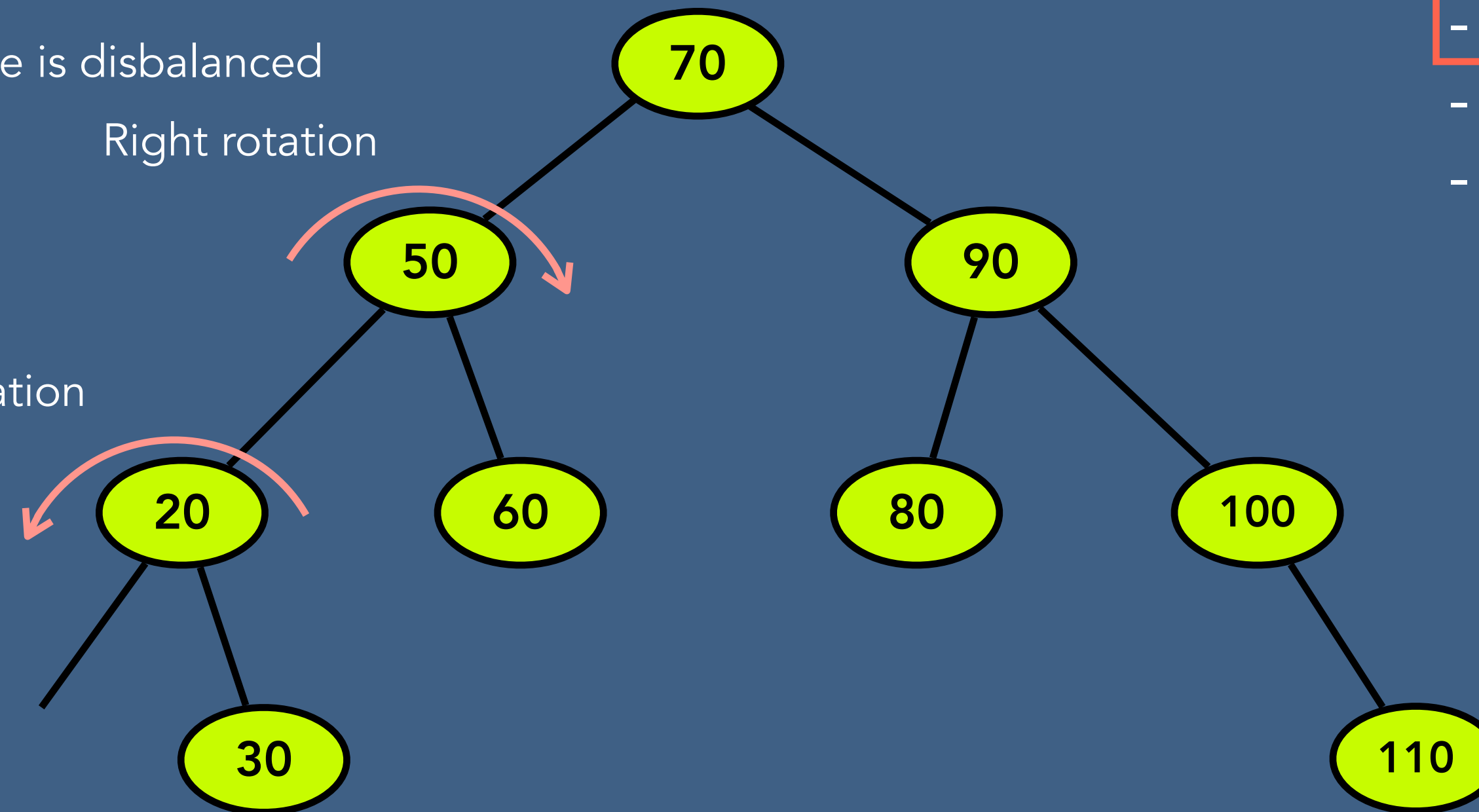
AVL Tree - Delete a Node (all together)

Delete 60

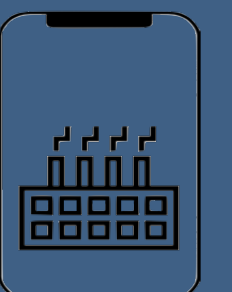
The tree is disbalanced

Right rotation

Left rotation



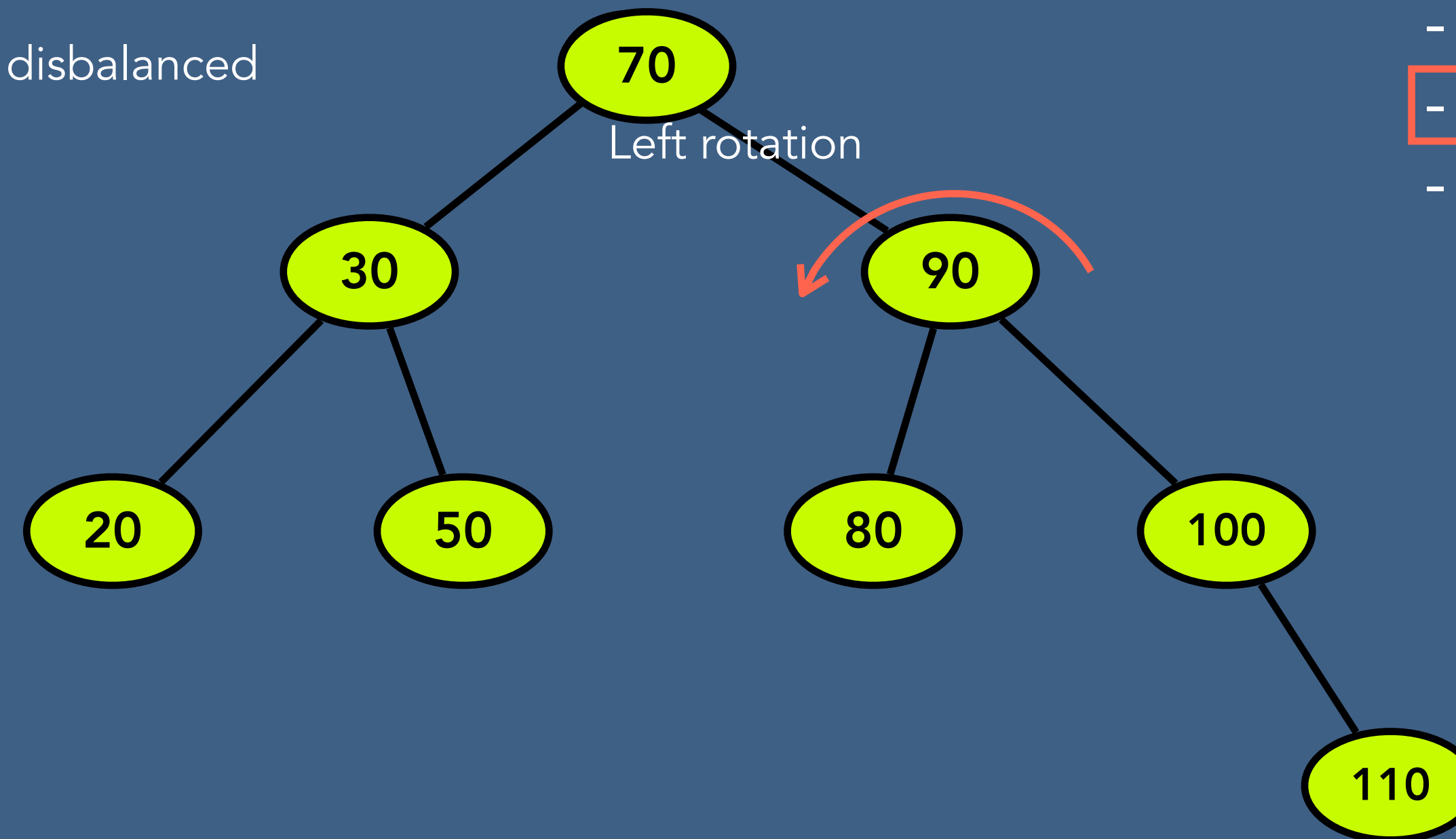
- Left Left Condition (LL)
- Left Right Condition (LR)
- Right Right Condition (RR)
- Right Left Condition (RL)



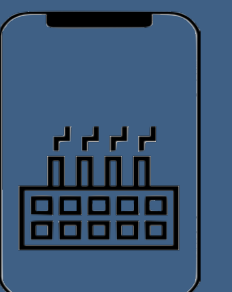
AVL Tree - Delete a Node (all together)

Delete 80

The tree is disbalanced

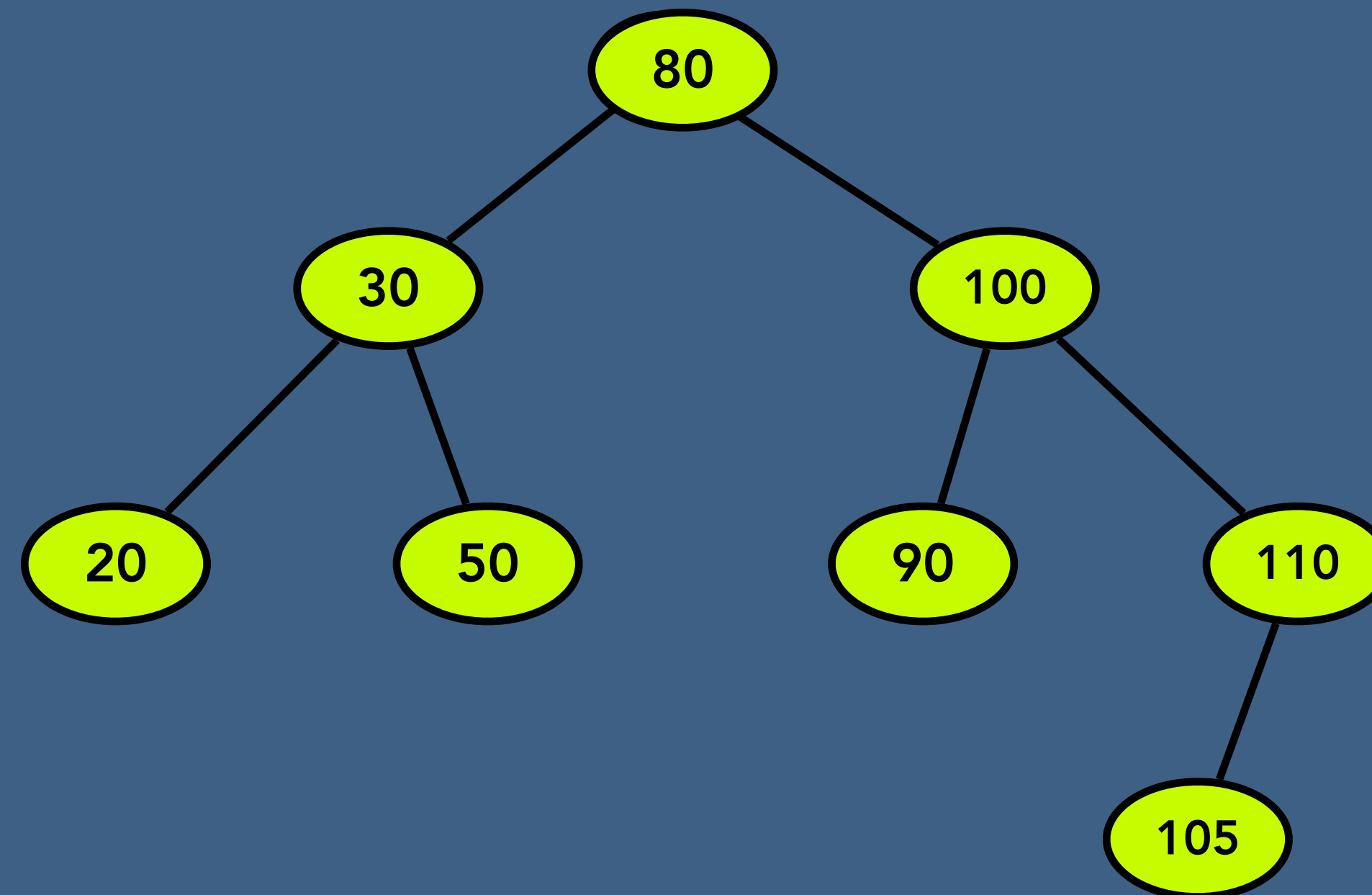


- Left Left Condition (LL)
- Left Right Condition (LR)
- Right Right Condition (RR)
- Right Left Condition (RL)

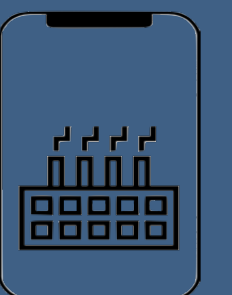


AVL Tree - Delete a Node (all together)

Insert 105

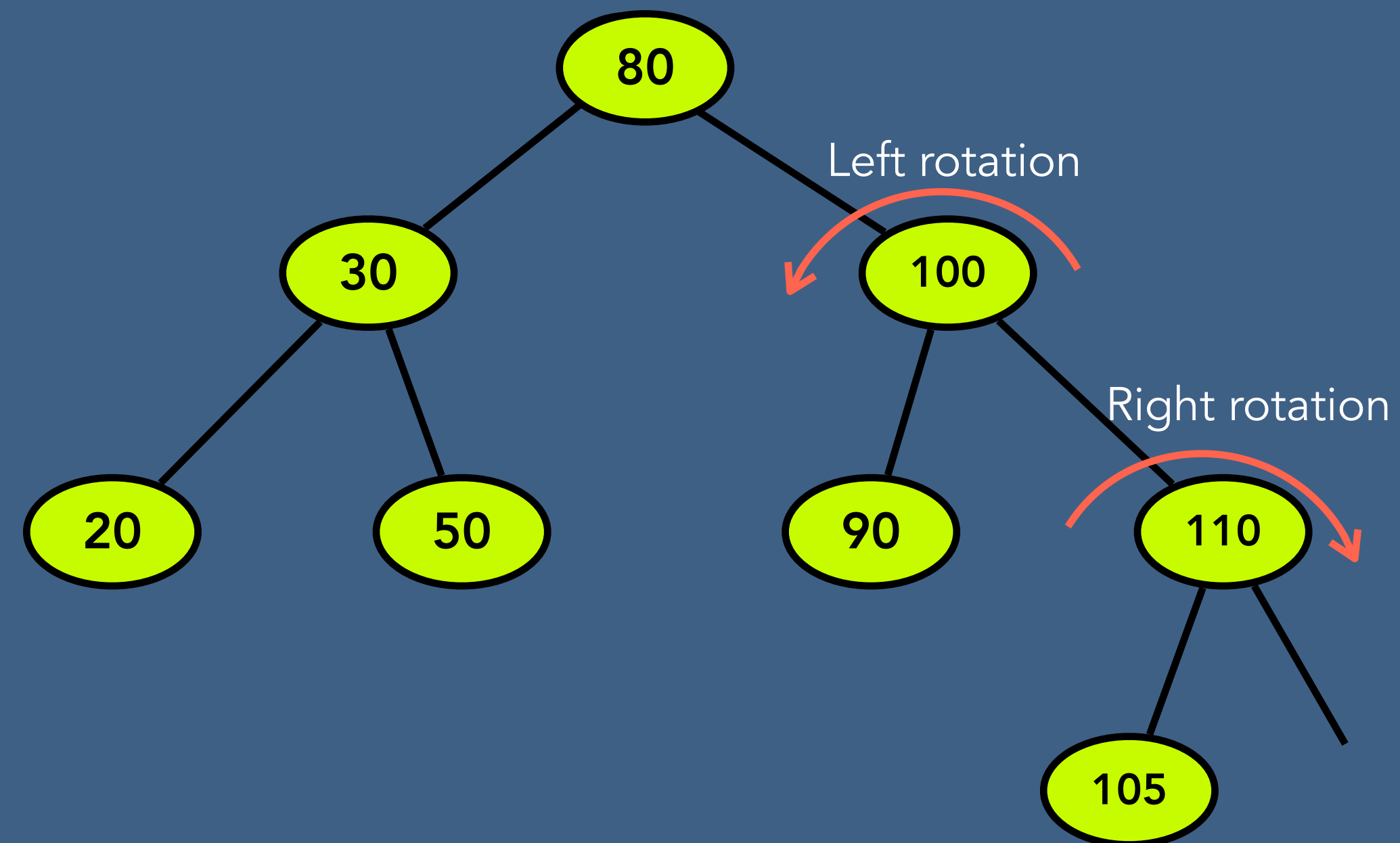


- Left Left Condition (LL)
- Left Right Condition (LR)
- Right Right Condition (RR)
- Right Left Condition (RL)

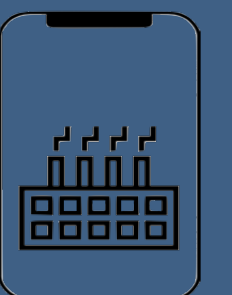


AVL Tree - Delete a Node (all together)

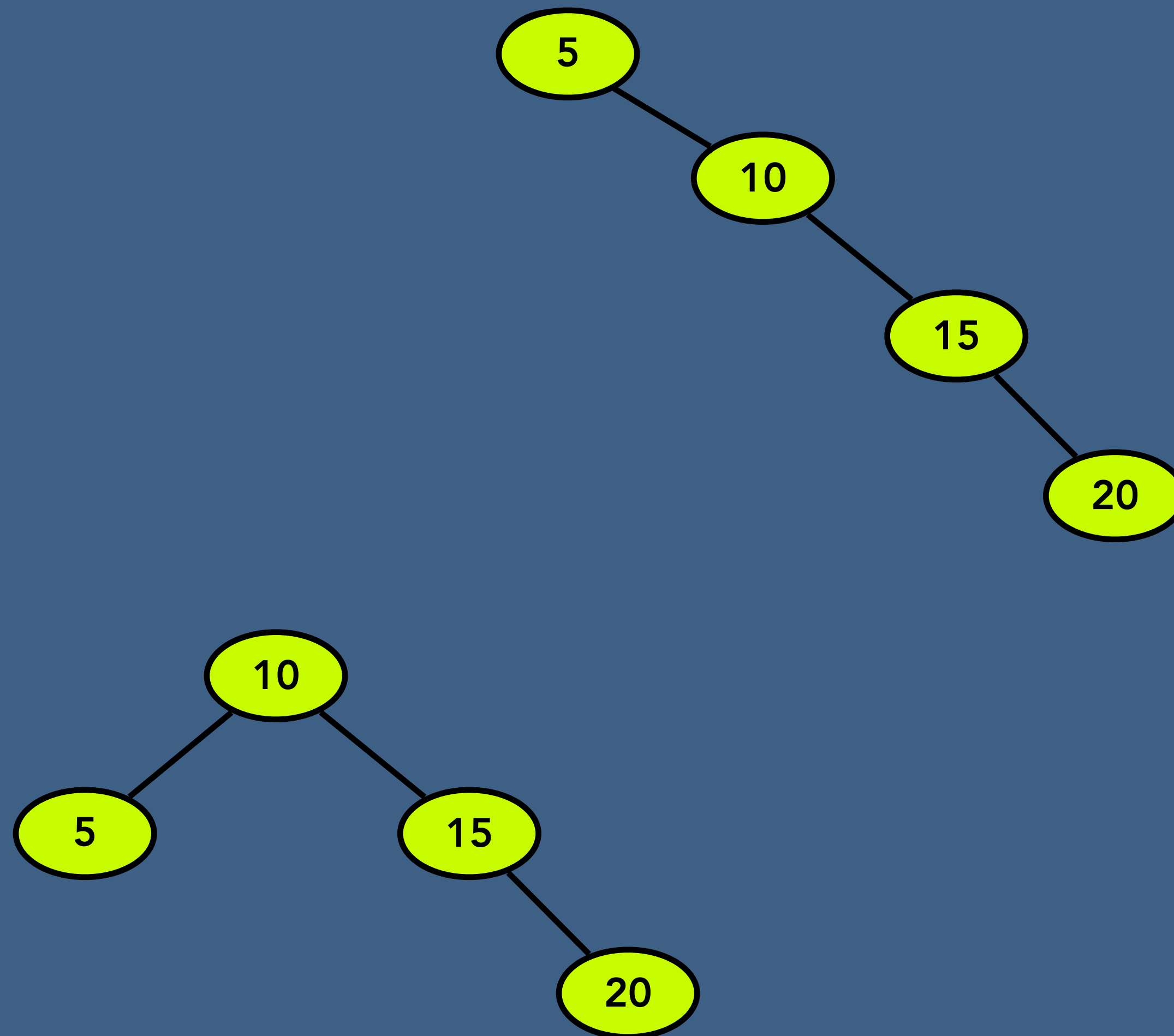
Delete 90



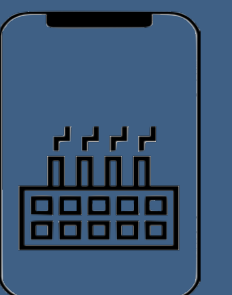
- Left Left Condition (LL)
- Left Right Condition (LR)
- Right Right Condition (RR)
- Right Left Condition (RL)



AVL Tree - Delete a Node (all together)

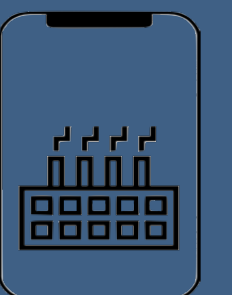
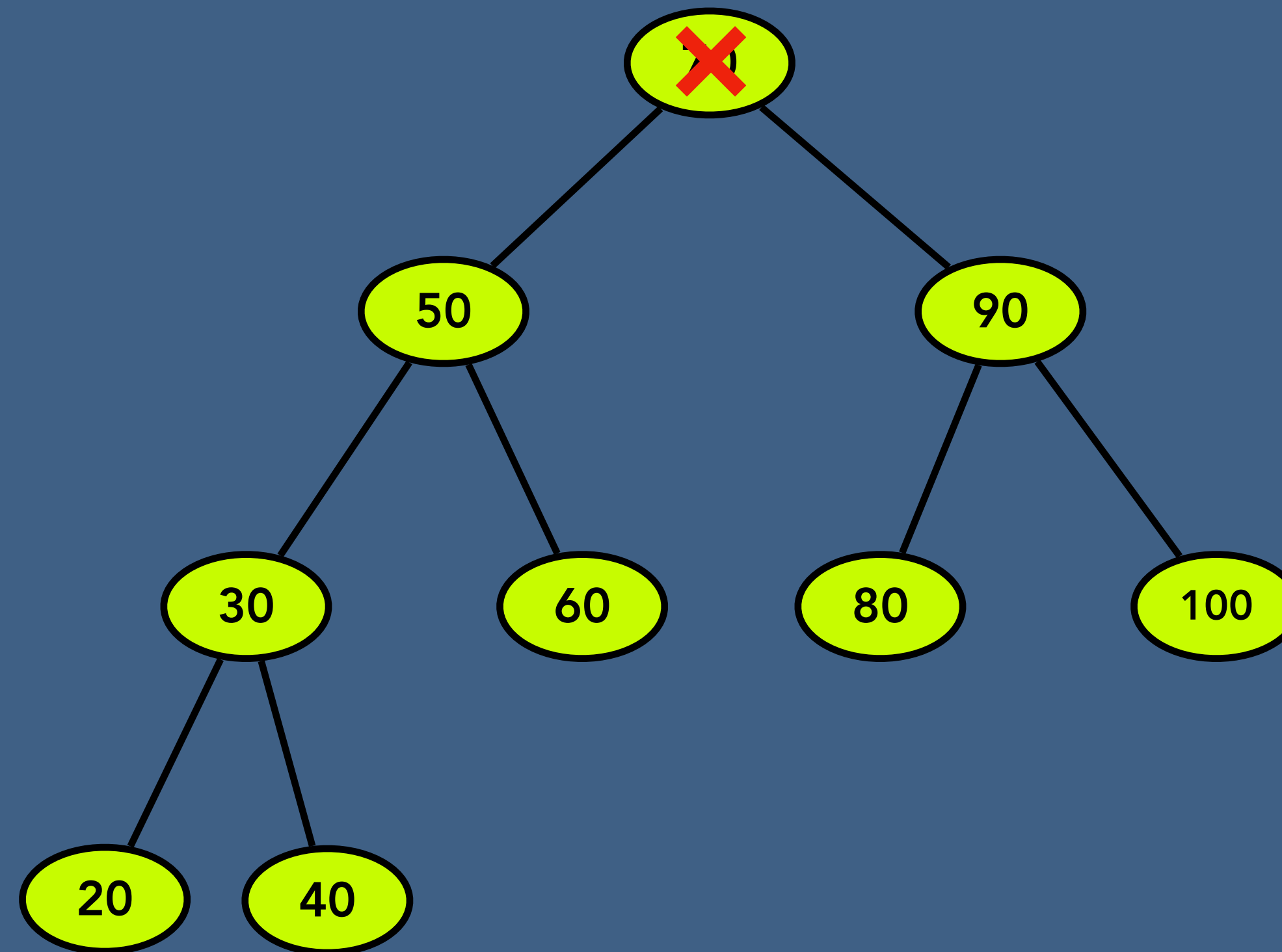


- Left Left Condition (LL)
- Left Right Condition (LR)
- Right Right Condition (RR)
- Right Left Condition (RL)



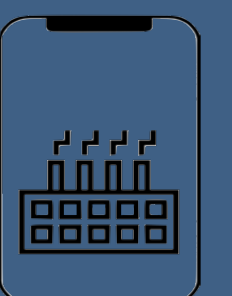
AVL Tree - Delete

```
rootNode = Null
```



Time and Space Complexity of AVL

	Time complexity	Space complexity
Create AVL	$O(1)$	$O(1)$
Insert a node AVL	$O(\log N)$	$O(\log N)$
Traverse AVL	$O(N)$	$O(N)$
Search for a node AVL	$O(\log N)$	$O(\log N)$
Delete node from AVL	$O(\log N)$	$O(\log N)$
Delete Entire AVL	$O(1)$	$O(1)$



Binary Search Tree vs AVL Tree

	BST	AVL
Create Tree	$O(1)$	$O(1)$
Insert a node Tree	$O(N)$	$O(\log N)$
Traverse Tree	$O(N)$	$O(N)$
Search for a node Tree	$O(N)$	$O(\log N)$
Delete node from Tree	$O(N)$	$O(\log N)$
Delete Entire Tree	$O(1)$	$O(1)$

